University of Cabuyao
(PAMANTASAN NG CABUYAO)

COLLEGE OF COMPUTING STUDIES

HORIZONTAL INTELLIGENCE: OPTIMIZING SMALL LANGUAGE MODEL REASONING VIA ROLE-DRIVEN MULTI-AGENT ORCHESTRATION

A Thesis

Submitted to the Faculty of

The College of Computing Studies

PAMANTASAN NG CABUYAO

City of Cabuyao, Laguna

In Partial Fulfillment

of the Requirements for the Degree:

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

By:

Balinado, Christian V.

Dejito, Andrew D.

Lopez, Joshua P.

Varde, Jericho F.

September 2026

CERTIFICATE OF ORIGINALITY AND AUTHENTICITY

PNC:PRE-FO-46 rev.0 03082023

Republic of the Philippines
Pamantasan ng Cabuyao
(UNIVERSITY OF CABUYAO)
Planning, Research, and Extension Division
Research and Development Department
Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

CERTIFICATION OF ORIGINALITY AND AUTHENTICITY

Research Title: **HORIZONTAL INTELLIGENCE: OPTIMIZING SMALL LANGUAGE MODEL REASONING VIA ROLE-DRIVEN MULTI-AGENT ORCHESTRATION**

Department: **COLLEGE OF COMPUTING STUDIES – BS COMPUTER SCIENCE**

I hereby declare that this submission is my own work, original, and authentic and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which, to a substantial extent, has been accepted for the award of any other degree or diploma of a university or other institute of higher learning, except where due acknowledgment is made in the text. The author takes full responsibility for the accuracy of the data and the interpretation of findings.

I hereby confirm that all the data collected, analyzed, and interpreted in this submission are original and of high quality.

I certify that I followed all ethical guidelines and research protocols, and that the research methodology used is valid and reliable.

I also declare that the intellectual content of this research is the product of my work, even though I may have received assistance from others on style, presentation, and language expression.

Pamantasan ng Cabuyao (University of Cabuyao) is hereby granted the right to publish the research work, either in full or in part, in any academic or scientific publication.

CHRISTIAN V. BALINADO
Signature over printed name of Researcher
Date: 09-23-2026

ANDREW D. DEJITO
Signature over printed name of Researcher
Date: 09-23-2026

JOSHUA P. LOPEZ
Signature over printed name of Researcher
Date: 09-23-26

JERICHO F. VARDE
Signature over printed name of Researcher
Date: 09-23-26

ASST. PROF. JANUS RAYMOND C. TAN
Signature over printed name of Research Adviser
Date: 9/23/26

Dangal ng Bayan.
bringing pride and honor to the nation.

ABSTRACT

Title	:	Horizontal Intelligence: Optimizing Small Language Model Reasoning Via Role-Driven Multi-Agent Orchestration
Researchers	:	Balinado, Christian V. Dejito, Andrew D. Lopez, Joshua P. Varde, Jericho F.
Degree Program	:	Bachelor of Science in Computer Science
Adviser	:	Asst. Prof. Janus Raymond C. Tan
School	:	University of Cabuyao

RECOMMENDATION FORM

PNC:PRE-FO-71 rev.0 03082023

Republic of the Philippines
Pamantasan ng Cabuyao
(UNIVERSITY OF CABUYAO)
Planning, Research, and Extension Division
Research and Development Department
Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

STUDENT RESEARCH RECOMMENDATION FORM

Group No.	**CSA6**	Date Filed:	**SEPTEMBER 23, 2026**	
Researchers:	**CHRISTIAN V. BALINADO**	**ANDREW D. DEJITO**	**JOSHUA P. LOPEZ**	**JERICHO F. VARDE**
Research Title:	**HORIZONTAL INTELLIGENCE: OPTIMIZING SMALL LANGUAGE MODEL REASONING VIA ROLE-DRIVEN MULTI-AGENT ORCHESTRATION**			

This research has been thoroughly examined and is now recommended for Final Defense.

ASST. PROF. JANUS RAYMOND C. TAN
Adviser

TABLE OF CONTENTS

Title Page		i
Certificate of Originality and Authenticity		ii
Abstract		Iii
Recommendation Letter		iv
Table of Contents		v
List of Tables		viii
List of Figures		x
List of Appendices		xii
CHAPTER		**Page**
I	**THE PROBLEM AND ITS SETTING**	
	Introduction	1
	Statement of the Problem	2
	Scope and Limitation	4
	Significance of the Study	6
II	**REVIEW OF RELATED LITERATURE AND STUDIES**	
	Conceptual Literature	8
	Standard Features	8
	Guidelines for Using the Algorithm	11
	User-Centric Design	13
	Testing Tools and Procedure	14
	Research Literature	15
	Review of Related Literature	15
	Horizontal Intelligence and Agentic Decomposition	15
	Neuro-Symbolic Governance and Deterministic Safety Nets	17
	Algorithmic Resource Scheduling and Mutual Exclusion	19
	Frameworks for Transparency and Trust in AI	20

Review of Related Studies 22
Multi-Agent LLM Systems for Code Generation and Review 22
Comparative Success Rates of Multi-Agent vs. Monolithic Pipelines 23
Efficacy of Hybrid Deterministic-Neural Repair Loops 25
Computational Latency and VRAM Profiling in Edge Inference 27
Theoretical Background 31
Conceptual Framework 35
Synthesis 37
Definition of Terms 41

III **RESEARCH METHODOLOGY**
Research Design 47
Research Locale 48
Respondents of the Study 49
Data Gathering Procedure 50
Instrumentation 50
Validation 51
Discussions on Algorithms/Mathematical Concepts Used/Proposed Solutions 52
Fundamentals of the Algorithms/Mathematical Model/Formula 52
Theoretical Foundations 54
Algorithm Design 56
Mathematical Model Formulation 60
Algorithm Implementation 64
Complexity Analysis 68
System Development Methodology 69
Statistical Treatment of Data 85
Population and Sampling 85

Evaluation and Scoring 86
Ethical Considerations 92

IV RESULTS AND DISCUSSION
System Features in Automating Java Code Refactoring 93
Multi-Agent Parameter Distribution 94
Deterministic Validation Interception Rate 96
Iterative Feedback Resolution Rate 99
Refactoring Performance on the CodeEditorBench Dataset 104
Compilation Success Rate (CSR) 104
Behavioral Equivalence Rate (BER) 106
Cyclomatic Complexity 109
Maintainability Index (MI) Improvement 111
Computational Efficiency of HorizonAI During Local Execution 114
Inference Time 115
Peak GPU Memory Usage 119
Assessment of the End Users 122
Assessment of IT Experts 134
Implementation of the Multi-Agent Orchestration Architecture 145

V SUMMARY OF FINDINGS, CONCLUSIONS, AND RECOMMENDATIONS
Summary of Significant Findings 158
Conclusions 160
Recommendations 161
Literature Cited 164

LIST OF TABLES

Table		Page
1	Summary of related studies and research findings	30
2	Distribution of respondents by classification	50
3	Variable and Parameter Definition of AST Parsing	61
4	Variable and Parameter Definition of Cyclomatic Complexity	62
5	Variable and Parameter Definition of Intent and Plan Generation	62
6	Variable and Parameter Definition of Candidate Code Generation	63
7	Variable and Parameter Definition of Judge Adjudication	64
8	Variable and Parameter Definition of Bounded Optimization	64
9	Complexity Analysis of the Proposed System's Orchestration	69
10	User-Centric Evaluation Criteria (Glassbox Interface)	89
11	Technical Expert Evaluation Criteria (Product Quality)	90
12	Response Scale and Mean Interpretation	91
13	Parameter Distribution of HorizonAI and the Single-Model Baseline	94
14	Validation Interception Rates of HorizonAI by Tier	97
15	Iterative Feedback Resolution Results of HorizonAI	100
16	Compilation Success Rate of the Single-Model Baseline and HorizonAI	104
17	Behavioral Equivalence Rate of Compiled Outputs	107
18	Mean Cyclomatic Complexity of Compiled Outputs	109
19	Mean Maintainability Index of Compiled Outputs	112
20	Inference Time Statistics of HorizonAI and the Single-Model Baseline	115
21	Normality and Wilcoxon Signed-Rank Test Results for Paired Inference Time	118
22	Peak GPU Memory Usage of HorizonAI and the Single-Model Baseline	119

23	Normality and Wilcoxon Signed-Rank Test Results for Paired Break GPU Memory	121
24	End Users' Evaluation in Terms of Interaction Capability	125
25	End Users' Evaluation in Terms of Functional Suitability	128
26	End Users' Evaluation in Terms of Security	130
27	End Users' Evaluation in Terms of Reliability	132
28	Overall End-User Acceptability of the HorizonAI Glassbox Interface	133
29	IT Experts’ Evaluation in Terms of Functional Suitability	136
30	IT Experts’ Evaluation in Terms of Performance Efficiency	138
31	IT Experts’ Evaluation in Terms of Reliability	140
32	IT Experts’ Evaluation in Terms of Maintainability	143
33	Overall IT Expert Evaluation of the HorizonAI System	144
34	Phase 4 Validation Tiers, Checks, and Failure Responses	156

LIST OF FIGURES

Figure		Page
1	Distributed Cognition Model Applied to the Multi-Agent Java Code Refactoring System	31
2	Human-AI Collaboration Framework Applied to the Hybrid Multi-Agent Refactoring System	32
3	Constructivist Learning Framework Applied to the Glassbox Interface	33
4	Conceptual Framework of the Study	35
5	Algorithmic Workflow Flowchart	60
6	Strict Sequential Inference Scheduling from AgentService Class	66
7	Sample Execution Trace of HorizonAI	67
8	Software Development Lifecycle of the Developed System	70
9	System Use Case Diagram	72
10	Activity Diagram for Starting a New Session	74
11	Activity Diagram for Running Refactoring Pipeline	75
12	Activity Diagram for Renaming a Session	77
13	Activity Diagram for Viewing Previous Session	78
14	Activity Diagram for Deleting a Session	79
15	Activity Diagram for Executing an Agent Task	80
16	System Architecture Diagram	81
17	System Component Diagram	83
18	Sequence Diagram of Module Interactions	84
19	Parameter and Agent Role Distribution of the Single-Model Baseline and HorizonAI	95
20	Actual HorizonAI Interface Showing a Validation Interception During Refactoring	98
21	Actual HorizonAI Interface Showing an Initial Failure, Feedback Cycle, and Successful Resolution	102
22	Comparative Compilation Success Rate on the CodeEditorBench Dataset	105

23	Behavioral Equivalence Rate by Task Difficulty	107
24	Behavioral Equivalence Testing of a HorizonAI Refactoring Output	108
25	Overall Mean Cyclomatic Complexity Comparison	110
26	Maintainability Index and Change Distributions of Original and Refactored Outputs	113
27	Inference Time Comparison Between HorizonAI and the Single-Model Baseline	116
28	Actual HorizonAI Execution Showing the Sequential Refactoring Stages	118
29	Peak GPU Memory Profile of HorizonAI and the Single-Model Baseline	120
30	Paired GPU Memory Differences Between HorizonAI and the Single-Model Baseline (Successful Runs)	121
31	Glassbox Interface of HorizonAI	124
32	Complete Model and Orchestration Parameter Configuration	146
33	Source Code of the AgentService Sequential Model Lifecycle	148
34	Source Code of the OrchestrationState Model and Main Orchestration Loop	149
35	Source Code of Phase 2 Strategy Sub-Steps	150
36	Source Code of the Generator Multi-Sample Loop and Output Repair Step	151
37	Source Code of Phase 4 Tier 2 Structural Checks	152
38	Phase 5 Judge Model Swap, Constrained Inference, and Hallucination Override	153
39	Phase 6 Finalization: Fallback Selection, Insights Generation, and Session Persistence	154
40	Prompt Excerpt — Anti-Pattern Guidance for the Conditional-Flattening Refactor Intent	155
41	Implementation Excerpt — WebSocket Disconnection Handling and Session Continuity	155

LIST OF APPENDICES

	APPENDIX TITLE	Page
A	Confidentiality and Non-Disclosure Agreement	173
B	Short Report of Plagiarism Software	176
C	Report of Language Software	177
D	Bionotes of Student Researchers	178

CHAPTER I

THE PROBLEM AND ITS SETTING

This chapter outlines the study's foundation, setting, necessities, and limits. This chapter also addresses the important issues that underlie this investigation.

Introduction

Recent developments in artificial intelligence have heavily relied on vertical scaling. The tech industry continuously pushes for larger language models to achieve better performance. However, this brute-force approach demands massive computational power and energy [1], [2]. For developers and institutions operating in resource-limited environments, running these massive systems locally is simply impossible.

Hardware limits are only a piece of the puzzle. Monolithic AI models—where a single system attempts to handle every cognitive step—struggle with reliability. If a model makes a slight logic error early in its reasoning, that mistake easily spreads. This cascading error effect is especially dangerous in software engineering. When automated tools generate code without strict validation, they often introduce hidden defects or unwarranted complexity that damages overall software maintainability [3], [4]. As these tools become more widespread, the need for reliable verification is critical. To combat this, researchers are increasingly exploring horizontal intelligence. Instead of relying on one massive model, the workload is divided among smaller agents with specific, mathematically bounded roles—specifically a Planner, a Generator, and a Judge. To keep these probabilistic models in check, some recent approaches pair AI generation with deterministic rules, like syntax analysis [5], [6], [7]. This forces the AI's output to obey strict structural laws rather than relying entirely on prediction.

Despite this progress, significant research gaps remain. There is limited empirical evidence comparing structured multi-agent systems against traditional monolithic setups when both are restricted by severe hardware limits. Fully offline implementations for software engineering are rarely explored, even though organizations increasingly demand locally deployed AI to protect their sensitive data

[8]. Without thorough evaluations of these offline setups, their real-world viability remains unclear. The industry requires approaches that improve code verification and reliability assurance without relying exclusively on expensive cloud infrastructure.

To bridge these gaps, this study developed and evaluated HorizonAI, a resource-efficient, role-driven multi-agent refactoring system. Designed specifically for consumer-grade hardware, the system operates entirely offline within a strict 4-gigabyte VRAM constraint. Instead of relying on a single massive system, HorizonAI optimizes the reasoning capabilities of Small Language Models (SLMs) by distributing the workload across specialized agents. To move beyond standard model integration, this research introduced formal algorithmic governance to the orchestration process. It utilizes Strict Sequential Inference Scheduling (SSIS) to enforce mutually exclusive model execution, AST-Constrained Semantic Alignment to map user intent to rigid code boundaries, and a Dual-Loop Deterministic-Neural Routing (D2NR) Algorithm that traps errors in automated syntax and structural circuit-breakers. Using Java as the testbed, the system's refactoring performance was benchmarked against the CodeEditorBench dataset, while measuring local inference time and memory efficiency. Additionally, technical experts assessed the system based on ISO/IEC 25010 software quality characteristics. End users, composed primarily of students and selected faculty members from computing-related programs, evaluated the system's Glassbox interface in terms of interaction capability, functional suitability, security, and reliability. Through this framework, the study sought to determine whether mathematically governed, role-driven orchestration could enable compact SLMs to perform controlled Java code refactoring under severe physical constraints.

Statement of the Problem

Current studies on AI-assisted code refactoring offer only a partial view of how Small Language Models (SLMs) perform in resource-constrained environments. While structured multi-agent approaches theoretically improve outcomes, empirical evidence regarding their performance on limited hardware remains scarce. To address this, the study fundamentally aimed to develop and evaluate HorizonAI, a

locally deployed, multi-agent orchestration system utilizing deterministic algorithmic routing to automate Java code refactoring within a strict 4-gigabyte VRAM constraint.

Specifically, the study sought to answer the following questions:

1. What are the system features of HorizonAI in automating Java code refactoring based on its measurable algorithmic constraints in terms of:
   1.1. multi-agent parameter distribution;
   1.2. deterministic validation interception rate; and
   1.3. iterative feedback resolution rate?
2. When tested on the CodeEditorBench dataset, what is the refactoring performance of the system compared to a single-model baseline in terms of:
   2.1. compilation success rate (CSR);
   2.2. behavioral equivalence rate (BER);
   2.3. computation of cyclomatic complexity; and
   2.4. maintainability index (MI) improvement?
3. During local execution on consumer-grade hardware, what is the computational efficiency of the multi-agent system in terms of:
   3.1. inference time; and
   3.2. peak GPU memory usage?
4. What is the level of user acceptability of the developed system's Glassbox interface, based on the ISO/IEC 25010:2023 in terms of:
   4.1. interaction capability;
   4.2. functional suitability;
   4.3. security; and
   4.4. reliability?
5. What is the perception of Information Technology experts regarding the developed system based on the ISO/IEC 25010 in terms of:
   5.1. functional suitability;
   5.2. performance efficiency;
   5.3. reliability; and
   5.4. maintainability?

Scope and Limitation

This study focused on designing and evaluating HorizonAI, a resource-efficient, role-driven multi-agent system for refactoring Java code under constrained computing conditions. Conducted within a year at the University of Cabuyao, the study's strict hardware limitations intentionally mirror the realities of neighboring institutions like St. Vincent College of Cabuyao and Westbridge Institute of Technology, Inc., which operate without access to high-end GPUs. For evaluation, twenty-five (25) purposively sampled computing faculty and students will assess the system's interaction capability, functional suitability, security, and reliability using the ISO/IEC 25010 model. Concurrently, ten (10) technical experts evaluated the architecture's functional suitability, performance efficiency, reliability, and maintainability based on the ISO/IEC 25010 Product Quality framework.

HorizonAI relies on several core offline modules. The **Orchestrator** manages the overall workflow of the system. It uses the Strict Sequential Inference Scheduling (SSIS) protocol, commanding the **Agent Service** to load and execute only one model at a time to strictly conserve memory. To ensure code reliability, the **Validator** functions as a multi-tiered deterministic routing engine that actively traps syntax errors for rapid healing and enforces structural integrity. Simultaneously, the **Context Manager** functions strictly as an isolated logging mechanism, capturing the reasoning process and agent handoffs for future retrieval without interfering with the Orchestrator's active routing logic. Finally, the **Connection Manager** bridges the strictly offline backend with the local Glassbox web interface, allowing users to safely monitor the refactoring process in real-time without an active internet connection.

The physical execution environment was restricted to a consumer-grade laptop equipped with an Intel Core i5 processor, 16 GB of RAM, and an NVIDIA RTX 2050. To operate within the strict 4-gigabyte VRAM limit, all deployed models were quantized to 4-bit precision. This hardware boundary directly shaped the multi-agent orchestration. The Planner is responsible for taking user input and creating a refactoring plan. To reduce semantic inconsistency between stages, the system uses the Qwen2.5-Coder-3B-Instruct model for both the Planner and Generator, with the Planner producing the refactoring plan and the Generator producing the refactored code based on that plan. Before the Judge evaluation, the refactored code undergoes

multi-tiered deterministic validation to ensure it is syntactically correct. Finally, the Judge (Llama 3.2 3B) either accepts the output, generate insights, or issue another instruction for refactoring if any errors are identified. For rigorous benchmarking, a 4-bit quantized Qwen 2.5 7B served as the single-model baseline—selected because it represented the highest-capacity monolithic architecture capable of running under identical offline, memory, and token constraints. Given the severe VRAM restrictions, system interaction is entirely text-based and confined to processing isolated Java snippets rather than complex, multi-file software repositories, explicitly excluding model fine-tuning, multimodal inputs, or distributed deployment.

HorizonAI was benchmarked using the CodeEditorBench dataset to ensure a standardized, reproducible baseline. The evaluation was limited to the refactoring scenarios represented within the selected benchmark tasks and implemented system coverage; therefore, unsupported transformations outside this scope (e.g., Extract Class and similar transformations) were not considered indicators of system performance. Every chosen metric served a specific purpose. Compilation Success Rate and Behavioral Equivalence Rate were used to assess whether the generated code compiled successfully and preserved the original program behavior, respectively. Beyond the code itself, hardware efficiency metrics were also evaluated. This research tracked the local inference time and peak GPU memory usage to determine whether the system could operate within the strict 4-gigabyte VRAM limit on a consumer-grade laptop.

A hard boundary of this implementation was its strictly offline nature. The system operates fully offline from the initial allocation of quantized weights into local memory through to the final output generation and display. Because it cannot rely on external server calls, its reasoning capacity is entirely capped by the local quantized weights. Furthermore, the inherent limitation of using probabilistic language models remains. Even though the multi-tiered deterministic validation actively heals syntax errors and monitors structural metrics, these routing mechanisms cannot mathematically guarantee that every refactored output perfectly aligns with abstract developer intent. Ultimately, the findings are bound to this locally deployed, resource-constrained environment, where hardware ceilings inevitably dictate both performance speed and system responsiveness.

Significance of the Study

The study is significant to computer science because it moves beyond merely plugging into off-the-shelf AI tools. It introduces a structured architectural approach that guides probabilistic models through predefined mathematical and validation constraints. By developing HorizonAI, the research analyzed how systems can generate credible code without relying on massive, cloud-based models. It aligns with a broader shift toward resource-conscious computing, where efficiency is an absolute necessity. Most importantly, the study formalizes Sequential Model Loading (SML) through its Strict Sequential Inference Scheduling (SSIS) protocol. This highlights SML not just as a temporary workaround for hardware limits, but as a core resource-management mechanism within the architecture and a generalizable pattern that mirrors industry-standard memory management. These contributions specifically apply to building resource-efficient systems for locally deployed environments.

Specifically, the findings of this study may benefit the following communities:

AI Researchers and System Designers. The study provides a blueprint for how complex tasks can be strictly governed within a single local execution workflow. By introducing the Dual-Loop Deterministic-Neural Routing (D2NR), it demonstrates a substantial technical contribution beyond simple model orchestration. Researchers can see exactly where failures occur, how algorithmic circuit breakers trap them, and how the system recovers. Because the design is formalized, these algorithms serve as highly reusable patterns for horizontal intelligence and multi-agent coordination across entirely different domains.

Industry Practitioners in Software Engineering. In software engineering practice, this framework offers a structured approach for integrating AI-assisted refactoring. Rather than relying on opaque, one-step generation, the system utilizes a transparent process with deterministic validation, enhancing overall transparency. This "verify-first" approach aligns with current code review practices where output is verified prior to integration. Ultimately, the study aids in understanding when a locally

deployed solution is adequate and identifies when more resource-intensive methods are necessary for a production environment.

Local Development Teams and MSMEs. The framework offers a resource-conscious reference approach for smaller companies and development teams, particularly in the Philippines, that cannot afford high-end GPUs or the constant, recurring costs of cloud API tokens. It demonstrates that a multi-agent refactoring workflow can operate within the study's defined 4-gigabyte VRAM constraint. This may provide local startups a practical foundation for deciding between offline deployment and cloud services, keeping their operational costs sustainable while ensuring their proprietary code remains secure on local machines.

Academic Institutions and Computing Programs. Many Philippine state universities and local computing programs operate with limited infrastructure. Because this system leverages Sequential Model Loading to fit inside a strict 4GB VRAM constraint, it was demonstrated to operate within the study's specified consumer-grade laptop configuration and 4-gigabyte VRAM limit, serving as a viable solution for schools that lack specialized AI labs. This may make the approach more accessible to institutions with comparable hardware. Students and faculty can study advanced practices like multi-agent orchestration and deterministic routing without needing expensive equipment, facilitating a deeper understanding of how AI architectures actually work under the hood.

Future Researchers. For future work, this study lays a grounded basis that can be extended. The specific algorithms engineered here—whether the memory scheduling or the validation loops—can be applied to different programming languages or entirely new hardware constraints. It clearly records the drawbacks of small-model reasoning and shows exactly how structural routing mitigates them. These results help establish resource-efficient, algorithmically governed AI systems as a vital research area in industry-driven computing.

CHAPTER II
REVIEW OF LITERATURE AND STUDIES

This chapter reviews the related literature and studies that provide the conceptual and empirical basis for the developed role-based multi-agent framework and contextualize the findings of the study. The review examines existing work on multi-agent systems, small language models, reasoning and validation frameworks, resource-constrained inference, and transparency in AI-assisted systems.

Conceptual Literature

Standard Features

Theoretical cognitive architectures show that monolithic neural models suffer dilution when handling diverse tasks such as planning and validation, as attention mechanisms fail to isolate subtasks. As Goyal and Bhasin [9] conceptualize, monolithic architectures create complex dependency chains that lead to cascading exceptions across modules. A mistake in early planning stages no longer remains isolated; instead, it contaminates the entire downstream generation process. Single-model setups therefore struggle to audit their own outputs without introducing confirmation bias. In response to this structural limitation, contemporary system designs shift toward multi-agent architectures, where cognition is modularized into specialized roles. By replacing monolithic dependencies with role-delegated agents, failures can be isolated and verification can be performed independently across components [9], [10]. HorizonAI adopts this principle as a defining feature, structurally separating planning, execution, and evaluation to prevent the propagation of unverified reasoning.

To operationalize this architecture, the system applies role-based task delegation logic, where each agent is assigned a singular and bounded responsibility. Rather than distributing tasks arbitrarily, delegation follows a structured hierarchy that decomposes complex objectives into verifiable units. As Belcak et al. [11] and Ren et al. [12] describe, assigning distinct objectives prevents cognitive overload and ensures that each stage of processing remains accountable. Within this structure, the planner defines the sequence of actions, the executor performs the actual

transformation or generation, and the evaluator independently validates the result. Each role contributes a function that the others cannot substitute: planning establishes direction, execution produces outputs, and evaluation enforces correctness. This separation ensures that generative intent does not interfere with validation logic, forming a controlled pipeline of reasoning.

While such specialization introduces clarity, it also requires a formal mechanism for coordination. Instead of relying on loosely connected model calls, the system implements custom orchestration logic, where task execution follows a deterministic sequence governed by predefined control rules. This avoids the unpredictability of unconstrained API chaining and ensures that transitions between agents are explicit and traceable. The orchestration layer acts as a controller that regulates when each agent is invoked, what input it receives, and how its output is validated before proceeding. In this way, the system behaves less like a collection of independent models and more like a coordinated pipeline with enforceable checkpoints.

This orchestration is further supported by a structured inter-agent communication system, where information exchange is treated as a formal subsystem rather than conversational interaction. Standard AI swarms often rely on natural language communication, but such exchanges introduce token overhead and contextual ambiguity [13]. To address this, the system adopts structured payloads such as JSON and Abstract Syntax Trees (AST), allowing agents to exchange only the necessary information in a controlled format [14]. This minimizes context expansion and ensures that each agent receives inputs that are both precise and machine-interpretable. At the same time, this design acknowledges a practical trade-off: while structured communication improves efficiency and control, it limits the expressive flexibility of free-form dialogue. The system therefore prioritizes reliability and resource stability over conversational richness.

Complementing this communication layer is a controlled approach to shared memory and context handling. Rather than allowing unrestricted accumulation of conversational history, the system treats memory as a managed resource composed of localized context and a minimal shared state. Each agent retains only the information relevant to its role, while essential outputs are passed forward through a

constrained context channel. This prevents context bloat and reduces the risk of exceeding memory limits. Conceptually, this reflects layered memory designs in modern agent systems, where short-term task context and shared knowledge are separated to maintain clarity and efficiency [15], [16], [17].

Given the constraints of local deployment, particularly under a 4GB VRAM ceiling, the system enforces resource-aware sequential execution as a core feature. Instead of loading multiple models simultaneously, agents are executed one at a time under the Strict Sequential Inference Scheduling (SSIS) protocol. This treats memory as a critical section, ensuring that only a single model occupies VRAM at any given moment [18]. Combined with 4-bit quantization techniques, this supports operation within strict hardware limits while preserving the intended multi-agent workflow [19]. Sequential execution is therefore not merely a limitation, but a deliberate architectural decision that ensures stability and predictability under constrained environments.

Beyond execution, the system introduces reasoning workflow traceability as an essential feature. Rather than producing outputs in isolation, each step in the agent pipeline is recorded and can be reconstructed as a sequence of decisions. This enables the visualization of reasoning paths, where the contributions of each agent can be inspected and evaluated. Such traceability aligns with broader principles of transparency and explainability in intelligent systems [20], [21]. By exposing intermediate reasoning steps, the system allows users to understand not only the final output, but also how that output was produced.

Finally, the system is conceptually framed against a single-agent baseline, establishing a comparative perspective on its design. While multi-agent systems introduce additional coordination overhead, they also provide advantages in modularity, fault isolation, and verification. The inclusion of this comparative framing ensures that the system's features are not viewed in isolation, but as deliberate responses to the known limitations of monolithic architectures. Rather than assuming superiority, the design acknowledges trade-offs and positions multi-agent orchestration as a structured alternative to single-model reasoning.

In summary, the standard features of the system are defined not as independent components, but as an integrated framework consisting of: (1) multi-agent role specialization, (2) structured task delegation, (3) deterministic

orchestration, (4) formalized inter-agent communication, (5) controlled shared context, (6) resource-aware sequential execution, (7) reasoning traceability, and (8) comparative design grounding. These features collectively establish a system that prioritizes clarity, control, and verifiability in resource-constrained environments where unconstrained model execution may be less practical.

Guidelines for Using the Algorithm

The orchestration of multiple language models within resource-constrained environments necessitates rigid architectural guidelines that transcend standard application programming interface (API) chaining. While conventional setups merely chain multiple API calls, custom orchestration logic is required to combine probabilistic generation with deterministic verification [22]. This neuro-symbolic approach ensures that the flexibility of large language models is structurally bounded by formal software engineering rules, treating automated code refactoring as a predictable, mathematical state machine rather than an unconstrained generative process.

The primary design constraint governing this system is the Strict Sequential Inference Scheduling (SSIS) protocol, which explicitly dictates shared memory and context handling. Because the physical deployment environment is restricted to a 4-gigabyte VRAM capacity, the literature emphasizes the need for strict memory governance in edge environments [23]. Drawing upon the operating system concept of mutual exclusion in memory management, the SSIS algorithmic guideline enforces mutually exclusive model execution. It mandates a strict load-swap-unload lifecycle, ensuring that the active context window is completely cleared before the next computational phase begins. This scheduling mechanism reduces the risk of out-of-memory failures during agent handoffs by preventing simultaneous model residency and supporting operation within the defined hardware constraint without external computational offloading.

Algorithm execution follows a strictly delineated sequence through three specialized roles: the Planner, the Generator, and the Judge. Theoretical frameworks in multi-agent software engineering emphasize that dividing cognitive tasks reduces context pollution and prevents sycophantic validation, wherein a single model blindly

approves its own flawed logic [24]. Initially, the Planner analyzes the structural objective and formulates a strategic blueprint. The system then unloads the Planner and instantiates the Generator, which synthesizes the candidate code based exclusively on the formulated blueprint. Finally, the Judge acts as an independent evaluation stage that assesses whether the generated candidate remains consistent with the intended refactoring and original program behavior.

To satisfy the architectural need for a robust inter-agent communication system, the framework explicitly rejects conversational natural language between agents, which wastes compute resources and context tokens [25]. Instead, the system utilizes structured state passing. Agents communicate strictly through JSON handoffs and Abstract Syntax Tree (AST) error logs. This shared memory is buffered and handled by a dedicated Context Manager, ensuring that the output of one agent serves as a mathematically parsable input for the next. This formal inter-agent communication reduces semantic drift and hallucination across the multi-agent pipeline.

Data progression and communication are governed by the Dual-Loop Deterministic-Neural Routing (D2NR) algorithm, acting as a neuro-symbolic routing mechanism [26]. Before the neural Judge evaluates a generated candidate, the output first passes through deterministic validation gates beginning with syntax checking. This architectural rule ensures that computationally expensive neural evaluations are only performed on structurally sound code. By utilizing deterministic tools as circuit breakers, the system can instantly trap syntax errors and generate precise AST error logs without relying on the probabilistic reasoning of a language model.

Finally, the algorithm operates as a bounded finite state machine, enforcing explicit failure handling and termination conditions. If the Generator produces syntactically invalid code, the deterministic loop triggers bounded correction iterations, routing the exact parser errors back to the Generator without invoking the Judge [6]. To prevent infinite recursion, the system imposes a maximum iteration threshold on both the inner syntax loop and the outer logic loop. The algorithm terminates when the candidate satisfies the required validation conditions or exits

with a controlled failure state when the defined iteration budget is exhausted, rather than accepting a potentially invalid output.

User-Centric Design

Advanced neural networks are often characterized as opaque or difficult-to-interpret systems. They rely on highly dimensional mathematical transformations that prevent human observers from verifying their internal logic. In software engineering, practitioners frequently refuse to adopt black-box AI tools because they cannot verify how the system arrived at a specific code modification [27]. Without the ability to trace the underlying reasoning, developers cannot trust the safety, compliance, or correctness of automated refactoring suggestions.

The architectural resolution to this dilemma is the implementation of a glassbox interface. Research establishes that visualizing the step-by-step execution trace of an AI agent directly increases user acceptability and professional trust [28]. By actively exposing internal system states, transition parameters, and inter-component dialogues, a glassbox transforms unpredictable neural generation into a debuggable workflow. Within the context of Southeast Asian technology adoption, Bernardo and Seva [29] emphasize that the affective design of Explainable AI (XAI) requires such transparent, user-centric mechanisms to mitigate cognitive dissonance.

The ISO/IEC 25010:2023 framework identifies interaction capability and security among its product quality characteristics [30], which contribute to the usability, protection, and perceived trustworthiness of a software system. To address the black-box limitations highlighted in XAI literature and build this operational trust, HorizonAI features a Glassbox interface. By exposing the Context Manager's logs, the system provides real-time workflow visualization, allowing users to inspect structured agent outputs, validation results, processing states, and handoffs among the Planner, Generator, and Judge.

Testing Tools and Procedure

To test tools and procedures applied to automated software engineering, standard probabilistic metrics fall short. Natural language processing metrics, such as BLEU or ROUGE scores, measure superficial semantic similarity but cannot guarantee execution safety. Compilation Success Rate (CSR) and behavioral equivalence verified through testing provide complementary measures for assessing whether AI-generated code is syntactically executable and its observable behavior is preserved [31]. True behavioral equivalence ensures that architectural modifications do not alter the external observable behavior of the legacy application [31].

Beyond basic syntactic correctness, the true quality of a refactoring operation is objectively quantified through software maintainability metrics. Measuring Cyclomatic Complexity (CC) serves as an objective method for evaluating software maintainability [32]. A reduction in cyclomatic complexity indicates fewer independent control-flow paths and may reflect a simplification of the program's control structure.

Following established software-testing procedures, HorizonAI evaluates refactored outputs through a combination of deterministic compilation and structural checks, behavioral testing, and model-assisted evaluation. The system's refactoring performance was evaluated on the CodeEditorBench dataset using Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change [33], providing an objective basis for comparing the multi-agent architecture with the single-model baseline.

Evaluating multi-agent architectures designed for localized, offline edge environments requires a theoretical shift in benchmarking. In resource-constrained deployments, software efficiency must consider both execution time and memory utilization [18]. Consequently, measuring peak graphical processing unit (GPU) memory allocation serves as the standard architectural procedure for validating algorithmic constraints. Because software efficiency in offline environments requires explicit memory-space validation, HorizonAI applies rigorous hardware profiling. Monitoring peak GPU memory usage provides a measurable basis for determining whether the multi-agent pipeline can execute within the study's defined 4-gigabyte VRAM constraint.

Research Literature

Review of Related Literature

Horizontal Intelligence and Agentic Decomposition

A growing body of literature challenges the assumption that a single, large AI model is the most effective way to handle complex tasks. Jahrens and Martinetz [34] establish theoretically that current decoder-only transformers are inherently incapable of forming discrete thoughts within their feature space; until token sampling collapses the output distribution, the model must maintain a superposition of multiple solution paths simultaneously, leaving only a fraction of its computational state available for any specific line of reasoning. This architectural property explains why monolithic systems deteriorate over sustained tasks. Rather than concentrating on all reasoning within one system, scholars increasingly advocate distributing work across multiple smaller, specialized agents—an approach broadly referred to as horizontal intelligence.

Technical literature on multi-agent construction emphasizes structured agent loops and tool orchestration as practical mechanisms for breaking tasks into discrete, manageable roles that may be more difficult to isolate within monolithic designs [35]. Industry handbooks on agent protocols detail standardized communication mechanisms—such as Model Context Protocol and Agent-to-Agent frameworks—that enable modular workflows where each agent contributes a specific function without overlapping responsibilities [36]. The orchestration of such systems requires designing specialized roles, establishing a coordination layer that governs their interactions, and defining protocols that standardize how information is represented and transferred. Pantazis [37] establishes that the value of multi-agent design lies not merely in dividing labor but in structuring collaboration so that each agent's output directly informs the next.

Role-based orchestration refines this architectural principle into explicit, non-overlapping responsibilities. The Planner-Executor-Evaluator loop represents a modular control paradigm that separates high-level planning, low-level execution, and real-time evaluation, enabling autonomous systems to operate robustly in dynamic environments [38]. Fadlon's conceptual framework extends this by separating reasoning, planning, and execution into distinct components, with the

reasoning component determining what the system should do and the orchestration component translating that intent into a structured execution plan [39]. This separation addresses a failure mode common in less structured systems: context drift, where a model that must reason, plan, and act within a single undifferentiated process loses track of the original intent as intermediate results accumulate [39].

Structured coordination completes this architectural picture by replacing conversational ambiguity with governed handoffs. In orchestrated multi-agent systems, specialized agents operate as autonomous components executing well-defined tasks, while an orchestration layer determines execution order, manages dependencies, and aligns individual outputs into a coherent operational flow [40]. Communication protocols standardize how information is represented and transferred, ensuring semantic consistency and enabling the orchestration layer to maintain synchronized, interpretable system behavior [40]. The critical distinction from conversational multi-agent designs is that structured handoffs prevent the compounding of ambiguity: each agent receives precisely defined inputs and produces precisely defined outputs, creating a pipeline where errors remain localized rather than propagating through unstructured dialogue.

While monolithic architectures rely on scaling parameters for reasoning, horizontal intelligence emphasizes decomposing tasks into specialized roles. HorizonAI follows this by separating planning, generation, and evaluation, and by using structured data handoffs instead of conversational interaction to reduce overhead. The Planner decomposes refactoring objectives into discrete, verifiable subtasks; the Generator executes transformations within bounded scopes; and the Judge evaluates the generated candidate against the intended refactoring and original program behavior before finalization. This arrangement mirrors the theoretical finding that decomposition and solving are separable cognitive functions best handled by distinct components [41], while the structured handoff mechanism between agents reflects the insight that standardized outputs prevent the ambiguity and error compounding inherent in free-form dialogue [40]. By grounding its architecture in these principles, HorizonAI transforms the limitations of monolithic reasoning into a governed, multi-stage workflow where each role's output directly

informs the next— creating a pipeline designed to provide greater structural control, fault isolation, and traceability across stages.

Neuro-Symbolic Governance and Deterministic Safety Nets

A recurring concern in the literature is the tendency of purely neural systems to produce outputs that appear plausible yet fail under closer structural inspection. This limitation is not simply about accuracy; it stems from how neural models generate outputs—through probability distributions rather than fixed rules. As discussed by Garcez and Lamb [42], this characteristic makes neural systems flexible but also difficult to constrain when correctness depends on strict formal requirements. In tasks like code refactoring, where even a minor syntactic inconsistency can invalidate the entire output, this limitation becomes especially critical.

To address this, a complementary line of work introduces neuro-symbolic integration, where neural generation is paired with symbolic reasoning. Instead of relying on a single mode of processing, these systems separate concerns: the neural component handles pattern recognition and candidate generation, while the symbolic component evaluates those candidates using explicit rules. Hitzler and Sarker [43] describe this setup as a way to combine adaptability with verifiability, allowing outputs to be checked against conditions that are not easily learned from data alone. Manhaeve et al. [44] offer a concrete architectural perspective on this integration, demonstrating how probabilistic logic programming can serve as an interface between neural and symbolic layers. Their work illustrates that real-valued probabilistic quantities—natural to neural networks—can be grounded in discrete logical structures without sacrificing gradient-based training, a principle that directly supports the design of systems where generation remains flexible but acceptance is strictly controlled.

This distinction becomes clearer when viewed through the lens of programming languages. Unlike natural language, code follows strict grammatical structures that can be formally defined and systematically verified. Concepts such as abstract syntax trees and formal grammars provide a concrete basis for this verification. Once code is represented as a structured tree, it can be analyzed deterministically, making it possible to detect violations before execution. In this

sense, symbolic methods do not replace neural models; they act as a filter that ensures only structurally valid outputs move forward.

The interaction between these two components naturally introduces a checkpoint mechanism. Neural outputs are not treated as final; instead, they pass through a validation layer that enforces structural and logical constraints. This architectural separation is demonstrated in frameworks such as CORE by Wadhwa et al. [45], which adopts a Proposer-Ranker approach where generated code revisions are treated as candidate solutions and are subjected to deterministic validation before being presented to developers. Their work emphasizes that code generation alone is insufficient and that multi-stage filtering is necessary to reduce the risk of introducing unintended changes. Similarly, runtime enforcement approaches such as AgentSpec by Wang et al. [46] highlight the importance of external deterministic controls that evaluate proposed agent actions against predefined constraints before execution. Therefore, when generated outputs fail these validation conditions, the system should not treat the result as an acceptable replacement. Instead, rejection or controlled fallback becomes a necessary mechanism to prevent unverified modifications from propagating into the user's existing codebase. This creates a controlled loop where errors are intercepted early rather than allowed to propagate, while preserving the reliability of the original artifact when no validated alternative can be established.

In code-related workflows, this combination of interception and feedback is essential. Brain and Polgreen [47] frame deterministic validation as part of a broader classification of verification techniques, distinguishing between over-approximate methods that may produce false alarms and under-approximate approaches that guarantee any defect found is real. Rather than relying on the model to self-correct through additional prompts, the system introduces an external mechanism that evaluates explicit structural constraints independently of the model's generative reasoning.

Within this context, HorizonAI's design aligns with these principles through its Dual-Loop Deterministic-Neural Routing (D2NR). Instead of treating generation and validation as a single continuous process, the system separates them into distinct phases. Neural components produce candidate refactorings, while deterministic

AST-based checks act as circuit-breakers that intercept invalid outputs. When violations are detected, structured feedback is returned to earlier stages, allowing the system to adjust before proceeding. This setup reflects the broader direction in the literature: combining probabilistic generation with rule-based validation to maintain both flexibility and control, especially in environments where correctness cannot be left to chance.

Algorithmic Resource Scheduling and Mutual Exclusion

Operating within strict hardware constraints requires not only capable models but a formally principled strategy for managing shared computational resources. Classical synchronization theory establishes mutual exclusion as a fundamental mechanism for coordinating access to shared resources: the guarantee that when a critical resource is occupied by one process, all competing processes must wait until that resource is explicitly released, enforced through synchronization primitives such as locks and semaphores [48]. Yang [49] reinforces the theoretical foundations of these mechanisms in the context of concurrent programming, demonstrating that mutual exclusion and condition synchronization are the two indispensable primitives for preventing process interference and deadlock in multi-threaded systems—and that without appropriate synchronization, concurrent access to shared resources can produce race conditions and inconsistent system states. HorizonAI applies these principles to multi-model inference by restricting model residency so that only one agent model is actively loaded into GPU memory at a time under SSIS, and the transition between models must be governed by a formally specified eviction protocol to prevent memory exhaustion.

Parallel to the scheduling problem, the theoretical challenge of fitting neural model parameters within a fixed physical memory budget has been addressed through the mathematical framework of weight quantization: the systematic reduction of the numerical precision of learned weight representations from high-precision floating-point encodings to lower-precision integer approximations [17]. As a theoretical framework, quantization exploits the empirical observation—formalized through information-theoretic analysis—that the majority of a neural network's representational capacity is preserved under aggressive precision reduction,

provided that quantization error is distributed across layers in a principled manner rather than accumulated uniformly. This mathematical argument establishes that reducing weight precision from 32-bit floats to 4-bit integers does not simply truncate model capacity but redistributes it, preserving the most semantically critical weight structures while compressing the overall memory footprint by a factor commensurate with the precision reduction. Belcak et al. [11] complement this theoretical framework by establishing that Small Language Models—with their reduced parameter counts—are inherently more amenable to quantization-based compression than frontier-scale LLMs, as the density of semantically irreplaceable weights per parameter is higher in smaller, task-specialized architectures, making precision-reduction techniques less disruptive to core task performance. Critically, while GPU synchronization theory, and weight quantization theory each independently provide rigorous frameworks for resource governance and memory compression respectively, no existing theoretical literature formalizes their integration into a single orchestration protocol governing multi-agent SLM inference. Within this study, SSIS integrates mutually exclusive model execution with sequential loading of 4-bit quantized models as a unified resource-management strategy for operating the multi-agent pipeline within the defined 4-gigabyte VRAM constraint.

Frameworks for Transparency and Trust in AI

Trust and inspectability in AI-assisted software engineering can also depend on whether users can trace the observable workflow, intermediate outputs, and validation processes that produced a result. Bersani et al. [50] frame this clearly by treating explainability as a non-functional requirement in software systems, outlining levels that range from local explanations to system-wide transparency. Mitchell [51] approaches the same issue from a different angle, arguing that trust in AI is not built through full exposure alone, but through a balance—enough transparency to allow verification, without stripping the system of its autonomy. This distinction between type transparency and token transparency helps clarify how systems can remain inspectable without becoming rigid or overly constrained.

These ideas are also reflected in established software quality standards. The updated ISO/IEC 25010:2023 framework places a strong emphasis on the need for

systems to be examined and understood internally, anchoring this requirement within the analysability and testability sub-characteristics of maintainability. Furthermore, it redefines interface acceptability and user trust through factors such as interaction capability, security, and reliability—criteria that directly relate to how transparent and safe a system is in practice. At the core product level, functional suitability and performance efficiency remain central. Von Eschenbach [52] highlights how opacity disrupts this entire structure, pointing out that black-box AI systems make it difficult for users to form justified confidence in their outputs. When these systems are applied to sensitive or proprietary code, the lack of visibility into their reasoning becomes more than an interaction issue—it becomes a barrier to responsible use.

Despite these clear directions from both theory and standards, many AI-assisted development tools still provide limited visibility into intermediate processing and validation steps. Users are typically shown only the final output, with no access to intermediate reasoning, decision points, or validation steps. There is no way to see how one stage led to another, or where things might have gone wrong. Winfield et al. [53] stress that transparency in autonomous systems has to be designed into the system itself, not added later as a surface feature. Without that, core requirements such as inspectability and traceability remain unmet.

HorizonAI approaches this gap directly through its Glassbox interface. Rather than treating transparency as an add-on, it exposes the full reasoning trace of each refactoring process—from agent interactions and decision points to validation outcomes and correction cycles. This makes it possible to follow how a given output was produced, step by step. In doing so, the system aligns with Bersani et al.'s [50] view of self-explainable architectures, where explanation is built into the system rather than generated externally. More importantly, it shifts away from the black-box model that dominates current tools, offering instead a clear, inspectable workflow that supports both trust and maintainability as defined in existing software quality standards.

Review of Related Studies

Multi-Agent LLM Systems for Code Generation and Review

The empirical question of whether parameter-compact language models can perform competitively on code-specific tasks has been systematically addressed across several recent benchmark studies, providing the evidentiary basis for HorizonAI's decision to deploy sub-3B quantized architectures. The findings collectively demonstrate that SLMs, when properly selected and quantized, produce competitive coding outputs while remaining viable on edge hardware—but only under conditions that include structured orchestration and deterministic validation.

Hui et al. [54] provide the most directly relevant benchmark evidence for HorizonAI's model selection. Their evaluation of the Qwen2.5-Coder family reported that the 1.5B-parameter variant achieved an average performance improvement of 3.7% over comparable models in its parameter class, rivaling the majority of models exceeding 6 billion parameters across more than ten benchmarks spanning code generation, completion, reasoning, and repair domains. The entire Qwen2.5-Coder family demonstrated state-of-the-art performance within its size category, findings the authors attributed to thorough data cleaning, scalable synthetic data generation, and balanced data mixing strategies. These results are significant not as isolated benchmarks but as an empirical ceiling: they establish what a 1.5B–3B model is capable of achieving under optimally curated training conditions, defining the upper bound of performance that HorizonAI's 3B Qwen-Coder instances can realistically approach during Java refactoring.

Complementing this benchmark data, Nguyen and Nguyen [55] provide empirical evidence specifically addressing the trade-off between model size, inference speed, and practical usability on consumer hardware. Their findings establish that models in the 1B–1.5B parameter range respond with minimal device stress, while models in the 3B range, though slower, remain fast enough to be practically useful. Critically, they confirmed that 4-bit quantization is essential for local inference on memory-limited devices, reporting that this precision reduction can be applied with minimal speed penalty—a finding that directly justifies HorizonAI's use of 4-bit precision across all three pipeline agents. Together, the Hui et al. and Nguyen and Nguyen results empirically establish that 3B quantized SLMs occupy a viable

operating range: capable enough for substantive code tasks, compact enough to survive a 4GB VRAM ceiling, and fast enough to be practically deployable.

However, the same body of evidence reveals a structural limitation that benchmarks alone cannot expose. Golnari et al. [56] report that sub-3B SLMs generating Java code under standard single-model conditions fail to produce syntactically valid output in approximately half of evaluated cases. This compilation failure rate is not a marginal anomaly; it is a systematic empirical regularity that carries direct architectural implications. A pipeline that forwards all generated outputs to a downstream evaluation agent without intermediate deterministic validation will inevitably produce unreliable acceptance decisions, because the evaluation agent—itself a probabilistic SLM—cannot reliably distinguish a subtly malformed output from a correctly refactored one. Kim et al. [57] corroborate this risk at the architectural level, demonstrating that confidence blindness in monolithic LLMs causes early generation uncertainties to amplify across subsequent reasoning steps, producing cascading failures that probabilistic self-assessment mechanisms are structurally ill-equipped to detect. The empirical picture that emerges is therefore not a simple endorsement of SLMs but a more nuanced finding: SLMs are capable but probabilistically brittle, and their viability in code-generation pipelines depends critically on whether a deterministic gate is placed between generation and evaluation.

Comparative Success Rates of Multi-Agent vs. Monolithic Pipelines

A converging body of empirical research demonstrates that partitioning code-related AI tasks across specialized agents produces measurably superior outcomes compared to monolithic, single-model architectures. The evidence spans ablation studies, multi-dataset evaluations, and domain-specific benchmarks, consistently reporting that role separation reduces inter-role bias, enables systematic feedback-driven refinement, and directly increases the rate at which generated outputs meet correctness criteria.

Xu et al. [58] provide the most direct empirical evidence through their evaluation of MANTRA, a multi-agent system in which one agent rewrites code while a second critically reviews it in an iterative back-and-forth loop. Tested against over

seven hundred real examples drawn from ten different software projects, MANTRA's performance was evaluated through a systematic ablation: when one model was removed from the pipeline, the number of successful rewrites dropped by 40% to 60%. This performance gap is not marginal—it constitutes empirical proof that the roles are not redundant but functionally necessary, and that removing even a single agent from a multi-role pipeline degrades outcome quality in a quantitatively significant and reproducible way.

AgentCoder [59] extends this empirical foundation by evaluating a three-role decomposition analogous to HorizonAI's own architecture. The system separates code-related responsibilities into a test designer (equivalent to HorizonAI's Planner), a programmer agent (equivalent to the Generator), and a test executor (equivalent to the Judge), arranged in a structured iterative loop. The results demonstrate that this separation of design, implementation, and verification responsibilities reduces inter-role bias and enables feedback-driven refinement that single-agent baselines cannot replicate—an empirical finding directly applicable to HorizonAI's own three-role Planner-Generator-Judge pipeline.

He et al. [24] reported that agent specialization can improve selected outcomes in complex software engineering tasks by distributing responsibilities among specialized roles. These findings suggest that role decomposition may provide advantages in task organization, validation, and maintainability; however, performance improvements remain dependent on the evaluation criteria, system design, and execution constraints.

Importantly, multi-agent performance advantages extend beyond large-scale cloud systems and have been empirically validated in contexts directly relevant to HorizonAI's deployment environment. Local research by Fernando et al. [60] and Guia and Ballera [61] provides Philippine-context empirical evidence that distributed agents can collaboratively resolve highly constrained problems that otherwise paralyze generalized, monolithic systems— findings that are particularly significant given HorizonAI's target context of resource-constrained Philippine academic institutions, such as the University of Cabuyao, St. Vincent College of Cabuyao, and Westbridge Institute of Technology, Inc. These results establish that the performance benefits of multi-agent decomposition are not contingent on cloud-scale

computational resources, but are achievable under the kinds of hardware and connectivity limitations that characterize HorizonAI's intended deployment environment.

Taken together, however, these prior systems shared a foundational limitation that the present study addressed directly. He et al. [24] noted that agents in prior multi-agent systems depend on general-purpose large language models such as ChatGPT, presupposing cloud connectivity and excluding offline deployment. None of the surveyed systems employed a strict deterministic gate—such as a mandatory Java compilation check—as an acceptance precondition, leaving unverified whether generated outputs are structurally executable before they are evaluated or accepted. Unlike these prior systems, HorizonAI operates entirely offline using compact sub-3B models and validates every generated output through a mandatory compilation check before it reaches the Judge agent—an architectural combination that no prior multi-agent study has empirically evaluated.

Experiments conducted on established datasets indicate that multi-agent pipelines may provide advantages in logic-intensive tasks through role specialization and structured coordination. These findings supported the design premise of HorizonAI, where refactoring was partitioned into Planner, Generator, and Judge roles to investigate whether structured orchestration could improve selected refactoring and efficiency metrics compared with a single-model baseline.

Efficacy of Hybrid Deterministic-Neural Repair Loops

Automated Program Repair (APR) research has produced a clear and reproducible empirical pattern: systems that rely exclusively on neural self-correction to handle structural code errors consistently underperform relative to systems that incorporate deterministic, rule-based validation as a feedback mechanism. The evidence draws a sharp distinction between probabilistic inconsistency—the tendency of neural models to generate plausible but structurally invalid outputs—and the near-perfect compilation reliability achievable when symbolic circuit-breakers intercept errors before they reach downstream evaluation stages.

The critical methodological gap in existing neural-only refactoring studies is most clearly illustrated by Cordeiro et al. [62] and Shirafuji et al. [63], both of which

report positive quality improvements without verifying that the refactored code remained executable. Cordeiro et al. [62] conducted an empirical study using 30 open-source Java projects, demonstrating that LLM-generated refactoring outperformed developer-performed refactoring in simplifying code structures and improving overall readability. Shirafuji et al. [63] similarly reported improvements, including a 17% reduction in cyclomatic complexity, by providing models with high-quality rewrite examples. However, a critical methodological limitation is shared by both studies: neither subjected refactored outputs to compilation or unit-test execution before measuring quality metrics. Both frameworks proceeded directly from generation to quality assessment, meaning that a syntactically invalid—and therefore non-compiling—output could still exhibit apparent improvements in cyclomatic complexity or readability metrics, producing inflated quality scores for artifacts that would fail in a production environment. Neither study employed an adversarial evaluation agent to formally assess semantic equivalence, leaving unverified whether refactored code preserved the behavioral contract of the original. The quality scores reported by these studies are therefore epistemically unreliable as evidence of functional improvement: they measure the appearance of correctness, not its substance.

The empirical literature on hybrid deterministic-neural approaches reveals why this distinction matters quantitatively. Golnari et al. [56] report that sub-3B models generating Java code under standard conditions fail to produce syntactically valid output in approximately half of evaluated cases—meaning that approximately 50% of outputs assessed by Cordeiro et al.'s and Shirafuji et al.'s methodologies may have been non-compiling artifacts measured as if they were functionally correct. Studies that have specifically compared neural self-correction mechanisms against deterministic validation loops report a dramatic performance gap Zhao et al. [63] found that fully neural self-repair loops (pure LLM-generated patches without structured feedback) achieve a repair “pass rate” of roughly 25–30% on large competitive-code datasets, whereas AST-structured process-based feedback raises the success rate to over 40%, significantly reducing hallucination loops and compilation-only-valid patches.

Li et al. [64] further demonstrated that hybrid repair loops—where deterministic APR components filter and refine LLM-generated candidates—achieve up to 86–90% “pass rate” on compilation-driven benchmarks, with near-perfect compilation success (≈95% in some pipelines) when integrated with evidence-based pre-filters such as Historian-style correctness verifiers. This task separation mirrors HorizonAI's D2NR architecture, in which the deterministic inner loop intercepts syntax violations and returns exact AST error logs to the Generator before the Judge is ever invoked, reserving neural evaluation for outputs that have already passed structural verification.

Comparative studies highlight a significant performance gap: fully neural systems relying on 'self-correction' frequently enter 'hallucination loops,' failing to resolve structural errors. Conversely, studies utilizing hybrid deterministic feedback (AST-based) reported near-perfect compilation success. This empirical evidence validates HorizonAI's D2NR algorithm as a definitively more reliable alternative to purely probabilistic error-correction methods.

Computational Latency and VRAM Profiling in Edge Inference

The empirical literature on locally deployed AI inference establishes that consumer-grade hardware imposes non-negotiable constraints on how neural models are loaded, executed, and transitioned—and that these constraints require purpose-built architectural strategies rather than scaled-down versions of cloud deployment patterns. Benchmark studies across edge inference environments reveal that memory management and inference scheduling are primary determinants of system survivability on hardware with fixed VRAM budgets.

Nguyen and Nguyen [55] provide the most directly applicable empirical baseline for HorizonAI's hardware context. Their benchmarks on consumer-grade devices confirm that 4-bit quantization is essential for local inference on memory-limited hardware, producing minimal speed penalty relative to higher-precision alternatives. Their findings establish that models in the 3B parameter range remain practically deployable on edge devices—slower than sub-1.5B models but within usable response windows—and that quantization-based compression is the enabling condition for this deployability. These results define HorizonAI's empirical operating

baseline: that a 4-bit quantized 3B model is at the practical frontier of what consumer hardware can run without external offloading, and that any multi-agent pipeline attempting to run three such models must implement explicit memory governance to avoid simultaneous loading failures.

Alistarh [65] measured VRAM utilization on consumer-grade GPUs (4–8GB VRAM) and showed that sequential model loading can reduce peak VRAM by 20–30% compared with parallel loading, because framework overhead and KV-cache buffers are reused across steps rather than duplicated per model. Their profiling data established that peak memory events occur during model initialization and weight loading, not during inference itself, making the sequencing of load-and-unload operations the primary determinant of whether a multi-model pipeline survives within a fixed VRAM ceiling. Nyamsuren [66] corroborate this finding in a controlled benchmark of 1–7B quantized code LLMs on edge GPUs, reporting that sequential inference scheduling lowers peak VRAM by up to 25% on 4–8GB devices, while maintaining near-identical pass@1 scores on HumanEval and MBPP.

The socioinstitutional context for these hardware constraints has also been empirically documented. The World Bank [67] records that Philippine state universities and local colleges face infrastructure deficits so severe—including insufficient high-end GPUs and inadequately funded digital infrastructure—that cloud-dependent LLM solutions are structurally inaccessible as everyday tools for the computing faculty and students who constitute HorizonAI's primary user cohort. This documentation transforms HorizonAI's 4GB VRAM constraint from an experimental parameter into an empirically grounded representation of a real and widespread deployment environment, giving the system's hardware profiling results direct generalizability to the institutional contexts it is designed to serve.

Taken together, these empirical findings establish that the combination of sequential model swapping, 4-bit quantization, and explicit VRAM profiling is the only documented approach capable of enabling multi-model inference on consumer hardware within the 4GB constraint. No prior study has evaluated all three components operating simultaneously within a role-differentiated multi-agent pipeline— the gap that HorizonAI's SOP 3 metrics were designed to address.

Benchmark results from Alistarh's study [65] on consumer-grade GPUs indicate that sequential model swapping significantly reduces peak VRAM spikes compared to simultaneous loading. These empirical metrics serve as the technical benchmark for the system, providing a data-driven expectation that HorizonAI will survive rigorous execution constraints.

Table 1. Summary of related studies and research findings

Study	Year	Model Type	Deployment	Validation Method	Offline?	Gap
[42]	2025	Large LLMs (GPT-4 class	Cloud (internet req.)	Compilation + test-run (82.8% success)	No	No sub-3B agents; cloud-only
[37]	2024	Large LLM	Cloud	Static quality checker (76.8% pass rate)	No	No offline path
[38]	2024	Large LLM	IDE plugin / cloud	Static analysis filter (53.4% correct)	No	Discards 57.4% of outputs; cloud-only
[36]	2025	Deterministic tool (RefactoringMiner)	Local (deterministic)	AST diff precision 99.7% / recall 94.1%	Yes	No neural generation; no offline SLM pipeline
[44]	2024	Large LLM	Cloud	Developer comparison (17.4% CC reduction)	No	No compilation gate; no Generator; cloud-only
[45]	2023	Large LLM (few-shot)	Cloud	CC reduction 17.35%; 95.68% correctness	No	No compilation gate; no offline config; no adversarial agent
[39]	2024	phi-3-mini 3.8B (4-bit)	On-device (phone)	Benchmark accuracy; >12 tok/s	Yes	Single model; no multi-agent roles; no code refactoring task
[41]	2025	SLMs 1B–3B (4-bit quant)	Single-board computers	5–15 tok/s throughput	Yes	Single models; general tasks; no multi-agent pipeline
[46]	2024	Qwen2.5-Coder SLMs (1.5B+)	Benchmark eval	HumanEval + 10 code benchmarks	Yes (local)	Single model; no Generator;

Table 1 consolidates the studies discussed in the Research Literature section above, enabling direct comparison across deployment context, validation method, and the specific gap each leaves relative to the present study.

Theoretical Background

Rewriting code automatically is not just a technical problem; it is a cognitive challenge for the models. No single AI model is capable of handling the entire cognitive load of refactoring autonomously. This framework mitigates this by decomposing the workload. A Planner agent formulates the refactoring strategy, a Generator agent executes the transformation, and a Judge agent evaluates the outcome. Each component is mathematically bounded to a singular function. In addition, two separate deterministic tools operate alongside the models: one validates syntactic execution, while the other computes cyclomatic complexity. This design is grounded in distributed cognition theory, which posits that complex reasoning does not need to occur within a single entity [68]. By distributing the computational burden across specialized components, the system is designed to produce controlled and verifiable outputs within the study's defined hardware constraints. This framework functions similarly. The code has to pass through every step sequentially, and the outcome is never reached until all the checks have been passed [68].

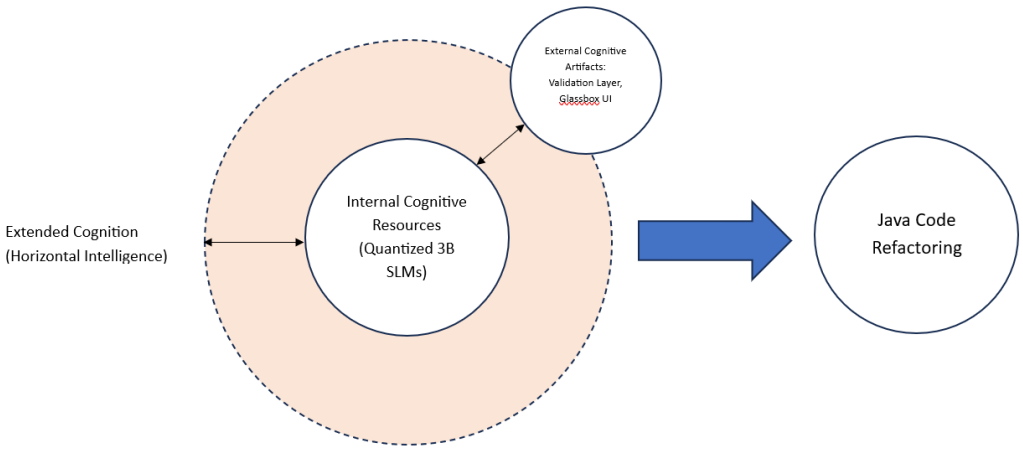


Figure 1. Distributed Cognition Model Applied to the Multi-Agent Java Code Refactoring System

Figure 1 shows how the different parts of this system work together. Three small AI models handle the core thinking work. One plans the rewrite, one carries it out, and one Judges whether the result is acceptable. Each of these models has real

limits on its own. To make up for those limits, the system adds three supporting tools. One tool checks that the rewritten code is free of errors, while another shows the developer exactly how every decision was made. Together, these parts do something no single large AI model could do alone. The work is split across several smaller, specialized pieces that each handle one job well, rather than relying on one powerful piece to handle everything. Raw Java code goes in. Compiled Java code is generated, which is simple and verified.

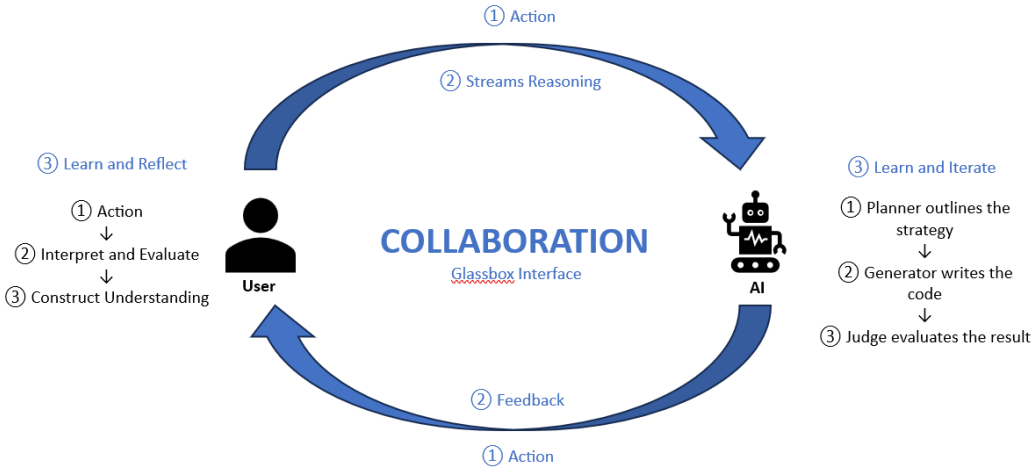


Figure 2. Human-AI Collaboration Framework Applied to the Hybrid Multi-Agent Refactoring System

The framework relies on a clear division of labor between the user and the AI, as shown in Figure 2. As highlighted by The Decision Lab [69] in their research on Human-AI collaboration, cooperation works best when each side focuses on its specific strengths. The AI system handles the heavy computing tasks through three distinct roles to keep the process reliable and prevent errors from piling up. First, the Planner reviews the user's request and outlines a clear strategy. Next, the Generator takes that plan and builds the new code. Finally, the Judge reviews the completed work to offer helpful insights or demand further revisions if something looks wrong.

Meanwhile, the human student takes on the reflective role that an AI cannot fully replicate. Because the Glassbox interface provides step-by-step explanations

for every change, the student is brought directly into the thinking process. The AI's output is treated as a highly informed proposal, not a final mandate. By reviewing the reasoning logs, the student acts as the ultimate decision-maker—auditing the multi-agent deliberation, verifying the logic, and deciding whether to accept, reject, or further modify the generated code. This setup ensures the student retains full agency, actively learning from the structural improvements rather than passively accepting a black-box answer.

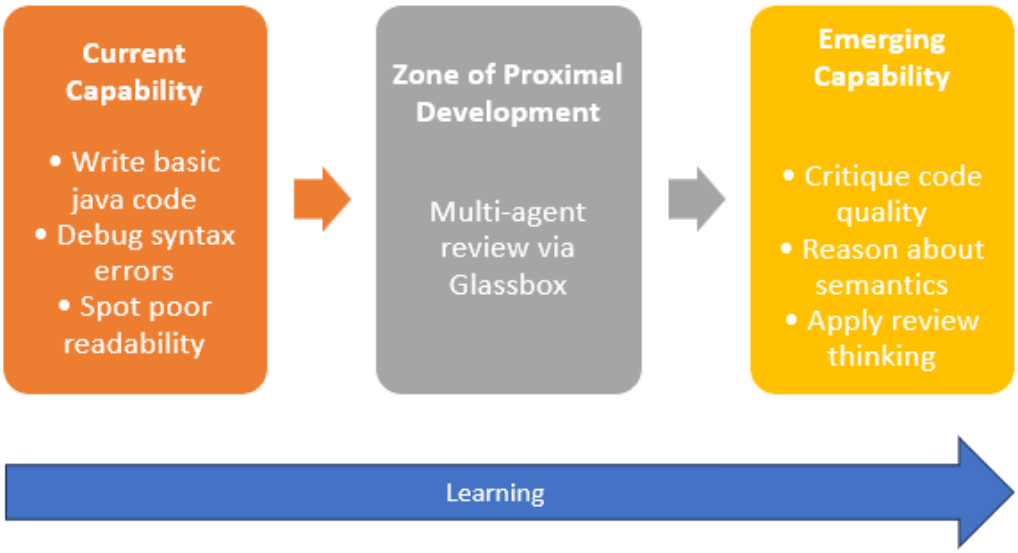


Figure 3. Constructivist Learning Framework Applied to the Glassbox Interface

As Figure 3 demonstrates, ELM Learning [70] emphasizes the need for a Glassbox interface: end-users must be exposed to how a system arrives at its conclusions. An instrument that merely provides the developer with a completed product without a description is not beneficial to them. They are not even aware whether the rationale behind the result was good. They are not able to know where things may not have gone right. They do not even have an incentive to believe the output and doubt it. The Glassbox remedies this by displaying one step as it comes in real time. The developer is able to observe what each section of the system does and why it performs that function. This transparency is not an additional tool; it is a fundamental condition of trust in AI systems. They must have the capacity to look within an AI tool in order to trust it. Once developers can view the rationale behind a

decision, they are able to discover errors. These challenge decisions appear incorrect, and remain in control of the end product rather than passively receiving what the system provides them.

Three concepts define this structure, and they all lead in one direction. The former describes why the work is divided into multiple AI models: no single one of them is reliable in all tasks, and each performs the task in which it is most skilled. The second describes the purpose of the human: the AI does the tedious repetitive labor, and the developer adds judgment and common sense absent in the AI. The third states the reason why the system displays its work: a developer cannot trust a result they cannot see the logic behind. Each concept temporarily bridges the gaps that the rest of them create. The two describe a system that is trustworthy and transparent in its boundaries, as well as one that allows the developer to remain truly in control.

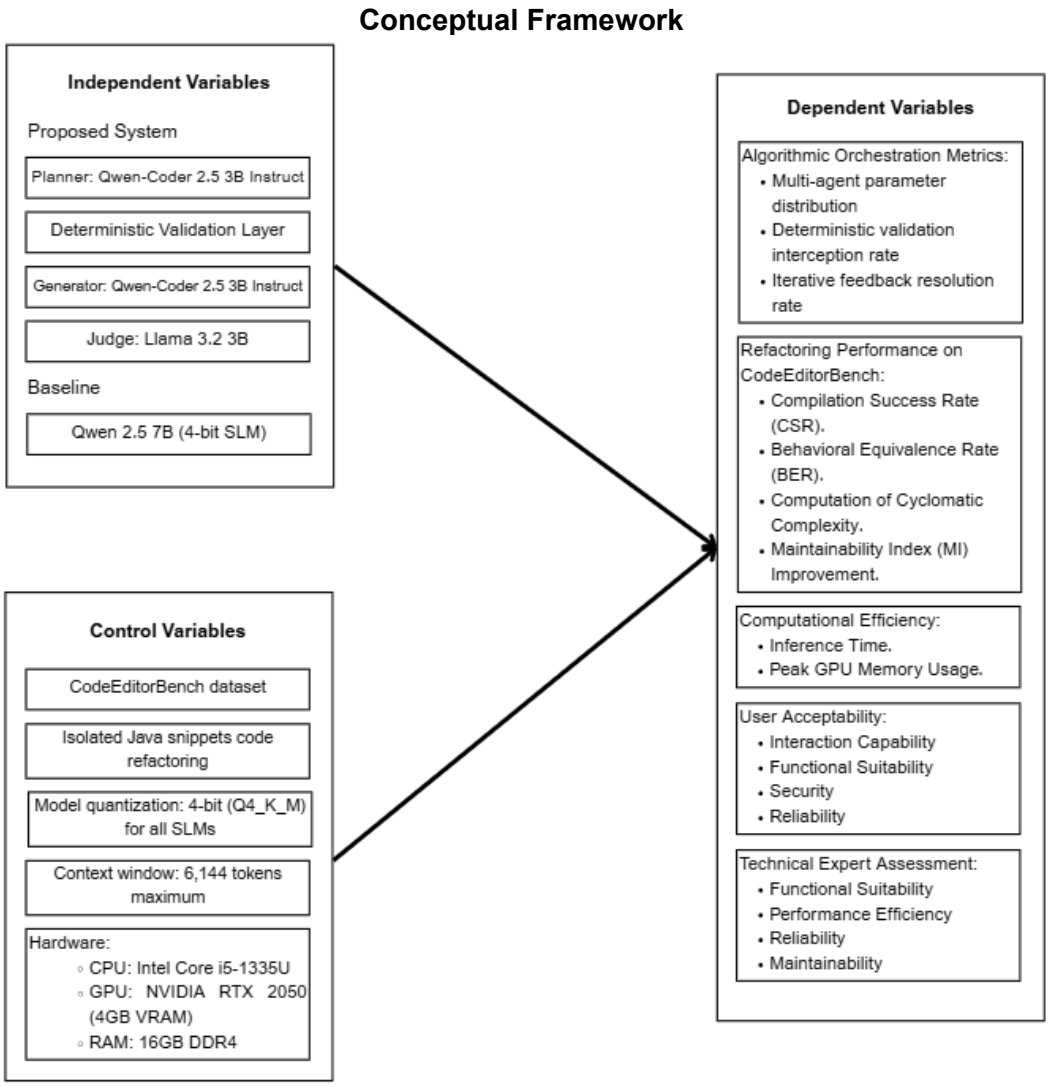


Figure 4. Conceptual Framework of the Study

Figure 4 illustrates the conceptual framework of the study, which evaluates HorizonAI, an architecture designed as a self-regulating ecosystem that automates Java code refactoring while thriving under extreme hardware limitations. The analysis centers on a comparison between two distinct architectural approaches (the Independent Variables). The first is the proposed multi-agent system, which is engineered to be robust by decoupling logic into specialized roles: a Planner for strategy, a Generator for syntax, and a Judge for logic. Crucially, this system is self-correcting; it utilizes a deterministic validation layer that functions as a Dual-Loop Deterministic-Neural Routing (D2NR) engine. This loop intercepts and "heals" syntax

errors in real-time before the code ever reaches the final adjudication stage, ensuring that the system fixes its own mistakes rather than propagating them. This multi-agent setup was tested against a single-model baseline (Qwen 2.5 7B) to determine if horizontal intelligence provides a superior structural advantage over raw model scale.

The framework measures system performance through five categories of outcomes (the Dependent Variables). First, the system's internal health is quantified through orchestration metrics, where the iterative feedback resolution rate serves as the primary evidence of its self-correcting capability. Second, the refactoring success is benchmarked against the CodeEditorBench dataset, tracking the Compilation Success Rate (CSR) and Behavioral Equivalence Rate (BER) alongside structural improvements like reduced cyclomatic complexity. Third, to assess the architecture's resource efficiency, the study monitored inference time and peak GPU memory usage. This was used to determine whether the Strict Sequential Inference Scheduling (SSIS) protocol successfully "clears the stage" for each model, preventing the hardware crashes typically associated with running large models on a 4-gigabyte VRAM limit. Finally, auditability and quality were measured using ISO/IEC 25010:2023-based criteria, capturing how both end-users and IT experts perceive the system's reliability and performance.

Several control variables are strictly enforced to ensure the comparison remains equitable. All models, including the baseline, were quantized using 4-bit quantization and constrained to a maximum 6,144-token context window. Furthermore, all testing was conducted entirely offline on consumer-grade hardware, specifically an NVIDIA RTX 2050 GPU, to maintain a rigid 4-gigabyte VRAM ceiling. By holding these environmental factors constant, the study sought to ensure that any variance in performance is not a fluke of hardware power, but a direct result of how a resource-aware architecture can maximize limited computational assets through strategic, self-regulating loops.

Synthesis

The reviewed literature converges on a single organizing distinction that structures this study's contribution: whether structural error correction in AI-assisted code generation is delegated to further probabilistic generation, or to a fixed, non-learned verification layer that generation cannot override. Five overlapping themes anchor this distinction: the theoretical grounding for decomposing reasoning into distinct cognitive functions, the documented limitations of monolithic probabilistic models on formally constrained tasks, the empirical performance gap between fully neural self-correction and hybrid deterministic-neural validation, the measurable advantage of role-specialized multi-agent decomposition over single-model architectures, and the practical feasibility of deploying such systems entirely offline under severe hardware constraints. Taken together, these themes do not merely describe separate research directions; they converge on a single unoccupied position that this study was designed to fill.

The theoretical literature establishes that decomposition and verification are separable cognitive functions best handled by distinct components rather than compressed into a single reasoning process. Dual-process accounts distinguish fast, probabilistic generation from deliberate, rule-governed evaluation, framing this separation as architecturally required, not merely advantageous, for tasks in which a single structural error constitutes complete failure. Complementary work on neuro-symbolic integration reinforces this position, arguing that neural components are well suited to pattern recognition and candidate generation but are poorly suited to enforcing correctness conditions that must hold without exception; symbolic or rule-based components are needed to check candidates against those conditions. This theoretical foundation directly motivates HorizonAI's separation of probabilistic generation from a fixed, rule-governed validation stage, rather than treating both functions as tasks a single model can perform reliably at once.

Empirical work on monolithic, single-model systems corroborates this theoretical concern. Language models of varying scale, including those specifically optimized for code, have been shown to produce fabricated references, structurally invalid syntax, and incorrect method signatures because they lack access to a compiler or interpreter at the moment of generation. Increasing model size does not

resolve this limitation, and compressing models through quantization further widens the gap between linguistic fluency and functional reliability: a model may read as competent while remaining unable to complete a multi-step coding task correctly. This finding matters directly to the present study because it undermines the assumption that a single compact model, however capable, can reliably both generate and verify its own output.

The most heavily evidenced theme in the literature is the performance gap this limitation produces once correction is left entirely to further probabilistic generation. Controlled evaluations of compact models generating Java code under standard conditions report that syntactically invalid output occurs in approximately half of all cases, meaning failure is not an edge case but a near-coin-flip outcome for models operating without external verification. When correction is attempted purely through additional neural generation, without structured, rule-based feedback, self-repair loops plateau at a pass rate of roughly 25 to 30 percent on large benchmark datasets, because the same probabilistic mechanism responsible for the original error is also responsible for detecting it, reproducing the same blind spots on repeated attempts. Introducing a fixed, deterministic verification stage changes this outcome by an order of magnitude rather than a marginal improvement: hybrid repair pipelines that filter and refine generated candidates against rule-based checks report pass rates of 86 to 90 percent on compilation-driven benchmarks, with near-perfect compilation success achieved when evidence-based pre-filtering is added. This roughly threefold gap between neural-only and hybrid correction is the empirical foundation on which HorizonAI's deterministic-neural routing is built, and it directly justifies treating compilation as a hard precondition for acceptance rather than an optional refinement step.

This same body of evidence also reveals why the gap is frequently underreported in prior refactoring studies. Several widely cited studies report genuine quality improvements, including double-digit reductions in cyclomatic complexity, yet proceed from generation directly to quality assessment without first confirming that the refactored output compiles or preserves the original program's behavior. Read against the roughly fifty percent syntax-failure rate documented for compact models, this omission means a meaningful share of the improvements such studies report

may describe code that would not run at all, a methodological blind spot the field has not systematically addressed. HorizonAI's mandatory compilation gate and behavioral-equivalence check exist specifically to close this blind spot: a candidate is credited with improvement only after it has been proven structurally executable and behaviorally equivalent, not because it merely reads as improved.

A second, complementary theme establishes that decomposing a coding task across specialized agents produces measurable gains independent of the deterministic-gate question. Controlled ablation studies of multi-agent code systems consistently find that removing a single agent role from a coordinated pipeline reduces successful task completion by roughly 40 to 60 percent, evidence that specialized roles are functionally necessary rather than redundant. Three-role decompositions that separate planning, implementation, and verification responsibilities, structurally analogous to HorizonAI's own Planner-Generator-Judge pipeline, extend this finding by demonstrating that role separation reduces inter-role bias and enables feedback-driven refinement that single-agent systems cannot replicate. This advantage has also been demonstrated outside large-scale cloud settings, including in evaluations conducted under the same resource and infrastructure constraints that characterize the Philippine academic context this study targets, confirming that the benefit of role specialization does not depend on cloud-scale computational resources. What this body of literature does not establish, however, is a role-specialized system that also enforces a deterministic compilation gate: every surveyed multi-agent pipeline assumes general-purpose, cloud-connected models, and none tests whether role separation and hard syntactic validation can operate together under strict local hardware constraints. HorizonAI occupies precisely this intersection, combining the proven benefit of role specialization with the deterministic safeguard shown necessary in the preceding theme, a pairing the current literature has not tested.

A third theme, concerned less with correction strategy and more with deployment feasibility, establishes that the hardware ceiling this study imposes is not an arbitrary experimental choice but a documented real-world condition. Benchmark studies on consumer-grade devices confirm that 4-bit quantization is the enabling condition for running billion-parameter-class models on memory-limited hardware at

usable response speeds, while independent profiling studies on consumer GPUs show that sequential, rather than simultaneous, model loading reduces peak memory usage by 20 to 30 percent without degrading benchmark accuracy, since peak memory demand occurs during model loading rather than during inference itself. This is the specific mechanism the study's scheduling algorithm relies on to keep three compact agents within a shared memory budget. Documented infrastructure assessments of higher-education institutions in the Philippines further confirm that deficits in computing hardware and digital infrastructure are severe enough to make cloud-dependent AI tools structurally inaccessible as everyday resources for the computing faculty and students such institutions serve. Taken together, these findings mean that any performance gains identified in the preceding themes are only practically relevant to this study's target institutions if they can be achieved within a strict local memory ceiling, which is the condition this study's efficiency objectives were designed to test.

Read together, these four empirical and theoretical themes leave a specific, unoccupied position in the literature rather than a general one. The theoretical necessity of separating generation from verification is established. The performance advantage of deterministic validation over neural self-correction is established. The performance advantage of role specialization over monolithic generation is established. And the hardware feasibility of running multiple compact models under a strict memory ceiling is established. No reviewed study, however, combines all four: a role-differentiated, multi-agent pipeline that treats deterministic compilation as a hard precondition rather than a post hoc check, that does so using only quantized, sub-billion-scale-class compact models, and that operates entirely offline within a severely constrained memory budget. HorizonAI's proposed architecture, paired with a reasoning-transparency interface that makes each agent's routing decision auditable rather than opaque, is the direct response to this unoccupied position. It extends the proven reliability advantage of hybrid deterministic-neural governance into a deployment context, role-specialized, resource-constrained, and fully offline, that the literature has established as both necessary and, until now, empirically untested, giving the present study's objectives a clearly demonstrated evidentiary basis rather than a merely plausible one.

Definition of Terms

This section defines the conceptual and operational terms used in the study to ensure clarity and consistency in interpreting the framework, methodology, and findings. Conceptual terms refer to the principal technical concepts and standards underlying the study, while operational terms describe how specific components, metrics, and processes were applied within HorizonAI.

Conceptual Terms

Terms	Definitions
Abstract Syntax Tree (AST)	A tree-structured representation of source code in which each node represents a syntactic construct of the program, allowing code structure to be analyzed independently of its textual formatting [71].
Code Refactoring	The restructuring of existing source code to improve its internal structure while preserving its externally observable behavior [31].
Consumer-Grade Hardware	Computing hardware intended for general personal or workstation use rather than specialized data-center acceleration. In this study, this class of hardware is represented by a consumer laptop operating under a 4-gigabyte VRAM constraint [72].
Deterministic Validation	A rule-based verification process in which outputs are evaluated against explicitly defined structural or logical conditions, producing reproducible validation results independent of probabilistic model judgment [47].
Hallucination	The generation of plausible or fluent model output that contains factual, structural, or logical inaccuracies that are not supported by the actual program context or required constraints [56], [57].

Horizontal Intelligence	An architectural approach in which complex reasoning is distributed across multiple specialized agents or components instead of concentrating the complete task within a single monolithic model [34], [37], [40].
ISO/IEC 25010:2023	An international product quality model that defines characteristics for evaluating software quality, including functional suitability, performance efficiency, interaction capability, reliability, security, and maintainability [30].
Monolithic Model	A single-model architecture in which multiple cognitive functions, such as planning, generation, and evaluation, are performed within the same model or inference pathway rather than being distributed among specialized agents [73].
Multi-Agent System	A computational architecture composed of multiple autonomous or specialized agents whose individual roles are coordinated to accomplish a larger task through structured interaction and orchestration [35], [40].
4-Bit Quantization	A model-compression technique that represents model weights at reduced numerical precision to lower memory requirements and support inference on resource-constrained hardware while retaining usable model performance [74].
Small Language Model (SLM)	A relatively compact language model designed to perform language or reasoning tasks with substantially fewer parameters and lower computational requirements than large-scale language models, making it suitable for resource-constrained deployment [75].
Vertical Scaling	A method of enhancing AI functionality by adding more parameters to a single model, at the price of a significantly larger computational resource and infrastructure demands [1].

Operational Terms

Terms	Definitions
Behavioral Equivalence Rate (BER)	The proportion of compiled refactoring outputs that preserved the observable behavior of the original Java code by passing the required behavioral test cases. In this study, BER was used to evaluate whether the refactored output remained functionally equivalent to the original program.
CodeEditorBench	The benchmark dataset used to evaluate the Java code-refactoring performance of HorizonAI and the single-model baseline. In this study, 279 Java refactoring tasks covering multiple refactoring intents and difficulty levels were used for evaluation.
Compilation Success Rate (CSR)	The proportion of evaluated refactoring outputs that successfully compiled using the study's Java compilation procedure. CSR was used as an indicator of the syntactic executability of generated code.
Cyclomatic Complexity (CC)	A software metric representing the number of linearly independent paths through a program's control flow. In this study, CC was computed before and after refactoring to determine the change in structural complexity of the generated code.
Deterministic Validation Interception Rate	The proportion of benchmark runs in which at least one deterministic validation tier detected a condition requiring intervention before finalization. The validation process included checks related to syntax, cyclomatic complexity, structural boundaries, and intended refactoring constraints.

Dual-Loop Deterministic-Neural Routing (D2NR)	The bounded routing mechanism implemented in HorizonAI that combines deterministic validation with neural correction. When a generated candidate fails a defined validation condition, the system redirects the output for correction until the required conditions are satisfied or the permitted iteration limit is exhausted.
Generator Agent	A dedicated SLM object that is dedicated to the translation of the refactoring blueprint of the Planner into executable Java code. In this research, this role is played by Qwen-Coder 2.5 3B Instruct, which overwrites its output as it iterates when the validation layer gives out a compilation error.
Glassbox Interface	An interface that logs out and provides the developer with real-time feedback on each agent's line of reasoning and verification results. It maintains the developer as a proactive and knowledgeable reviewer who is able to audit or challenge the choices of every agent, as opposed to being given an end product.
HorizonAI	The locally deployed, role-driven multi-agent system developed in this study for Java code refactoring. It coordinates specialized Planner, Generator, and Judge roles with deterministic validation and sequential model execution under a constrained local-computing environment.
Inference Time	The elapsed execution time required to process a benchmark task under the tested architecture. In this study, it was measured in seconds and used to assess the computational efficiency of HorizonAI during local execution.
Iterative Feedback Resolution Rate	The proportion of refactoring attempts that initially required additional feedback or correction and subsequently reached an acceptable system outcome within the defined iteration

	limit. It was used to evaluate the effectiveness of HorizonAI's bounded retry mechanism.
Judge Agent	A specialized SLM instance, the refactored Java code is compared to the original input to verify that the program behavior has been maintained, and structural enhancements are legitimate, and a final ACCEPT or REVISE verdict is made. This role is performed by Llama 3.2 3B Instruct, which acts once all deterministic checks have passed to identify semantic regressions oracle to rule-based tools.
Maintainability Index (MI)	A composite software-maintainability metric derived from measures including Halstead Volume, cyclomatic complexity, and lines of code. In this study, the change in MI between the original and refactored code was used to compare maintainability outcomes.
Multi-Agent Parameter Distribution	The allocation of model capacity and functional responsibilities across HorizonAI's specialized agents and distinct model sets. In this study, it was examined through the number of logical agents, model sets, total parameter capacity, maximum individual model size, and model execution requirements.
Peak GPU Memory Usage	The maximum amount of GPU video memory consumed during local execution of HorizonAI. It was measured to determine whether the system operated within the study's defined 4-gigabyte VRAM constraint.
Planner Agent	The specialized agent responsible for interpreting the user's Java refactoring instruction and producing a structured plan that guides the Generator in performing the requested transformation. In the current study, this role is played by Qwen-Coder 2.5 3B Instruct.

Single-Model Baseline	The comparison architecture used in the study in which a single larger language model performed the Java refactoring task end-to-end without HorizonAI's role-driven multi-agent orchestration and staged validation workflow.
Strict Sequential Inference Scheduling (SSIS)	The resource-management protocol implemented in HorizonAI that executes agent models sequentially rather than simultaneously, allowing GPU memory to be released and reassigned between model stages to support operation within the 4-gigabyte VRAM constraint.

CHAPTER III

RESEARCH METHODOLOGY

In this chapter, the methods and procedures employed for this study are presented. It includes research design, research locale, study respondents, data gathering procedure, ethical considerations, and statistical treatment of data.

Research Design

The research design employed in this study was a quantitative research design involving a combination of a computational comparative method and a descriptive evaluation method. Since the study involved quantifiable outputs such as GPU memory usage, inference time, cyclomatic complexity, and maintainability changes, the researchers used this quantitative design to generate comparable and objective data.

In the comparative aspect, the performance of two different system architectures was evaluated under the same strictly offline, resource-constrained hardware conditions. This arrangement was selected to determine whether differences in system design—specifically, horizontal multi-agent orchestration versus a monolithic baseline—affected the system's refactoring performance. By comparing these methods, the performance and efficiency issues described in the Statement of the Problem were addressed through standardized computational benchmarking.

To supplement the technical benchmarking, descriptive evaluation techniques were incorporated to evaluate software quality and user acceptability. Since performance metrics alone could not fully capture real-world system behavior, the Glassbox interface was evaluated through direct feedback from end users and IT experts based on the quality characteristics specified in the study. This descriptive layer was used to ensure that the acceptability and quality of the system are formally assessed without relying on long-term longitudinal observations.

Combining both techniques, the methodology of the study was in accordance with current evaluation practices in AI-based software engineering, as Hou et al. [4] underline in their systematic review. Technical metrics alone are insufficient to evaluate language models, and to achieve sound validation, computational execution metrics were combined with human-centered evaluation to assess the system's practical performance and acceptability.

Research Locale

The study involved respondents from the University of Cabuyao (Pamantasan ng Cabuyao), St. Vincent College of Cabuyao, and Westbridge Institute of Technology, Inc., specifically from their computing-related programs. These institutions were selected as sources of end-user respondents because their computing-related programs provided access to students and faculty members with relevant programming backgrounds for the system evaluation.

The selection of these places was not primarily due to their role as academic institutions, but to the type of respondents available in this setting, as detailed under Respondents of the Study. Accordingly, these respondents served as representative end users of the developed system, like entry-level software developers who might use such systems in actual development work. The computational evaluation was performed using the consumer-grade hardware configuration defined in the study, while the three academic institutions served as sources of end-user respondents.

Furthermore, involving respondents from the three institutions broadened the context of the end-user acceptability evaluation. In local setups where cloud API costs were a burden, user trust and satisfaction heavily depend on whether a tool could reliably run on standard laptops. Evaluating the Glassbox interface with respondents from these institutions provided user feedback from computing students and faculty members within the study's intended academic context.

Respondents of the Study

Based on what they knew and what they did in software development and system evaluation, the respondents in this study were divided into two main types.

The first group comprised ten (10) IT experts. Each IT expert had at least three (3) years of relevant professional or academic experience in software development, system design, artificial intelligence, or related computing fields. These respondents evaluated the system's functional suitability, performance efficiency, reliability, and maintainability based on the ISO/IEC 25010 Product Quality model.

The second group was composed of twenty-five (25) computing students and faculty members from the University of Cabuyao, St. Vincent College of Cabuyao, and Westbridge Institute of Technology, Inc. Specifically, the student respondents were currently enrolled in a computer programming course, and the faculty respondents were currently teaching the same course, within their respective institutions at the time of data collection. This shared course context ensured that all end-user respondents, regardless of role, were actively engaged with the same programming curriculum relevant to the Java refactoring workflow under evaluation. Serving as end-user evaluators, they had basic to intermediate exposure to Java programming. They interacted with the system's Glassbox interface and evaluated its interaction capability, functional suitability, security, and reliability based on ISO/IEC 25010:2023.

The study used purposive sampling, a non-random sampling method, to select respondents whose qualifications and backgrounds were relevant to the system evaluation [76], [77]. For the computing students and faculty members, purposive sampling ensured that the end-user evaluation involved respondents with relevant computing and programming backgrounds. For the IT experts, purposive sampling ensured that the evaluators met the technical qualifications required to assess the developed system.

Table 2 presents the distribution of the twenty-five (25) computing students and faculty members and ten (10) IT experts who participated in the system evaluation.

Table 2. Distribution of respondents by classification

CLASSIFICATION	NO. OF RESPONDENTS
Computing Students and Faculty	25
Technical / IT Experts	10
TOTAL	**35**

Data Gathering Procedure

Data collection proceeded in two coordinated phases. The computational benchmarking phase was conducted first, in which the multi-agent system and the single-model baseline processed the identical set of CodeEditorBench Java snippets under the automated Python suite, with telemetry logged for each run. Following the completion of benchmarking, the human evaluation phase was conducted, in which the ten (10) IT experts and twenty-five (25) computing students and faculty members were briefed on the study's purpose, provided informed consent, interacted with the Glassbox interface, and completed their respective ISO/IEC 25010-based questionnaires

Instrumentation

The primary computational instrument used in this study was a customized Python benchmarking suite designed to execute Java refactoring tasks using the CodeEditorBench dataset. This automated suite functioned as the central monitoring tool during the comparative evaluation, systematically recording local inference time and peak GPU memory usage. By utilizing these scripts, the researchers tracked whether the multi-agent orchestration operated within the defined 4-gigabyte VRAM constraint under the Strict Sequential Inference Scheduling (SSIS) protocol, ensuring consistency and eliminating manual recording errors across all experimental runs.

To capture human-centered data, two distinct structured questionnaires adapted from ISO/IEC 25010:2023 were utilized. The first questionnaire was administered to ten (10) IT experts using a four-point Likert scale to evaluate the system's functional suitability, performance efficiency, reliability, and maintainability. The second questionnaire was administered to twenty-five (25) computing students and faculty members to evaluate the Glassbox interface. This questionnaire evaluated the Glassbox interface in terms of interaction capability, functional suitability, security, and reliability.

Validation

To ensure technical rigor, an internal technical evaluation was conducted as part of the system testing and benchmarking process. The system was evaluated using Java refactoring tasks from the CodeEditorBench dataset. This phase evaluated the system using Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change. This evaluation was used to examine how the system's validation and routing mechanisms intercepted invalid conditions before output finalization.

Regarding the human-centered evaluation instruments, initial validity was inherently established through their direct adaptation from the internationally recognized ISO/IEC 25010:2023 software quality models. Because the core constructs are already standardized, the questionnaires underwent external expert review by a qualified computing faculty from the University of Cabuyao and two external IT experts. This review was conducted to ensure that the adapted items were clearly articulated and relevant to the study's evaluation objectives, guaranteeing that the items appropriately represented the intended evaluation characteristics.

Discussions on Algorithms/Mathematical Concepts Used/Proposed Solutions

Fundamentals of the Algorithms/Mathematical Model/Formula

The developed computational framework was based on a role-driven and sequential multi-agent architecture designed for fully offline execution. This approach is essentially an automated software development team with various entities involved in the process of planning, writing, and reviewing code. According to Nguyen and Nguyen [78], artificial intelligence systems were usually unreliable in their outputs, especially when they were implemented on small language models on local computers. This issue arises when one model was required to plan, generate, and evaluate its own output. Sharma et al. [79] described that the model might generate incorrect syntax or flawed logic, yet still treat it as valid and present errors as correct to the system.

Rather than using a single model to process a task all the way through, this platform addressed these issues by breaking down the task into clear stages and distributing the workload across three specialized roles: a Planner, a Generator, and a Judge. Crucially, the Orchestrator utilizes the SSIS protocol to maximize the parameter count of each individual agent within the constrained 4-gigabyte VRAM. This design allowed each active model to use the available GPU memory without requiring simultaneous residency of the other agent models. While their cognitive tasks were divided, the Planner and Generator shared the identical underlying weights of the Qwen-Coder 2.5 3B model to eliminate semantic friction. Conversely, the Judge role was fulfilled by a 4-bit quantized Llama 3.2 3B model. That specific model was selected to perform a technical cross-examination because its generalized reasoning architecture provided an independent, rigorous logic review of the Generator's output, ensuring the system did not validate its own biases.

According to Gu et al. [80], this design was consistent with the concept of divide-and-conquer, in which the system splits the overall refactoring problem into planning, code generation, and evaluation, as opposed to solving it at once. Each stage focused on a specific part of the problem, allowing the system to handle tasks

more systematically and reduce the risk of compounded errors. Sun et al. [71] clarified that isolating such responsibilities ensured that errors introduced in one stage can be detected and corrected in the succeeding stage rather than being carried forward without intervention.

Moreover, the technological architecture of the model was premised on clearly defined inputs, outputs, variables, and parameters. To manage resource exhaustion, the entire architecture is mathematically governed by the Strict Sequential Inference Scheduling (SSIS) protocol. SSIS served as the core memory management formula, explicitly commanding the Agent Service to load, execute, and unload only one model at a time. The inputs consisted of a Java code snippet that required refactoring, paired with a natural language instruction that described the intended improvement. The system operated under several parameters, including a strict 4-gigabyte video memory limit and a maximum iteration constraint that controlled how many times the system could revise the code. During execution, the system tracked dependent variables such as the cyclomatic complexity of the original and generated code, the result of the parsing check, and the current iteration count. Lastly, the outputs consisted of generated Java refactoring candidates together with their corresponding validation and structural metric results. Candidate outputs were evaluated for compilation success, behavioral equivalence, cyclomatic complexity change, and maintainability index change.

To explain this process, consider a scenario where the system is given a highly nested Java method along with the instruction: "*Simplify this code by extracting reusable functions*." The Planner first analyzed the raw code and produced a step-by-step strategy for decoupling the nested structures without generating actual syntax. This plan was then read by the Generator, which produced modularized Java methods based on the plan. Before the code was finalized, the system executed the Dual-Loop Deterministic-Neural Routing (D2NR) algorithm. In the first loop, AST-Constrained Semantic Alignment acted as a rapid circuit-breaker, mapping the generated code against rigid Abstract Syntax Tree (AST) rules to deterministically trap syntax errors. If an AST constraint was violated, D2NR routes the validation feedback back to the appropriate correction stage within the defined iteration limit.

Only after the required deterministic validation conditions were satisfied does the system proceed to the Judge evaluation stage. If the Judge detected deeper logical flaws, it flagged the error stack and passed it back to the Planner to revise the code in a new iteration.

Theoretical Foundations

The developed framework was grounded in distributed intelligence as its architectural foundation. According to Tran et al. [81], the **Theory of the Society of Mind**, originally proposed by Marvin Minsky, had recently been adapted to organize modular cognitive tasks in multi-agent large language models, suggesting that intelligence arose from the structured interaction of multiple simpler and specialized agents rather than a single perfect computational system. In contemporary software engineering, even the use of a single model to plan, generate, and verify code could be unreliable. Sharma et al. [79] noted that a model might introduce errors and fail to recognize them, a behavior commonly referred to as sycophantic validation, because the same model was responsible for both producing and evaluating its own output.

The study addressed these problems by applying a clear separation of tasks through a Planner–Generator–Judge pipeline. The separation of stages, according to Tran [60], eliminated the overlap between planning and execution, which often caused instability and logic blending in single-model systems. The Planner focused on analyzing the problem and outlining a solution. The Generator was responsible for producing the actual code. Finally, the Judge evaluated the result, making the workflow more controlled and reliable. According to Gu et al. [80], this design was also based on the concept of **Divide-and-Conquer**, where the system breaks down the refactoring problem into smaller and manageable subproblems instead of solving it in a single step. While the Planner and Generator acted as specialized mathematical roles, the Orchestrator assigned the exact same quantized Qwen-Coder 2.5 3B weights to both phases. The system handled planning, generation, and evaluation independently, reducing the cognitive load on the active context window during any single inference pass. It allowed errors introduced in one stage to be

detected and addressed in subsequent stages, reducing the possibility of unchecked error propagation within the workflow.

In addition to distributed cognition, the framework reflected the **Dual-Process Theory of Human Cognition**. Liang et al. [82] explained that this theory distinguishes two types of thinking: fast and intuitive processing (System 1) and slow and analytical reasoning (System 2). Within the framework, the probabilistic language-model components provided flexible generative processing analogous to System 1, while the deterministic validation mechanisms provided deliberate, rule-governed checking analogous to System 2. The Abstract Syntax Tree parser and related validation gates therefore served as the structured verification layer that evaluated generated outputs against explicit constraints. This validation is further reinforced through a loop that acted as a strict checkpoint in the pipeline, ensuring that any syntactic or logical errors were detected and corrected before progression. This combination allowed the system to balance flexibility and precision, ensuring that outputs were both plausible and verifiable.

The developed approach operated under several technical assumptions and constraints to function on consumer-grade hardware. Dettmers and Zettlemoyer [74], along with Nguyen and Nguyen [78], explained the key assumption that aggressive 4-bit quantization, which reduced model size to fit within local memory constraints, was assumed to retain sufficient model capability for the intended Java refactoring workflow while reducing memory requirements. At the same time, the framework had certain limitations, including orchestration overhead caused by sequential execution. Because of the strict 4-gigabyte video memory constraint, models must be loaded and unloaded one at a time rather than executed in parallel—a process formally governed by the Strict Sequential Inference Scheduling (SSIS) protocol. While this increased processing time and introduced latency, it served as the primary mechanism for maximizing the reasoning capability of the pipeline. The researchers engineered this "daisy chain" execution specifically to allow more complex, higher-parameter models to function on consumer-grade hardware that would otherwise be forced to run significantly smaller, less accurate architectures. Furthermore, Oueslati et al. [83] stated that there was a risk of compound reliability decay, where overall

system reliability may decrease due to cascading failures across agent handoffs when multiple stages with their own probability of success are executed sequentially.

To address these limitations, the framework applies the Dual-Loop Deterministic-Neural Routing (D2NR) algorithm as a checkpointing mechanism within the pipeline. Through this process, rigid AST constraints acted as the primary deterministic loop, intercepting syntax-related issues early and routing them for bounded correction before progression to the Judge evaluation. This early validation prevented issues from accumulating across agent transitions and supported more consistent and reliable outputs while maintaining practical constraints in terms of flexibility and computational resources.

Algorithm Design

The design was largely based on a divide-and-conquer way of breaking tasks down. Gu et al. [80] describe how, instead of feeding a raw refactoring request to a single language model and expecting a flawless output, splitting the problem into a sequence of isolated steps forces the artificial intelligence to perform focused micro-tasks, including planning, execution, and validation, one step at a time. This structural division addressed the problem of context pollution by ensuring that the model responsible for code generation was not confused by the complex logic required for planning the overall software architecture. Moreover, the algorithm was designed specifically for the physical hardware limitations of the deployment environment. The design strictly enforced the Strict Sequential Inference Scheduling (SSIS) protocol to prevent exceeding the 4-gigabyte Video RAM limit. The Orchestrator enforced SSIS by serializing model execution and allowing only one model instance to operate at a time. Planner and Generator reused the same Qwen-Coder model weights while clearing the active inference context between tasks, whereas transitions to a different model, such as the Judge, triggered the unloading of the current model before loading the next. This design choice prioritized model depth over parallel speed, ensuring that each task (planning, generation, or adjudication) was handled by the most capable model the hardware could support.

The workflow below outlines how a multi-agent architecture integrates with a deterministic validation layer. In this system, specialized AI agents collaborate to generate solutions, while the deterministic layer applies predefined rule-based checks to validate structural and syntactic conditions before output finalization.

Phase 1: Ingestion & Baseline

- **Step 1. Input:** The user submits a raw Java code snippet and a natural language instruction (e.g., "Clean up this nested if/else nightmare").
- **Step 2. Baseline Analysis:** The *javalang* tool performs orchestration-level structural parsing and identifies whether the code fell within the Class Unit, Method Unit, or Statement Unit. Separately, *javac* compilation is used during benchmark evaluation to determine Compilation Success Rate (CSR). The *lizard* tool calculates the baseline Cyclomatic Complexity (CC).
- *Outcome:* A mathematically grounded snapshot of the original codebase is established in memory.

Phase 2: The Strategy Block

- **Step 3. Intent Generation:** Under SSIS, the 4-bit Qwen-Coder 2.5 3B is loaded into VRAM to act as the Planner. It reads the user's natural language and maps it to the strictly defined Snippet-Level Enum List (e.g., FLATTEN_CONDITIONAL, EXTRACT_METHOD, EXTRACT_VARIABLE). It outputs the Intent Packet, defining the exact refactoring category and the strict target boundaries.
- **Step 4. Cognitive Reset (Prompt Engineering Isolation):** The active inference context is reset before the Generator phase while retaining the shared Qwen-Coder model weights in memory. This isolation is designed to reduce carryover from the Planner's preceding context into the Generator phase.
- **Step 5. Plan Generation:** The Planner receives the Intent Packet. It outputs the AST Modification JSON, detailing the exact sections to extract, replace, or insert.

Phase 3: Plan Execution

- **Step 6. Code Generation:** Retaining the Qwen-Coder model in VRAM but shifting its system prompt to act as the Generator, it receives the AST Modification JSON (The Plan) and the original Java code. Following the instructions, it generates the Candidate Java Code.

Phase 4: Deterministic Validation Routing (D2NR - Loop 1)

- **Step 7. Tier 1 Validation (Syntax Heal):** *javalang* attempts to parse the candidate code.
    - *IF FAIL (Syntax Error):* The system triggers the Inner Loop. The available parser error information is routed directly back to the Generator (Step 6) for another bounded correction attempt.
    - *Circuit Breaker:* If the Inner Loop exceeds 3 iterations, the syntax is deemed unrecoverable. The system breaks the Inner Loop and escalates to the Outer Loop (Step 5) with a "Syntax Unrecoverable" error.
- **Step 8. Tier 2 Validation (Structural Fix):** The candidate code goes through the three-checkpoint routing engine:
    - *Check A (Complexity):* Where required by the identified refactoring intent, does the refactored output satisfy the defined cyclomatic-complexity constraint?
    - *Check B (Global Boundaries):* Are all nodes outside the defined target scope preserved relative to the original AST?
    - *Check C (Intent-Specific Math):* Python reads the enum from the Intent Packet and runs the corresponding AST math (e.g., if EXTRACT_METHOD, verify AST1 Method Count = AST0 Method Count + 1).
    - *IF FAIL:* The system triggers the Outer Loop. Control routes back to the Planner (Step 5) with a "Strategy Invalid" error, forcing it to write a new AST Modification JSON.

Phase 5: Heuristic Adjudication (D2NR - Loop 2)

- **Step 9. Tier 3 Audit (Judge Evaluation):** SSIS unloads Qwen-Coder and loads the 4-bit Llama 3.2 3B into VRAM to act as the Judge. The Judge model evaluates the structural difference against the Intent Packet. It produces a structured evaluation rationale that examines relevant code differences and logical relationships to assess whether the generated candidate remains consistent with the intended refactoring and original behavior.
    - *IF REVISE (Logic Altered):* The system triggered the Outer Loop. The Judge's audit notes are passed back to the Planner (Step 5).
    - *IF ACCEPT:* The orchestration loop terminates successfully.
    - *Circuit Breaker:* If the maximum permitted outer-loop iterations are exhausted, the system terminates the refactoring attempt and records the corresponding controlled failure status.

Phase 6: Finalization

- **Step 10. Output Generation:** The system determines the final output based on the exit status of the orchestration loop.
    - *On Success:* The generated refactoring candidate, its structural metric changes, and the complete execution and validation trace (including the number of loops used) were recorded.
    - *On Abort (Max Iterations Reached):* To ensure edge case resilience, the system gracefully degrades. It records the corresponding abort status, retains the original code as the safe fallback, and stores the available validation and execution logs.

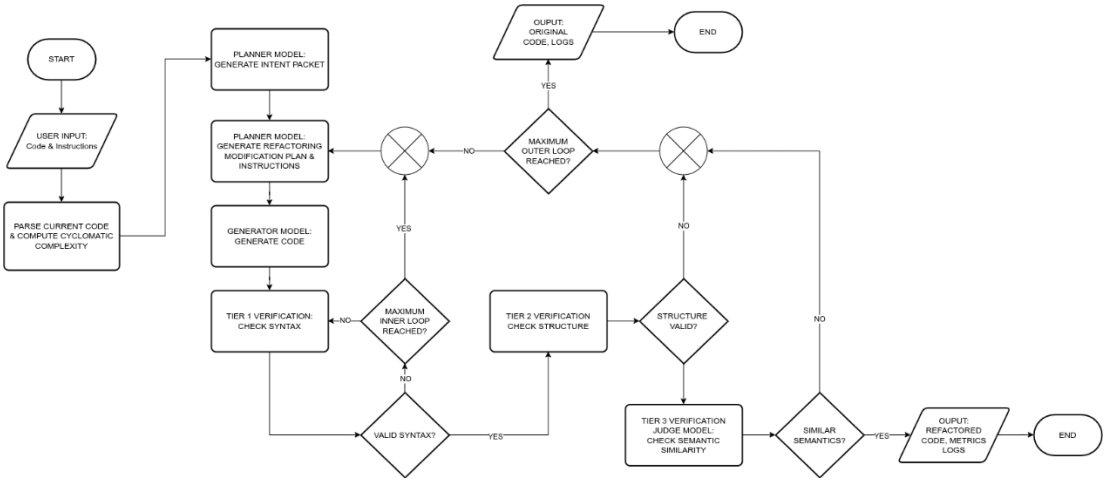


Figure 5. Algorithmic Workflow Flowchart

Figure 5 illustrates this algorithmic workflow, tracing the exact path from agent generation to final verification. This architecture provided a structured approach to improving control over probabilistic code generation by combining model-based generation with deterministic validation. The design imposed rigid role separation by structuring the multi-agent workflow as a sequential state machine instead of a free swarm, making planning, execution, and evaluation processes distinct and role-specific. Not only did this division avoid context pollution, but it also gracefully supported hard hardware constraints, reducing the risk of simultaneous model residency exceeding the available GPU memory. Moreover, to ensure that the AI-generated code was not merely heuristically plausible, it was checked against defined deterministic structural and syntactic constraints. This hybrid model was designed to address the gap between highly capable language model reasoning and hard software engineering standards, producing a refactoring pipeline designed to improve structural control, traceability, and resource-aware execution.

Mathematical Model Formulation

To systematically demonstrate how the multi-agent swarm executed code refactoring and mitigated errors, this section formalizes its operations into equations. The resulting model mirrors the framework's chronological pipeline to provide a clear roadmap for the reader. First, it defines the ingestion phase, mapping raw source

code into baseline syntax trees and complexity metrics. Building on this grounded state, subsequent functions quantify how the system translated natural language instructions into strict structural boundaries and generated candidate code. The formulation then details the core safety mechanisms, distinguishing between the iterative inner loops for syntax healing and the outer routing engine that enforced deterministic structural invariants. Finally, the model concludes with the semantic judge evaluation and the bounded objective function. Together, these mathematical constraints ensured that the architecture either produced a validated candidate output or safely reverted to the original baseline when validation requirements were not satisfied.

1. Source-to-AST Transformation (The Parsing Function)

The goal of this function was to map a character stream (C_0) into a hierarchical, directed rooted tree (A_0). In formal language theory, this was the transformation of a string into a derivation tree based on the Java Grammar (G_{Java}).

$$A_0 = \Psi(C_0, G_{Java})$$

Table 3. Variable and Parameter Definition of AST Parsing

Symbol	Definition	Description
A_0	AST Snapshot	The set of nodes V and edges E represents the structural hierarchy of the code.
Ψ	Parsing Function	The *javalang* implementation of the deterministic finite automaton for Java.
C_0	Input Stream	The raw string of Java source code submitted by the user.
G_{Java}	Java Grammar	The formal context-free grammar rules used to validate and tokenize C_0.

2. Complexity Quantification (The McCabe Function)

Cyclomatic Complexity (CC) was formally defined using graph theory. The code was treated as a Control Flow Graph (CFG) where nodes represented basic blocks and edges represented control flow paths.

$$CC(A_0) = E - N + 2P$$

Table 4. Variable and Parameter Definition of Cyclomatic Complexity

Symbol	Definition	Description
$CC(A_0)$	Cyclomatic Number	The number of linearly independent paths through the source code.
E	Edges	The number of flow transitions between nodes in the Control Flow Graph.
N	Nodes	The number of basic processing blocks (sequential statements).
P	Connected Components	The number of exit points (for a single method/unit, $P = 1$).

3. Intent and Plan Generation

The goal of this phase was to map natural language and raw source code into a structured intent, followed by a high-fidelity AST modification strategy.

$$I = (Act, Tgt) = \mathrm{P_{int}}(N, C_0)$$

$$Plan_i = Strat(I, A_0)$$

Table 5. Variable and Parameter Definition of Intent and Plan Generation

Symbol	Definition	Description
I	Intent Packet	The tuple (Agt, Tgt) defining the refactoring category and target boundaries.
Act	Action Enum	The classified refactoring operation.
Tgt	Target Scope	The strictly defined AST unit boundaries (Class, Method, or Statement).
$Plan_i$	AST Strategy	The modification JSON detailing exact node transformations
P_{int}	Intent Planner	The AI agent acting as an NLP classifier for intent mapping.
P_{strat}	Strategy Planner	The AI agent generating the technical modification plan.

4. Candidate Code Generation

The goal function mapped the modification strategy and the original source code into a candidate refactored codebase while allowing for iterative syntax healing.

$$C_{(i\,j)} = \left(Plan_i, \mathrm{C_0}, E_{syn}\right)$$

Table 6. Variable and Parameter Definition of Candidate Code Generation

Symbol	Definition	Description
G	Generator Model	The AI agent responsible for producing the candidate Java code/
$C_{(i,j)}$	Candidate Code	The code produced during outer loop i and inner loop j.
E_syn	Syntax Feedback	The exact parse error metadata used for the Inner Loop.

5. Deterministic Validation Routing (Tier 1 & Tier 2)

The system applied a dual-layered audit. Tier 1 ensured syntax recoverability, while Tier 2 enforced the 3 Vs through deterministic AST checkpoints.

Tier 1: Syntax Healing Function

$$V_{syn} = \begin{cases} 1, & \text{if parse}(C_{i,j}) \text{ is Successful} \\ 0, & \text{if parse}(C_{i,j}) \text{ yields } E_{syn} \end{cases}$$

Tier 2: The 3V Routing Engine

$$V_{val} = \begin{cases} 1, & \text{if } CC(A_{i,j}) \leq CC(A_0) \\ 0, & \text{if } CC(A_{i,j}) > CC(A_0) \end{cases}$$

$$V_{vic} = \begin{cases} 1, & \text{if } A(A_{i,j}, \backslash Tgt) \equiv A(A_0, \backslash Tgt) \\ 0, & \text{otherwise} \end{cases}$$

$$V_{ver} = \begin{cases} 1, \text{if } f_{Act}(A_{i,j}) = \text{True} \\ 0, \text{if } f_{Act}(A_{i,j}) = \text{False} \end{cases}$$

Orchestration Transition Logic

$$\text{State Transition} = \begin{cases} j \to j + 1, & \text{if } V_{syn} = 0 \wedge j < 3 \\ i \to i + 1, & \text{if } (V_{syn} = 0 \wedge j = 3) \vee (V_{val} \cdot V_{vic} \cdot V_{ver} = 0) \\ \text{Proceed}, & \text{if } V_{syn} \cdot V_{val} \cdot V_{vic} \cdot V_{ver} = 1 \end{cases}$$

6. Heuristic Judge Adjudication

The goal function evaluated semantic equivalence through a forced Chain-of-Thought audit.

$$V_{logic} = \mathcal{J}(C_0, C_{i,j}, I, R_{T2})$$

Table 7. Variable and Parameter Definition of Judge Adjudication

Symbol	Definition	Description
$\mathcal{J}$	Judge Model	The agent performing the Tier 3 audit using a CoT scratchpad.
R_{T2}	Tier 2 Results	The 3V validation metadata passed to the judge to ground the audit.
$\mathcal{J}$	Judge Model	The agent performing the Tier 3 audit using a CoT scratchpad.

7. Bounded Optimization (Objective Function)

The final output is the optimized candidate or a revert to the baseline upon breach of circuit breakers.

$$C^* = \begin{cases} C_{ij} \mid CC(A_{ij}) = \min(CC), & \text{if } (V_{syn} \cdot V_{val} \cdot V_{vic} \cdot V_{ver} \cdot V_{logic} = 1) \wedge (i < 3) \\ C_0, & \text{if } i = 3 \end{cases}$$

Table 8. Variable and Parameter Definition of Bounded Optimization

Symbol	Definition	Description
C^*	Final Output	The successfully refactored code or the safe baseline fallback.
i	Outer Index	Strategy/Logic fix attempts. Circuit breaker at 3.
j	Inner Index	Syntax healing attempts. Circuit breaker at 3.

Algorithm Implementation

The core architecture was implemented as a local, full-stack solution designed to function entirely within strict hardware environments using a Python 3.10 and TypeScript stack. To prevent Out-Of-Memory (OOM) exceptions on consumer hardware limited to 4 gigabytes of VRAM, the backend utilized the FastAPI framework alongside the llama-cpp-python library. This setup executed the Strict Sequential Inference Scheduling (SSIS) protocol to manage the 4-bit quantized GGUF models. Because loading a 3B parameter model consumed approximately 2.5 to 3 gigabytes of VRAM, the SSIS algorithm performed a strict "load-swap-unload" routine where the Qwen-Coder and Llama 3.2 models were never resident in the GPU memory concurrently.

While the CodeEditorBench dataset provided the raw Java code strings and refactoring instructions for the automated testing process, an independent SQLite database served as the persistence layer. This separation allowed the system to persist iteration histories, Abstract Syntax Tree (AST) states, and validation errors independently of the model-loading and unloading process. Consequently, the orchestration state was preserved while model resources were released from memory to make room for the next phase of the pipeline.

The frontend was constructed using Next.js 16 and Tailwind CSS 4.0, providing a reactive interface that monitored the offline orchestration process in real-time via REST and WebSocket connections. The user interface utilized Shadcn UI and Framer Motion to visualize the transition between the Planner, Generator, and Judge roles. Behind the scenes, the backend implemented the Dual-Loop Deterministic-Neural Routing (D2NR) using the javalang and lizard libraries to perform deterministic mathematical analysis on the code's AST. This architecture allowed the system to perform Java refactoring entirely through local computational resources without relying on external APIs.

To demonstrate how the SSIS protocol was realized in practice, the following Python snippet illustrates the memory-safe model instantiation and unloading process utilizing the llama-cpp-python bindings:

```
import asyncio
import gc
from pathlib import Path
from llama_cpp import Llama

MODELS_DIR = Path("/models")


class AgentService:
    """Handles lifecycle of Small Language Models. Optimised for 4 GB VRAM."""

    def __init__(self):
        self._model_lock = asyncio.Lock()   # Enforces sequential access — the core SSIS invariant
        self.model: Llama | None = None
        self.current_model_path: str | None = None

    # ── 1. LOAD ──────────────────────────────────────────────────────────────

    async def load(self, config: dict) -> None:
        async with self._model_lock:            # Blocks any concurrent caller
            path = str(MODELS_DIR / config["filename"])
```

```
        # Identity check — avoids VRAM churn when the same model is requested
        if self.current_model_path == path and self.model is not None:
            return  # Bypass I/O: model already resident in VRAM

        # Offload blocking C++ constructor to a thread pool; cap with a timeout
        self.model = await asyncio.wait_for(
            asyncio.to_thread(
                Llama,
                model_path=path,
                n_gpu_layers=config["layers"],
                n_ctx=config["context_size"],
                flash_attn=True,
                verbose=False,
            ),
            timeout=120,
        )
        self.current_model_path = path

    # ── 2. UNLOAD ──────────────────────────────────────────────────────────

    async def unload(self) -> None:
        async with self._model_lock:
            if self.model is not None:
                self.model.close()        # Release llama.cpp-level resources
                del self.model            # Drop the Python reference
                self.model = None
                self.current_model_path = None
                await asyncio.to_thread(gc.collect)   # GC in thread pool (non-blocking)
        # Outside the lock — lets CUDA driver unmap memory pages before next load
        await asyncio.sleep(0.5)

    # ── 3. SWAP ────────────────────────────────────────────────────────────

    async def swap(self, config: dict) -> None:
        path = str(MODELS_DIR / config["filename"])
        if self.current_model_path == path:
            return  # Same model already loaded — no-op
        await self.unload()
        await self.load(config)

    # ── 4. CLEAR CONTEXT ───────────────────────────────────────────────────

    async def clear_context(self) -> None:
        async with self._model_lock:
            if self.model is not None:
                # Purges KV-cache without evicting model weights from VRAM
                await asyncio.to_thread(self.model.reset)
```

Figure 6. Strict Sequential Inference Scheduling from AgentService Class

Figure 6 illustrates the four async primitives — load, unload, swap, and clear_context — that collectively govern the full VRAM lifecycle of a GGUF model within the AgentService class, with an asyncio.Lock enforcing strict sequential access at every boundary. By explicitly managing GPU memory states through this

programmatic structure, the implementation was designed to execute the selected 3B-parameter models within the study's defined local hardware constraints, with formal reproducibility findings presented in Chapter 4.

When the Python module was executed during testing, the terminal produced execution traces showing the sequence of agent and validation operations across all six phases. The execution trace on the next page presents a representative system log captured during a successful refactoring session, demonstrating the processing of a completed task from submission to finalization.

```
Input code:
public int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
User instruction: Refactor the recursive method into an iterative loop.

—— Phase 1: Baseline ——————————————————————————————————————
[Validator] Baseline: Structural analysis complete
            (Cyclomatic Complexity: 2, Lines: 4)

—— Phase 2: Strategy ——————————————————————————————————————
[Planner]   Strategy: Classifying intent (Strategy Iter 1)...
[Planner]   Intent Classified: REPLACE_LOOP_WITH_PIPELINE
[Planner]   Strategy: Analyzing code structure...
[Planner]   Structure analysis complete.
[Planner]   Strategy: Designing mutation plan...
[Planner]   Modification plan generated.

—— Phase 3: Execution ——————————————————————————————————————
[Generator] Execution: Implementing plan...

—— Phase 4: Validation ————————————————————————————————————
[Validator] Validation: Validating (Strategy 1, Syntax 0)...
[Validator] Syntax OK.

            {"syntax": {"passed": true}}

[Validator] Structural Checks — 3/3 passed

            {"checks": [
              {"name": "Cyclomatic Complexity", "passed": true,
               "before": 2, "after": 1},
              {"name": "Boundary Preservation", "passed": true},
              {"name": "Intent Match",          "passed": true}
            ]}

—— Phase 5: Adjudication ——————————————————————————————————
```

```
[Judge]     Adjudication: Running final audit...
[Judge]     Audit Finished: ACCEPT

—— Phase 6: Finalisation ——————————————————————————————————
[System]    Finalization: Finalizing session (Status: SUCCESS)...
[Judge]     Generating insights...

Output code:
public int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Figure 7. Sample Execution Trace of HorizonAI

As presented in Figure 7, the sample execution trace demonstrates HorizonAI's six-phase pipeline processing a refactoring task that converts a recursive Java factorial method into an iterative loop. The system progresses sequentially through baseline analysis, strategy planning, code generation, and validation — where all syntax and structural checks pass, including a reduction in Cyclomatic Complexity from 2 to 1. The Judge then issues an ACCEPT verdict in the adjudication phase, after which the session is finalized with a SUCCESS status. The trace illustrates the sequence of operations HorizonAI performs during a single refactoring session, from baseline analysis through the Judge's adjudication and final output, as an example of the pipeline's intended behavior rather than a formal evaluation result.

Complexity Analysis

The computational efficiency of the HorizonAI multi-agent orchestration framework was evaluated based on its time and space complexity during local execution. For this analysis, N represented the size of the input code snippet (measured either in abstract syntax tree nodes or contextual tokens), and W represented the static memory footprint of the loaded model weights. Table 9 presents the complexity of the developed system's multi-tiered algorithmic processes, including deterministic parsing, probabilistic generation, and bounded routing.

Table 9 summarizes how the computational complexity of the developed system was distributed across its major operations. The overall execution time was dominated by language-model inference, while the number of repeated inference cycles was bounded by the predefined orchestration circuit breakers.

Table 9. Complexity Analysis of the Developed System's Orchestration

OPERATION	TIME COMPLEXITY	SPACE COMPLEXITY
Baseline AST Parsing	$O(N)$	$O(N)$
LLM Inference (Transformer)	$O(N^2)$	$O(W+N)$
Structural AST Validation	$O(N)$	$O(N)$
Circuit Breaker Limits	$O(1)$	$O(1)$

The deterministic operations, specifically Phases 1 and 4, relied on the *javalang* and lizard libraries for baseline analysis and structural validation. Generating and traversing the Abstract Syntax Tree for structural checks was treated as linear with respect to the input size, resulting in a time complexity of $O(N)$.

Conversely, the probabilistic operations in Phases 2, 3, and 5 utilized the Qwen 2.5 and Llama 3.2 Transformer models for intent generation, code generation, and heuristic adjudication. For the standard self-attention operation, computational cost was modeled as quadratic with respect to the active token sequence length, resulting in a time complexity of $O(N^2)$. Unbounded retry loops can cause execution time to grow indefinitely when convergence is not reached. Because the maximum number of retry iterations was fixed, the circuit-breaker limit acted as a constant factor, bounding the number of repeated orchestration cycles.

Regarding space complexity, the memory footprint was engineered to operate strictly within the 4-gigabyte Video Random Access Memory (VRAM) constraints of the target edge hardware. The active model weights occupied a fixed memory footprint W relative to the input size and were therefore treated as $O(W)$ storage. As the models processed and generated tokens, the key-value (KV) cache grew with the active context length and was modeled as $O(N)$ relative to the token sequence. To prevent memory overflow during multi-iteration loops, the architecture enforced a

"cognitive reset" that actively cleared the context window between inferences. Consequently, the active memory requirement was modeled as $O(W + N)$, combining the fixed model-weight footprint with context-dependent memory growth. The SSIS protocol and context-reset mechanism were used to keep this memory usage within the study's defined hardware constraint.

System Development Methodology

The study utilized an Iterative Development Methodology to construct the framework. Rather than delivering the software in fragmented, incremental pieces, this cyclical approach focused on prototyping the entire orchestration pipeline early and subjecting the complete system to continuous cycles of evaluation and refinement. As noted by Martinez-Fernandez et al. [84], iterative strategies were highly effective for managing the inherent unpredictability of artificial intelligence.

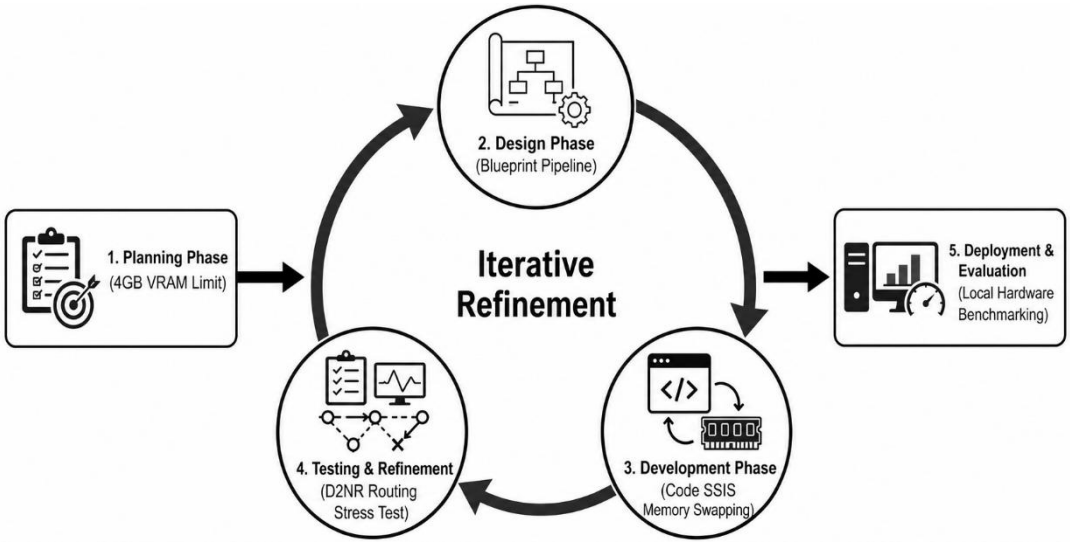


Figure 8. Software Development Lifecycle of the Developed System

As shown in Figure 8, the system's lifecycle begins with a Planning Phase targeting a strict 4GB VRAM limit before entering a continuous "Iterative Refinement" loop. This cyclical approach of designing pipelines, coding SSIS memory swapping, and stress testing was necessary to align the probabilistic nature of small language models with deterministic validation. By continuously tuning memory management

routines, this iterative method prevented hardware crashes and gradually improved overall orchestration logic. The stabilized system then advances to the Deployment & Evaluation phase for local hardware benchmarking.

a. Planning Phase

Rather than attempting to predict every technical detail upfront, the Planning phase was designed to establish the absolute hardware boundaries and evaluation metrics for the continuous iterations. During this stage, the researchers defined the required inputs—raw Java code fragments and natural language instructions—and the exact mathematical criteria for a successful output. The 4-gigabyte VRAM ceiling was identified as the primary hardware limitation, which immediately dictated the need for a sequentially executed pipeline. By establishing the cognitive roles of the Planner, Generator, and Judge early on, alongside defining the monolithic Qwen 2.5 7B baseline parameters, this phase provided the strict operational constraints necessary to ensure that every subsequent development cycle prioritized memory survival over unchecked feature expansion.

b. Design Phase

During the Design phase, the researchers translated the established hardware constraints into formal visual blueprints of the entire orchestration loop. Because their iterative development approach required testing the whole system rather than isolated parts, the researchers meticulously mapped the complete lifecycle of a refactoring request from start to finish.

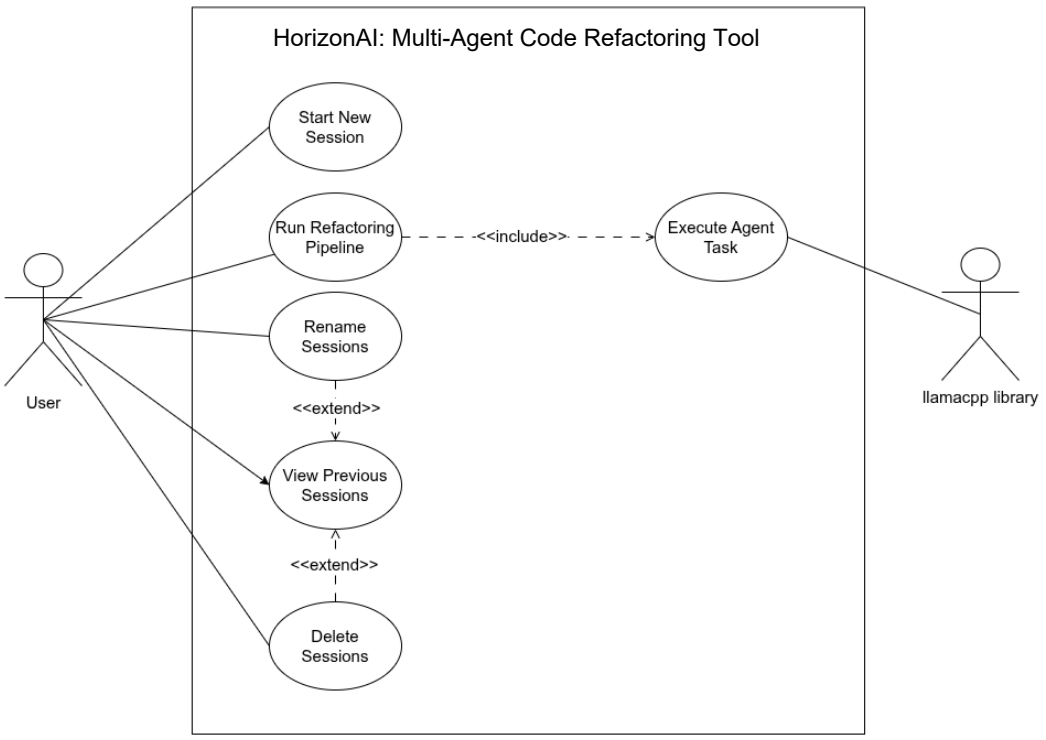


Figure 9. System Use Case Diagram

Figure 9 illustrates the System Use Case Diagram. The researchers designed this visual blueprint to explicitly define the functional boundaries between the human User, the internal HorizonAI system, and the supporting llama-cpp-python engine (llamacpp library). The primary workflow began when the User interacted with the system to initiate a new refactoring session, corresponding to the "Start New Session" use case, through which the system generated and maintained a distinct working session for the User's task.

Beyond session initialization, the User could directly invoke the "Run Refactoring Pipeline" use case to submit the target code for processing. This use case shared a strict <<include>> relationship with "Execute Agent Task," meaning that every invocation of the refactoring pipeline necessarily triggered the execution of the underlying agent task as a mandatory sub-process. The "Execute Agent Task" use case, in turn, interfaced directly with the llamacpp library, the external inference engine responsible for locally running the language models that power the system's

refactoring capabilities. This dependency reflects the researchers' design decision to rely entirely on localized AI models rather than external API calls.

In parallel with the refactoring workflow, the User was also given management functions over past sessions. The "Rename Sessions" and "Delete Sessions" use cases each carried an <<extend>> relationship with "View Previous Sessions," indicating that renaming or deleting a session were optional behaviors that could extend the base act of viewing historical session data, rather than being required steps within it. The User could likewise invoke "View Previous Sessions" directly and independently of the other use cases, allowing retrieval and review of past sessions and their associated results at any time, separate from the active refactoring pipeline.

To detail the dynamic flow of control identified in the Use Case modeling, the researchers constructed a series of Activity Diagrams to map the exact states of the system across different execution cycles.

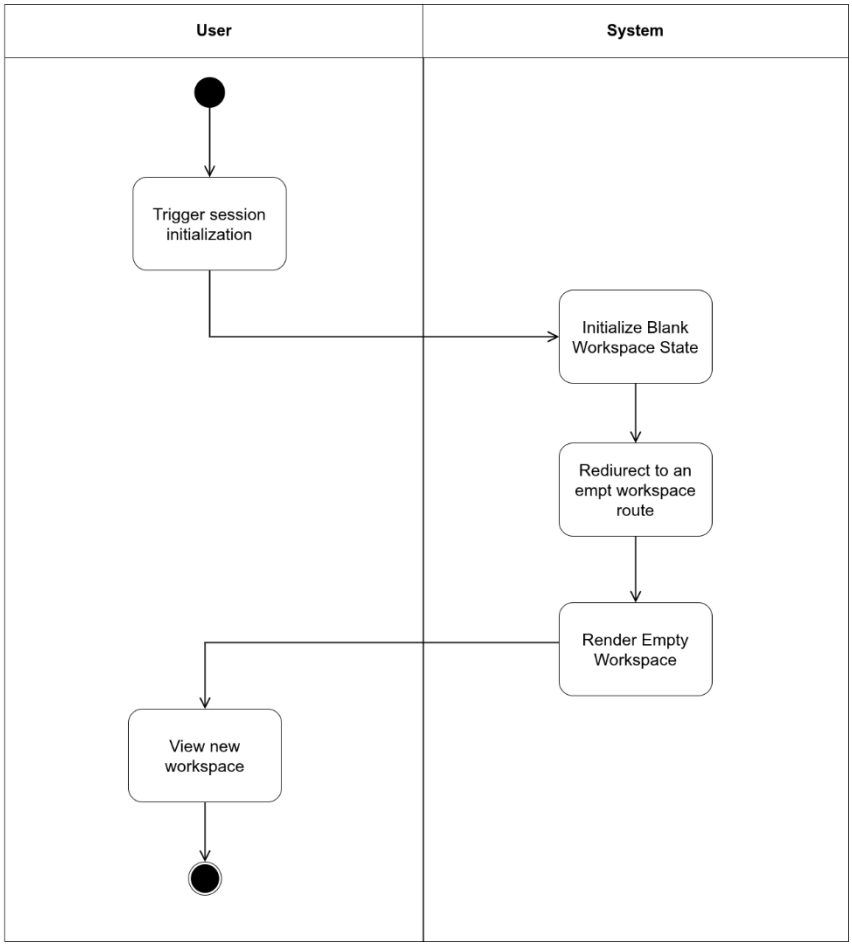


Figure 10. Activity Diagram for Starting a New Session

Figure 10 illustrates the Activity Diagram for Starting a New Session, mapping the swimlane workflow between the User and the System. The process began with the User triggering session initialization from the frontend interface, which prompted the System to initialize a blank workspace state as the internal foundation for the new session. Once this state was established, the System redirected the application to an empty workspace route and subsequently rendered the empty workspace layout accordingly. Control then returned to the User side of the workflow, where the newly rendered workspace became visible, and the activity concluded with the User viewing the new workspace, marking the end of the initialization process and leaving the system ready to receive the User's target code and refactoring instructions.

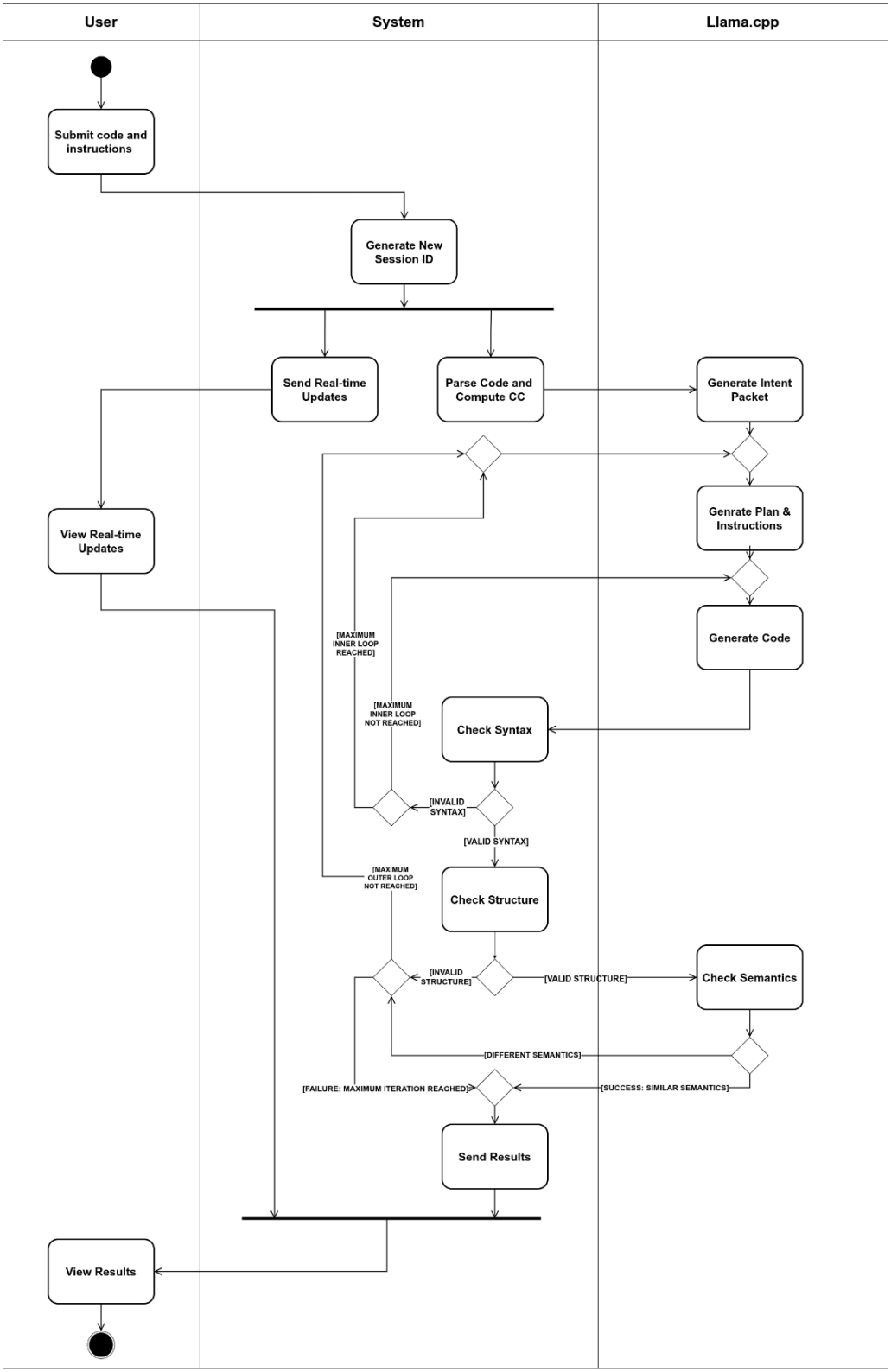


Figure 11. Activity Diagram for Running Refactoring Pipeline

Figure 11 illustrates the Activity Diagram for Running the Refactoring Pipeline, showing the swimlane interactions between the User, the System, and the Llama.cpp inference engine. The process began when the User submitted the target code and refactoring instructions, prompting the System to generate a new session ID to track the process. The flow then reached a synchronization (fork) bar, splitting execution into two concurrent threads.

The first thread handled real-time communication: the System sent real-time updates, which the User could continuously view on the frontend, keeping the User informed while the underlying computation proceeded independently in the background. The second thread drove the core refactoring logic. The System parsed the submitted code and computed its baseline cyclomatic complexity, then passed control to Llama.cpp, which generated an intent packet, formulated a plan with instructions, and generated the corresponding candidate code.

The generated code then passed through a series of deterministic validation checks handled by the System. First, the System checked the syntax of the generated code. If the syntax was invalid, the flow consulted an inner loop counter: provided the maximum inner loop limit had not yet been reached, the process looped back directly to code generation for another attempt; once the inner loop limit was exhausted, the System escalated by looping further back to regenerate an entirely new plan and instructions before generating code again.

Code that passed the syntax check proceeded to a structural check. An invalid structure triggered an outer loop check: if the maximum outer loop limit had not yet been reached, the System likewise escalated back to regenerate a new plan and instructions. Structurally valid code advanced to a semantic check performed against the original intent; if the semantics differed, the same outer loop condition was triggered again. This iterative cycle could only terminate under two conditions — either the semantics were confirmed similar, marking a successful outcome, or the maximum overall iteration limit was reached, marking a failure. Both outcomes converged into a single "Send Results" activity.

The two concurrent threads then rejoined at the bottom synchronization bar, after which the User was able to view the final results, concluding the refactoring session.

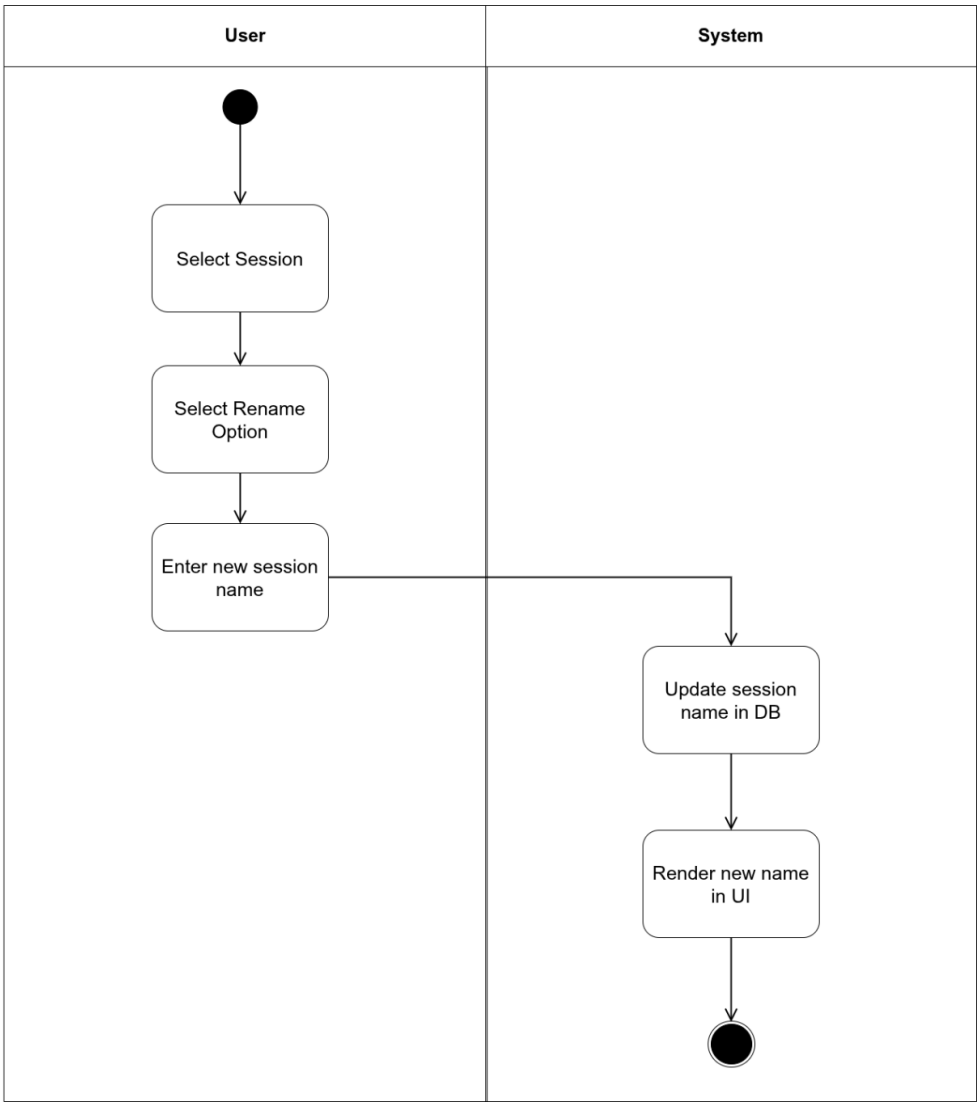


Figure 12. Activity Diagram for Renaming a Session

Figure 12 illustrates the Activity Diagram for Renaming a Session, showing the swimlane interaction between the User and the System. The process began with the User selecting an existing session, followed by selecting the rename option for that session. The User then entered a new session name, which was passed to the System for processing. Upon receiving the new name, the System updated the

session name in the database and subsequently rendered the updated name in the UI, concluding the renaming process.

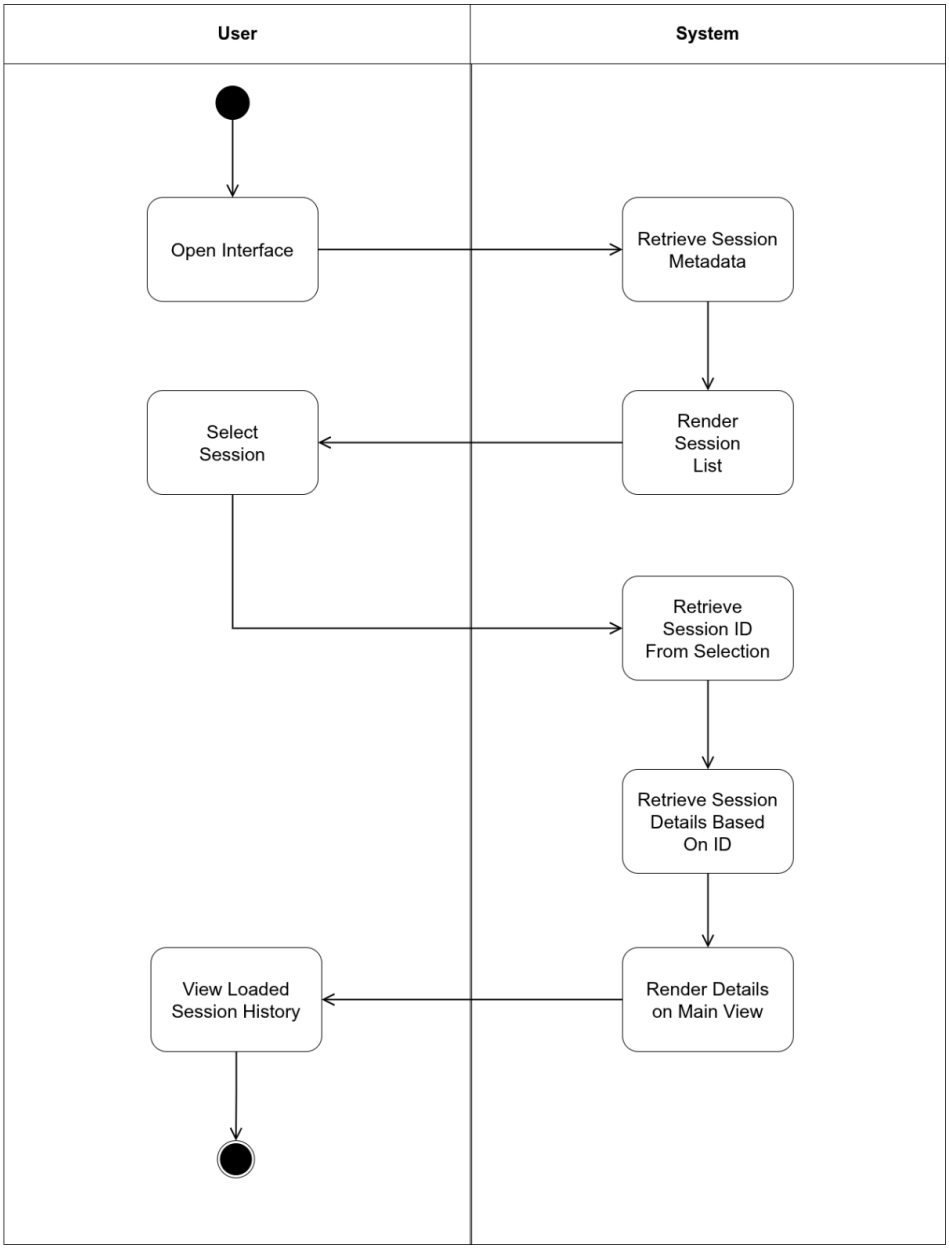


Figure 13. Activity Diagram for Viewing Previous Session

Figure 13 illustrates the Activity Diagram for Viewing a Previous Session, showing the swimlane interaction between the User and the System. The process began when the User opened the interface, prompting the System to retrieve the corresponding session metadata and render it as a session list for the User to view.

From this list, the User selected a specific session, which the System used to retrieve the associated session ID from the selection. The System then retrieved the full session details based on this ID and rendered these details on the main view. Finally, control returned to the User, who was able to view the loaded session history, concluding the process of retrieving and displaying a previously saved session.

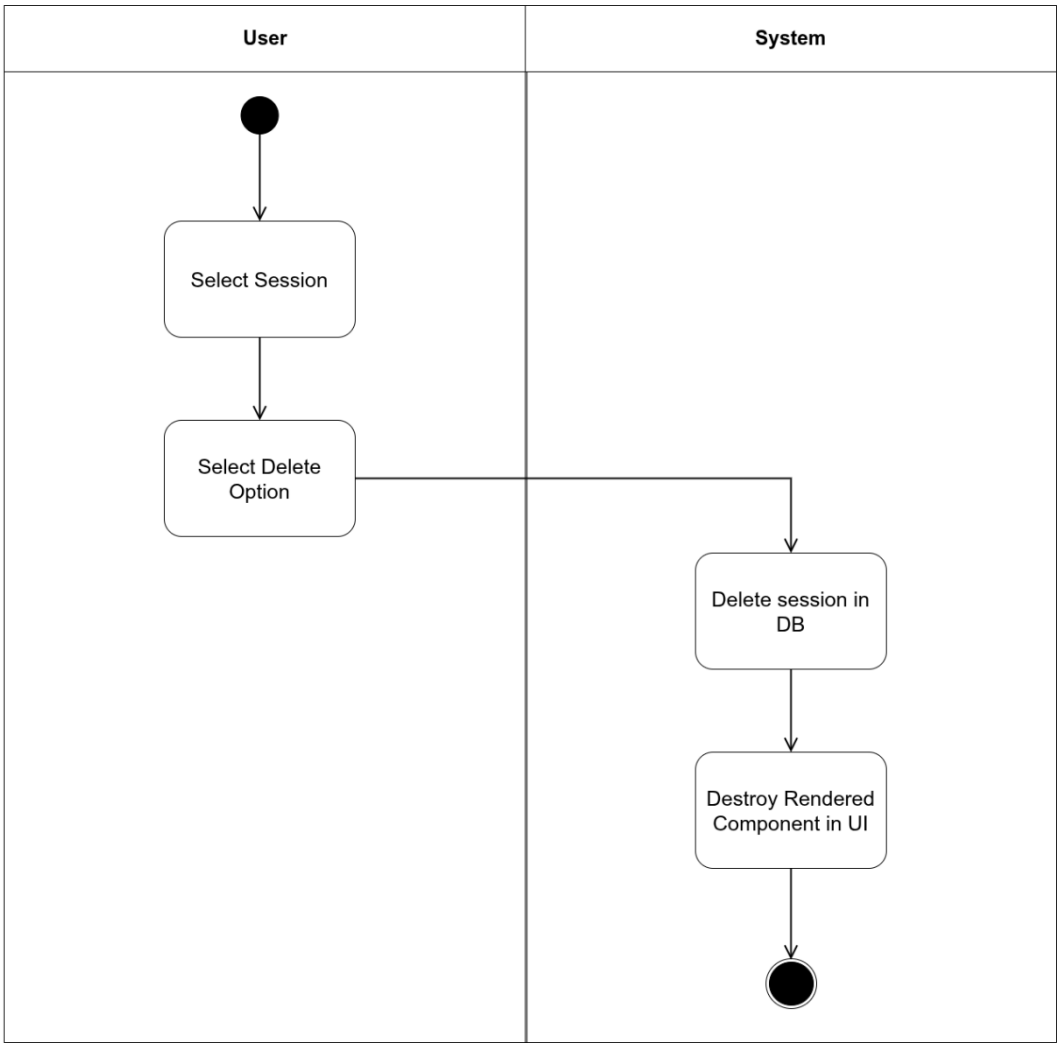


Figure 14. Activity Diagram for Deleting a Session

Figure 14 illustrates the Activity Diagram for Deleting a Session, showing the swimlane interaction between the User and the System. The process began with the User selecting an existing session, followed by selecting the delete option for that

session. This action was passed to the System, which proceeded to delete the corresponding session from the database. Once the deletion was completed, the System destroyed the associated rendered component in the UI, concluding the process of removing the session from both the persistent storage and the active interface.

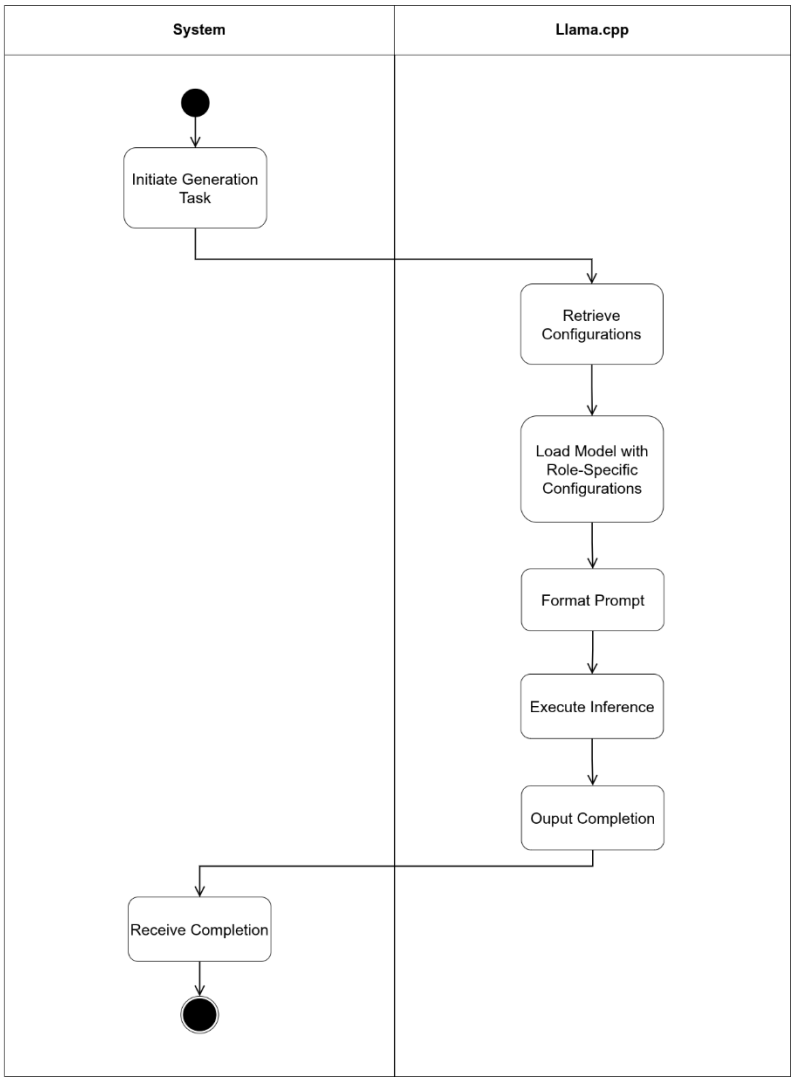


Figure 15. Activity Diagram for Executing an Agent Task

Figure 15 illustrates the Activity Diagram for executing an agent generation task. The researchers engineered this workflow to map the precise delegation of computational workloads between the internal system and the external llama-cpp-python engine. The execution cycle initiated within the internal system boundary

when it actively triggered a generation task. Control then transitioned explicitly to the external inference engine.

Upon receiving the command, the engine first retrieved the necessary operational configurations. It subsequently loaded the designated AI model into memory utilizing these role-specific settings. Once the model was securely loaded, the engine formatted the prompt payload and proceeded to execute the core inference process. After generating the required tokens, the engine output the final completion data and transferred control back across the boundary. The internal system received this generated completion, successfully concluding this specific task sequence and allowing the overarching orchestration pipeline to continue.

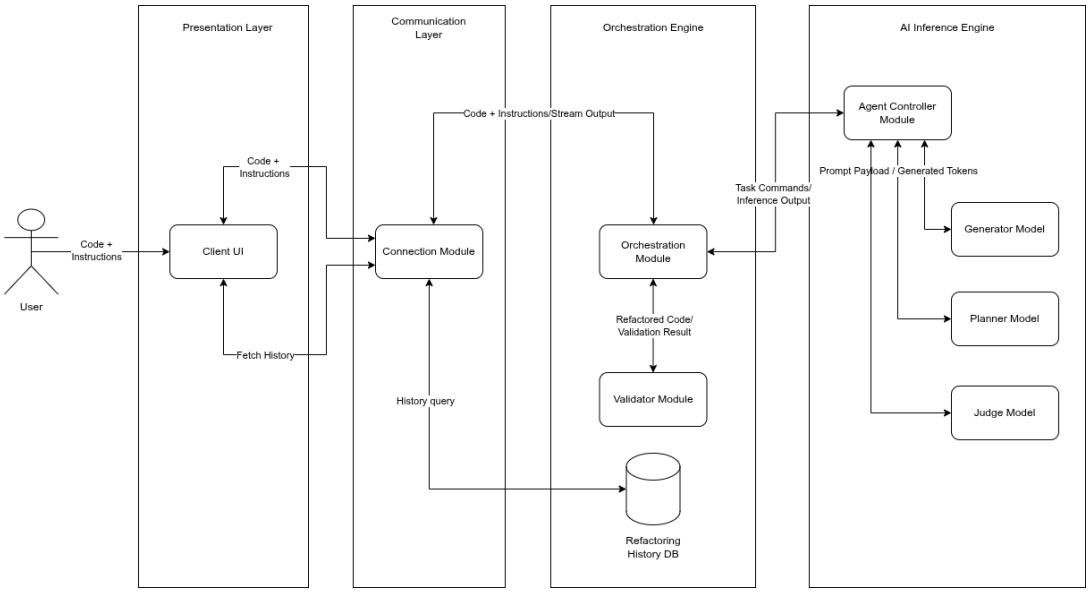


Figure 16. System Architecture Diagram

To finalize the Design Phase, the researchers constructed the System Architecture Diagram shown in Figure 16. This blueprint explicitly divided the framework into four distinct structural boundaries to securely separate computational concerns: the Presentation Layer, the Communication Layer, the Orchestration Engine, and the AI Inference Engine.

User interactions began in the Presentation Layer when an individual submitted code and instructions through the Client UI. These requests traveled into the Communication Layer and hit the Connection Module. Acting as a central router, this module handled two distinct data pathways. It forwarded active code and instructions to the Orchestration Module while streaming the output back to the interface. Alternatively, it could direct a history query straight to the Refactoring History DB to fetch past sessions for the client.

The Orchestration Engine served as the core logic center of the backend, housing both the persistent database and the primary orchestration logic. Inside this boundary, the Orchestration Module actively coordinated the internal data flow. It passed refactored code and received validation results back and forth with the deterministic Validator Module. To handle the neural processing, the Orchestration Module transmitted task commands across the boundary to the AI Inference Engine and waited to receive the inference output in return.

When a task reached the AI layer, it landed directly in the Agent Controller Module. The researchers engineered this specific controller to enforce the strict hardware limitations of the system. It actively governed the execution sequence by sending prompt payloads to the Generator Model, the Planner Model, and the Judge Model, subsequently collecting their generated tokens. This localized, targeted control guaranteed that the system properly orchestrated the distinct AI agents without overloading the available GPU memory.

c. Development Phase

In the Development phase, the researchers strictly avoided building isolated modules to piece together later. Instead, they adopted a highly iterative strategy with distinct operational goals for each cycle. The explicit goal of the very first iteration was to construct an end-to-end minimum viable pipeline. During this initial pass, the researchers wired the complete architecture from the user interface down to the local inference engine. This primary build served solely to establish a functioning baseline

and verify that the core software components could communicate without crashing the constrained hardware.

Once this skeletal pipeline was operational, the goal for subsequent iterations shifted entirely toward optimization and stability. In these later cycles, the researchers refined the prompt templates, hardened the error-trapping logic, and minimized the latency inherent in swapping heavy AI models. A primary focus of this full-system build involved programming the Strict Sequential Inference Scheduling (SSIS) protocol. To achieve this, the researchers wrote aggressive memory management routines in Python to ensure the system forcefully deallocated models from the VRAM prior to loading the next model.

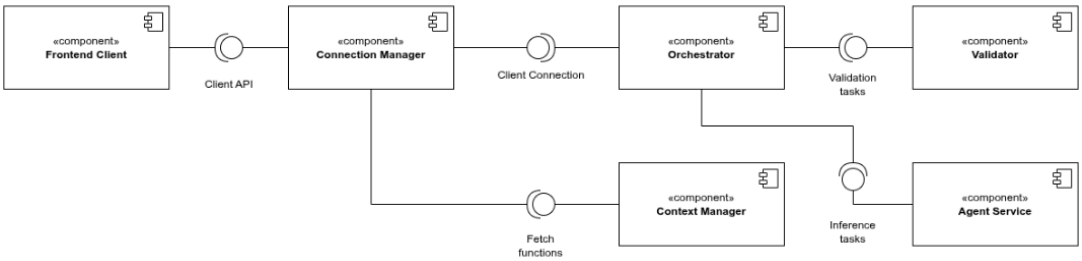


Figure 17. System Component Diagram

Figure 17 illustrates the resulting System Component Diagram and maps this decoupled execution. The central Orchestrator acted as a traffic controller, delegating tasks between the deterministic Validator and the Agent Service. Because the SSIS protocol constantly swapped neural models in and out of the GPU, the Context Manager acted as the crucial bridging component. It preserved the AI reasoning state and candidate code drafts in a persistent SQLite database while the system actively flushed the volatile hardware memory.

d. Testing and Refinement Phase

Because the researchers developed the system iteratively, testing was not reserved for the end of the project. Rather, the researchers repeatedly executed the entire end-to-end pipeline in continuous testing cycles. During these iterations, they monitored the volatile interactions between the Planner, Generator, and Judge under realistic hardware strain. The researchers continuously applied refinements to the

orchestration logic after each full-system pass to resolve memory leaks, adjust prompt templates, and eliminate sudden processing bottlenecks.

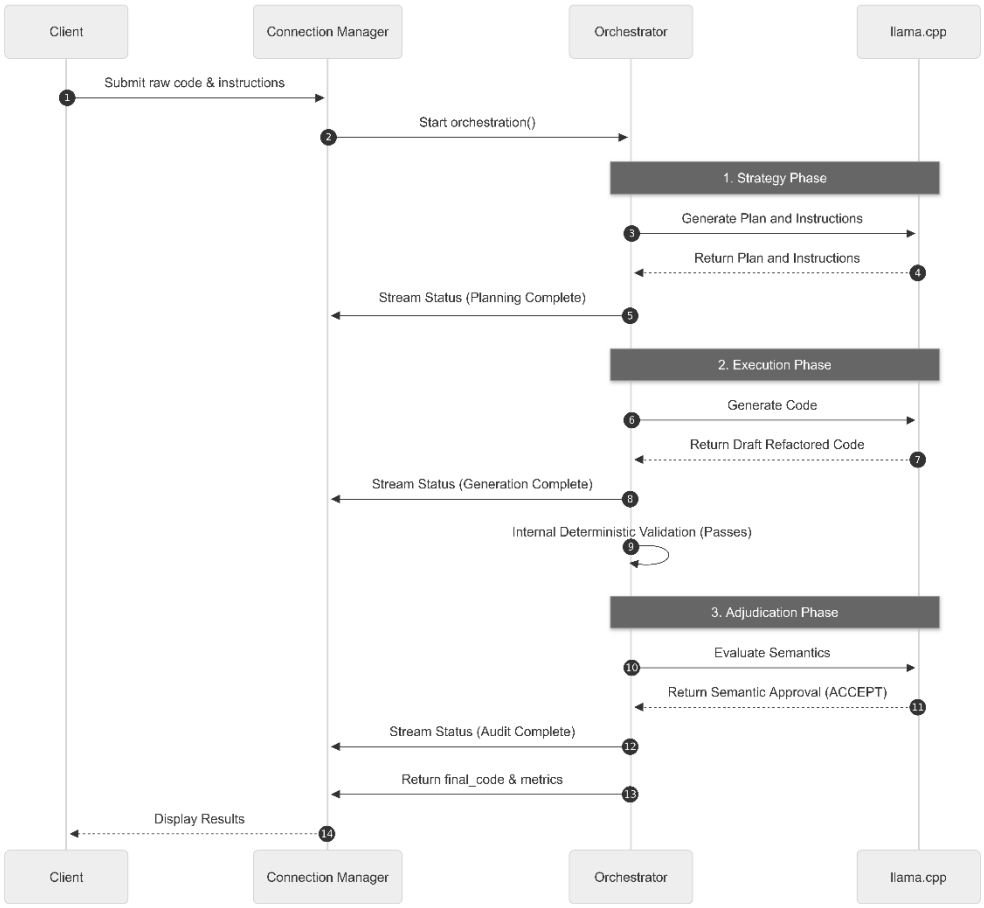


Figure 18. Sequence Diagram of Module Interactions

Figure 18 details the Sequence Diagram of Module Interactions, highlighting the exact chronological lifeline of a refactoring session stabilized during these rigorous cycles. It maps the asynchronous flow of the Orchestrator broadcasting real-time status updates to the client. Crucially, the researchers designed this diagram to chart the precise routing logic of the Dual-Loop Deterministic-Neural Routing (D2NR) mechanism. It explicitly illustrates that if the Validator detected a syntax failure, the sequence dynamically short-circuited. The Orchestrator routed the exact parser errors directly back to the Generator for rapid self-healing, entirely bypassing the

neural Judge to preserve computational overhead. This modeled sequence confirmed that the iterative cycles successfully produced a system that acted as an actively self-correcting state machine.

e. Deployment and Final Evaluation Phase

The Deployment phase represented the culmination of the researchers' iterative cycles, locking in the most stabilized version of the software for the formal study. The researchers deployed the fully offline multi-agent orchestration platform directly onto the targeted consumer-grade hardware. During this phase, the automated CodeEditorBench dataset benchmarking and the direct heuristic evaluations by the thirty (30) purposively sampled respondents were carried out. This phase yielded the rigorous empirical data necessary to validate the final iteration of the framework against the ISO/IEC 25010:2023 product quality standards.

Statistical Treatment of Data

The vast amount of data retrieved under the automated benchmarking processes and the human assessment surveys was analyzed strictly through statistical techniques. The researchers utilized these methods to draw empirical conclusions about the performance, efficiency, and acceptability of the system.

Population and Sampling

The population and sampling procedure for this statistical treatment followed the purposive sampling strategy detailed earlier under Respondents of the Study, comprising the same thirty-five (35) respondents: ten (10) Information Technology Experts and twenty-five (25) computing students and faculty members. Specifically, the twenty-five (25) end-user respondents were limited to students who were currently enrolled in a computer programming course and faculty members who were currently teaching the same course, within their respective institutions at the time of data collection. This shared course context ensured that all end-user respondents, regardless of role, were actively engaged with the same programming curriculum and terminology that the Glassbox interface presumes familiarity with, rather than being drawn from computing programs generally. This distribution ensured that the

statistical treatment applied to each data source, computational telemetry for the comparative benchmarking and survey responses for the descriptive evaluation, was drawn from a sample appropriately matched to its corresponding instrument.

Evaluation and Scoring

To convert system outputs and user responses into measurable data, the study employed a highly structured, two-pronged approach encompassing computational telemetry and standardized human assessment.

To assess the multi-agent system's benchmark performance against the single-model baseline, the researchers quantified the functional correctness of the outputs. Evtikhiev et al. [85] stated that judging code generation models by text similarity is insufficient; absolute functional correctness (the ability to compile and maintain original semantic behavior) is the true criterion of acceptance. Every Java snippet processed from the CodeEditorBench dataset was evaluated based on its Compilation Success Rate (CSR) and Behavioral Equivalence Rate (BER). Outputs satisfying both criteria receive a successful mark (1), whereas failures are rejected (0).

The resulting benchmark scores were calculated using the proportion of successful outputs relative to the total number of evaluated samples. The Compilation Success Rate (CSR) was computed using:

$$CSR = \frac{S}{N} \times 100$$

where S represents the number of successfully compiled outputs and N represents the total number of evaluated code snippets. Similarly, the Behavioral Equivalence Rate (BER) was calculated as:

$$BER = \frac{B}{C} \times 100$$

where B represents the number of outputs that successfully passed behavioral equivalence testing and C represents the total number of successfully compiled outputs. These measurements provide quantitative indicators of the functional

correctness and preservation of program behavior between the two evaluated architectures.

The study also relies on hardware telemetry to accurately measure efficiency during local execution. According to Zhang et al. [86], when artificial intelligence is applied to constrained local hardware, resource consumption becomes the most critical bottleneck. Consequently, the researchers recorded the inference time (in seconds) and peak GPU memory consumption (in megabytes) to determine whether the Strict Sequential Inference Scheduling (SSIS) could successfully operate within the 4-gigabyte VRAM limit.

Because the multi-agent framework (Group A) and the monolithic Qwen 2.5 7B baseline (Group B) process the exact same set of code snippets, their telemetry measurements are considered paired observations. For each benchmark task, the difference between the two systems was first computed using:

$$d_i = X_{M,i} - X_{S,i}$$

where: d_i represents the paired difference for task i;

$X_{M,i}$ represents the measurement obtained from the multi-agent framework;

$X_{S,i}$ represents the measurement obtained from the single-model baseline.

The average execution duration and peak GPU memory usage were summarized using the arithmetic mean:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

where: $\bar{x}$ represents the mean value;

x_i represents an individual observation;

n represents the total number of observations.

To compare these continuous measures, the researchers first applied a Shapiro-Wilk test for normality [87]. The Shapiro-Wilk statistic was calculated using:

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)}\right)^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

where W is the Shapiro-Wilk statistic, $x_{(i)}$ represents the ordered observations, $\bar{x}$ represents the sample mean, and a_i represents the weighting coefficients derived

from the expected values of normally distributed ordered samples. The test determines whether the distribution of paired differences follows a normal distribution. A significance level of 0.05 was used, where a p-value below this threshold indicates rejection of the normality assumption.

The analysis followed a decision rule based on the Shapiro-Wilk result: where the execution times and memory usages exhibited a non-normal distribution, a common occurrence in hardware processing metrics, the Wilcoxon Signed-Rank Test was applied in place of a paired t-test [88], [87]. The Wilcoxon Signed-Rank Test ranks the absolute paired differences and evaluates whether the median difference between the two systems significantly differs from zero. The absolute differences were ranked using:

$$R_i = rank(|d_i|)$$

The positive and negative rank sums were then calculated as:

$$W^+ = \sum R_i(d_i > 0)$$

$$W^- = \sum R_i(d_i < 0)$$

The final Wilcoxon test statistic was obtained through:

$$T = \min(W^+, W^-)$$

where R_i represents the rank of each absolute difference, W^+ and W^- represent the sums of positive and negative ranks, respectively, and T represents the Wilcoxon Signed-Rank statistic. This statistical comparison is used to determine whether the additional time introduced by sequential multi-agent execution represents a statistically significant difference in computational efficiency between the two architectures.

For the human evaluation, the study adopted the ISO/IEC 25010 System and Software Quality Model to guarantee stringent validation. Trichkova-Kashamova [89] highlighted that mapping evaluation questionnaires to specific ISO/IEC 25010 attributes is a recognized standard for evaluating AI-augmented software tools. The assessment is divided into two operational scenarios.

Table 10 presents the user-centric evaluation criteria used to assess the acceptability of the HorizonAI Glassbox interface based on the ISO/IEC 25010:2023 quality model. The evaluation focuses on the interaction between representative users and the system interface, particularly in terms of usability, functionality, security, and reliability during the Java code refactoring workflow. Each ISO/IEC 25010 quality characteristic was operationalized through its corresponding sub-characteristics, which served as the specific evaluation dimensions measured through the validated questionnaire items.

Table 10. User-Centric Evaluation Criteria (Glassbox Interface)

ISO/IEC 25010 QUALITY CHARACTERISTIC	SUB-CHARACTERISTIC	MEASUREMENT FOCUS
Interaction Capability	Appropriateness Recognizability	Measures how easily users recognize HorizonAI's purpose as a Java code refactoring tool and understand its intended function.
	Learnability	Measures the ease with which users learn the workflow, submit Java code, and understand the Glassbox interface.
	Operability	Measures the ease of controlling and navigating the interface during the refactoring process.
	Self-Descriptiveness	Measures how effectively the interface explains its functions, system status, and generated results.
Functional Suitability	Functional Completeness	Measures whether the system provides the necessary functions required to complete Java refactoring tasks.
	Functional Correctness	Measures whether displayed information, validation results, and outputs are accurate and dependable.
	Functional Appropriateness	Measures whether the workflow and provided functions appropriately support the user's refactoring objectives.
Security	Confidentiality	Measures user confidence that submitted code, generated outputs, and session information remain protected during processing.
Reliability	Fault Tolerance	Measures the ability of the system to handle failed refactoring attempts and maintain controlled operation.
	Availability	Measures whether the interface and generated outputs remain accessible throughout the evaluation process.
	Recoverability	Measures how effectively users can understand and recover from unsuccessful refactoring attempts.

Table 11 presents the evaluation criteria used by external Information Technology experts to assess the technical quality of the HorizonAI framework based on the ISO/IEC 25010:2023 Product Quality Model. Unlike the end-user evaluation, which focuses on interface interaction and acceptance, the expert assessment evaluates the underlying system architecture, algorithmic workflow, computational efficiency, reliability mechanisms, and maintainability of the framework.

Table 11. IT Expert Evaluation Criteria (Product Quality Assessment)

ISO/IEC 25010 QUALITY CHARACTERISTIC	SUB-CHARACTERISTIC	MEASUREMENT FOCUS
FUNCTIONAL SUITABILITY	Functional Completeness	Measures whether HorizonAI provides sufficient components and control functions for Java refactoring.
	Functional Correctness	Measures the accuracy of deterministic validation, routing behavior, and displayed system outputs.
	Functional Appropriateness	Measures whether the role-based workflow and validation design are appropriate for the intended refactoring task.
PERFORMANCE EFFICIENCY	Time Behaviour	Measures the acceptability of processing and inference time during local multi-agent execution.
	Resource Utilization	Measures the efficiency of model scheduling and GPU memory management under hardware constraints.
RELIABILITY	Faultlessness	Measures the stability of normal system operation without unexpected failures.
	Fault Tolerance	Measures the capability of the system to handle invalid outputs, iteration limits, and failed processes.
	Recoverability	Measures the ability of the system to preserve logs, states, and recoverable information after failed attempts.
MAINTAINABILITY	Modularity	Measures the separation and independence of system components.
	Analysability	Measures how effectively logs, reasoning traces, and validation outputs support system analysis.

	Testability	Measures whether system outputs and benchmark procedures support objective technical testing.

To summarize the responses obtained from both end-user and IT expert evaluations, the study utilized the weighted mean as the primary descriptive statistical measure. Each questionnaire item was assigned a numerical weight corresponding to the four-point rating scale. The weighted mean was computed for every evaluation item, aggregated to obtain the mean score of each sub-characteristic, and further summarized to determine the overall assessment of each ISO/IEC 25010 quality characteristic. The weighted mean was calculated using:

$$WM = \frac{\sum fw}{N}$$

where WM represents the weighted mean, f represents the frequency of responses, w represents the assigned numerical weight, and N represents the total number of respondents. The resulting scores were interpreted using equal-width intervals corresponding to the four-point evaluation scale. The response scale and corresponding mean interpretation are presented in Table 12.

Table 12. Response Scale and Mean Interpretation

Mean Range	End-User Rating	IT Expert Rating	INTERPRETATION
3.26–4.00	Highly Acceptable	Strongly Agree	Indicates that the system highly satisfies the evaluated criteria and demonstrates a strong positive assessment from respondents.
2.51–3.25	Acceptable	Agree	Indicates that the system satisfies the evaluated criteria and receives a generally positive assessment, with possible areas for improvement.
1.76–2.50	Unacceptable	Disagree	Indicates that the system does not sufficiently satisfy the evaluated criteria and requires improvement based on respondent assessment.
1.00–1.75	Highly Unacceptable	Strongly Disagree	Indicates that the system substantially fails to satisfy the evaluated criteria and requires major improvements.

Ethical Considerations

The research was conducted in accordance with established academic ethical standards to ensure the safety, privacy, and confidentiality of all respondents involved in the evaluation stages. Participation in the system testing and survey evaluation was voluntary. Prior to participation, all IT experts, students, and faculty members were provided with an informed consent form outlining the purpose of the study, the procedures involved, and their right to withdraw from participation at any stage without penalty. In cases where a participant chose to withdraw, any information they provided was excluded from the analysis, and replacement respondents were considered when necessary. The study involved minimal risk, which was limited to system interaction activities and completion of evaluation questionnaires.

To maintain confidentiality and anonymity, the data collection instruments did not collect any personally identifiable information. The questionnaires excluded names, identification numbers, contact information, and other personal identifiers. All responses gathered from the participants were recorded, processed, and analyzed only in aggregate form. The collected information was used exclusively for the purposes of this research.

All collected data were handled securely by the researchers throughout the study. Survey responses, evaluation results, and system testing records were stored in encrypted and password-protected local storage. Access to the collected data was limited only to the researchers during the data processing and analysis stages. After the completion and defense of the study, all collected research data were securely deleted in accordance with the established data management protocol.

CHAPTER IV
RESULTS AND DISCUSSION

This chapter presents the results of the development, testing, and evaluation of HorizonAI as a locally deployed multi-agent system for Java code refactoring. The findings provide empirical evidence on how the system operated under the study's resource-constrained setting, particularly the 4-GB VRAM limit on consumer-grade hardware.

The results cover the system's measurable algorithmic constraints, refactoring performance against a single-model baseline, and computational efficiency during local execution. The chapter also presents the acceptability assessment of the Glassbox interface by users and the technical evaluation of HorizonAI by Information Technology experts based on the selected characteristics of ISO/IEC 25010:2023.

The discussion interprets these findings in relation to the study's objectives and related literature. Particular attention is given to how the multi-agent workflow, deterministic validation, iterative feedback, and resource management contributed to the observed results, including areas where the system performed differently from the baseline or encountered limitations.

Overall, the findings provide the basis for determining how HorizonAI performed as a locally executable multi-agent approach to Java code refactoring within constrained computing resources.

System Features in Automating Java Code Refactoring

The first specific problem determined the system features of HorizonAI in automating Java code refactoring based on measurable algorithmic constraints. These were examined in terms of multi-agent parameter distribution, deterministic validation interception rate, and iterative feedback resolution rate. HorizonAI separates planning, code generation, and evaluation into defined processing stages instead of assigning the complete refactoring task to one model. Generated outputs

are also subjected to rule-based validation before proceeding to semantic evaluation, while detected failures may be returned for another controlled attempt. The system configuration and benchmark documentation were used to establish how model responsibilities and parameter capacity were distributed, while the benchmark execution provided the measured results for validation interception and iterative resolution.

Multi-Agent Parameter Distribution

Multi-agent parameter distribution describes how HorizonAI divides model capacity and refactoring responsibilities among its processing roles. The single-model baseline performs the complete refactoring task using one 7-billion-parameter model. HorizonAI follows a different arrangement. It divides the process among three logical roles composed of the Planner, Generator, and Judge, while these roles are supported by two distinct 3-billion-parameter model weight sets.

As shown in Table 13, HorizonAI uses three logical roles but only two distinct model weight sets. The Planner and Generator use the same Qwen2.5-Coder-3B-Instruct weights for different responsibilities, while the Judge uses Llama-3.2-3B-Instruct. The architecture therefore contains 6 billion combined parameters across its two distinct model weight sets, compared with the 7-billion-parameter single-model baseline. The largest individual model loaded during a processing stage is limited to 3 billion parameters. This distribution separates the refactoring responsibilities among multiple processing roles without requiring a different model weight set for each role.

Table 13. Parameter Distribution of HorizonAI and the Single-Model Baseline

Architecture Property	Single-Model Baseline	HorizonAI Multi-Agent System
Logical Agents	1	3
Agent Roles	Single end-to-end model	Planner, Generator, Judge
Distinct model weight sets	1	2
Combined Parameters Across Distinct Model Weight Sets	7B	6B
Maximum Individual Model Size	7B	3B

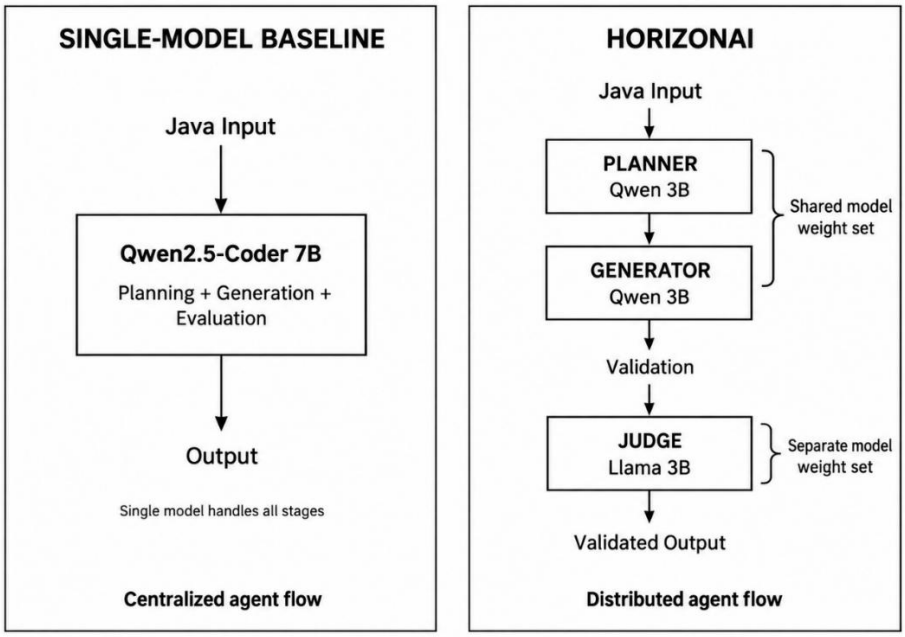


Figure 19. Parameter and Agent Role Distribution of the Single-Model Baseline and HorizonAI

Figure 19 illustrates the difference between the centralized agent flow of the single-model baseline and the distributed agent flow of HorizonAI. The baseline assigns planning, generation, and evaluation to a single Qwen2.5-Coder 7B model. In contrast, HorizonAI divides these functions among the Planner, Generator, and Judge. The Planner and Generator use the same Qwen 3B model weight set, while the Judge uses a separate Llama 3B model weight set. This arrangement separates the processing responsibilities while limiting each processing stage to a 3-billion-parameter model.

This arrangement is consistent with the literature discussed in Chapter II under Horizontal Intelligence and Agentic Decomposition. Jahrens and Martinetz [34] discussed the limitations of concentrating multiple reasoning responsibilities within a single monolithic model, while Fajardo [35] described structured multi-agent systems as an approach for dividing complex tasks into discrete and manageable roles. The empirical studies reviewed in the same chapter further support this division. Xu et al. [58] separated code rewriting and review in MANTRA, while Huang et al. [59] organized AgentCoder into distinct design, implementation, and testing responsibilities. HorizonAI applies a similar separation of responsibilities but within a

resource-constrained configuration by using two 3-billion-parameter model weight sets for three logical roles.

Overall, the parameter distribution established that HorizonAI implemented three specialized processing roles while restricting the largest individual model to 3 billion parameters. The use of two distinct model weight sets allowed the system to distribute refactoring responsibilities without requiring a separate model for each role. This configuration provided the architectural basis for executing the multi-agent workflow within the study's resource-constrained setting, while its actual computational cost was evaluated separately under the computational efficiency objective.

Deterministic Validation Interception Rate

The second indicator measured how frequently HorizonAI detected a generated candidate that required intervention before the refactoring process could proceed normally. Four deterministic validation checks were examined together with the subsequent Judge evaluation. Deterministic validation refers to checks governed by predefined program rules rather than language-model judgment. These checks evaluate syntax, cyclomatic complexity conditions, structural boundaries, and whether the generated transformation satisfies measurable requirements associated with the requested refactoring.

As shown in Table 14, Intent Math recorded the highest interception rate at 23.3%, corresponding to 65 flagged cases. Judge-related interceptions followed at 7.9%, with 22 cases. Cyclomatic Complexity and Boundary validation each accounted for 5.4%, with 15 cases each, while Syntax recorded the lowest interception rate at 3.2%, with 9 cases. These results show that the validation layer did not focus solely on syntactic correctness. Instead, generated candidates were also screened according to structural, complexity-related, and task-specific conditions before proceeding through subsequent evaluation and finalization.

Table 14. Validation Interception Rates of HorizonAI by Tier

Validation Component	Flagged Cases	Interception Rate
Syntax	9	3.2%
Cyclomatic Complexity	15	5.4%
Boundary	15	5.4%
Intent Math	65	23.3%
Judge	22	7.9%

The predominance of Intent Math shows that producing syntactically valid Java code was not the only condition considered by the system. A generated program may compile while still failing to carry out the requested structural change. For instance, code produced for a task requiring the extraction of a logic block into a separately declared helper method could remain executable even if the intended block was not actually transferred. Intent validation provides a means of checking whether the expected structural effect of the requested refactoring was present. Boundary checks serve a related purpose by determining whether the transformation remains within the intended portion of the source code.

Syntax validation serves a different function. Its interception rate was the lowest among the reported checks, but the lower frequency does not lessen its role in the pipeline. When malformed Java code is detected, the candidate should not proceed merely because the generated text appears reasonable. The syntax check provides a concrete condition that must be satisfied before later evaluation can be relied upon. Similarly, the Cyclomatic Complexity and Boundary checks provide measurable conditions for identifying transformations that do not satisfy the requirements of the requested refactoring.

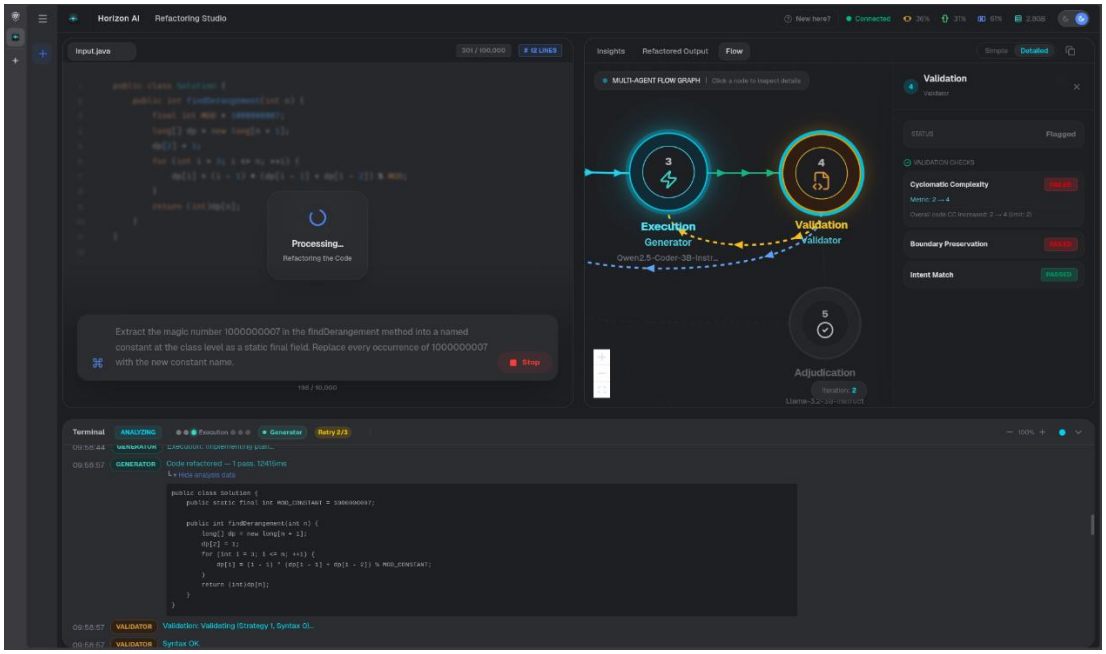


Figure 20. Actual HorizonAI Interface Showing a Validation Interception During Refactoring

Figure 20 shows an actual HorizonAI refactoring run in which the generated candidate was intercepted during the validation stage. The system marked the validation status as Flagged after the Cyclomatic Complexity and Boundary Preservation checks failed, while the Intent Match check passed. In the displayed case, the cyclomatic complexity increased from 2 to 4, exceeding the permitted condition for the current refactoring task. Rather than allowing the candidate to proceed directly to final adjudication, HorizonAI retained the output within the validation and feedback process. The interface also displays Retry 2/3, demonstrating that the intercepted result was subjected to another bounded processing attempt. This execution provides system-level evidence that a detected validation failure could affect the routing of a generated candidate during the refactoring process.

The finding can be related to the concerns discussed in Chapter II regarding the reliability of generated code. Golnari et al. [56] examined difficulties associated with smaller models producing syntactically valid code consistently, while Kim et al. [57] discussed how errors introduced in an earlier stage can propagate when

subsequent processing fails to identify them. In HorizonAI, the validation layer provided explicit checkpoints for detecting such conditions before the candidate proceeded through the remaining stages. This approach therefore placed deterministic checks between model-generated outputs and subsequent evaluation rather than relying solely on the generated response.

The result can also be compared with Li et al. [7], whose hybrid automated program repair approach combines language models with program analysis rather than relying entirely on model-generated correction. HorizonAI follows a comparable principle by applying deterministic checks before subsequent model evaluation. The difference lies in the specific conditions applied to the refactoring workflow, where HorizonAI combines syntax, cyclomatic complexity, boundary, and intent-related checks to intercept candidates that do not satisfy predefined requirements.

Overall, the interception results established that HorizonAI's validation layer actively screened generated candidates against multiple predefined conditions before finalization. Intent Math produced the highest observed interception rate at 23.3%, indicating that task-specific structural requirements accounted for more detected interventions than the other validation components in the benchmark. The remaining checks addressed syntax, complexity, boundary preservation, and subsequent Judge evaluation at lower observed rates. These results establish the interception behavior of the validation layer, while the extent to which intercepted cases were subsequently recovered through feedback and retry is examined separately under the iterative feedback resolution rate.

Iterative Feedback Resolution Rate

The third indicator measured the ability of HorizonAI to continue processing when an initial attempt did not satisfy the required conditions. Instead of treating every first output as final, the system could return a detected issue to the appropriate processing stage and perform another attempt. This process was bounded by defined retry limits so that unsuccessful cases did not continue indefinitely.

As shown in Table 15, HorizonAI reached a resolved state in 219 of the 279 benchmark cases, resulting in a system resolution rate of 78.5%. This resolution rate refers to reaching the system's defined exit condition and should not be interpreted as equivalent to successful functional correctness, which was separately measured through CSR and BER. The denominator of 279 represents the total number of benchmark cases evaluated. Of the 219 resolved cases, 177 were completed during the first processing pass, giving a first-pass success rate of 80.8% (177/219). This rate is calculated only among the cases that ultimately reached a resolved state and should therefore not be interpreted as the percentage of all 279 benchmark cases completed on the first pass.

Table 15. Iterative Feedback Resolution Results of HorizonAI

Metric	Value
System Resolution Rate	78.5% (219/279)
First-Pass Success	80.8% (177/219)
Retry Rescue Rate	41.2% (42/102)

The remaining values in Table 15 describe what happened to the cases that were not resolved during the first pass. Subtracting the 177 first-pass successes from the 279 total benchmark cases gives 102 cases that did not reach resolution on their initial processing pass. These cases formed the retry population under the benchmark's feedback workflow. Of these 102 cases, 42 eventually reached a resolved state after additional processing. The 42 cases can also be obtained by subtracting the 177 first-pass successes from the 219 total resolved cases. Thus, the relationship among the reported values is 219 total resolved cases = 177 first-pass successes + 42 cases resolved after additional processing. The retry rescue rate was therefore 42/102, or 41.2%.

The results show that the feedback mechanism contributed to the final resolution of cases that were not completed during the initial processing pass. However, the 41.2% retry rescue rate also indicates that additional processing did not resolve every case that entered the retry population. Of the 102 cases that were not resolved on the first pass, 42 reached a resolved state after additional processing, while the remaining 60 did not. The feedback mechanism therefore provided an

additional opportunity for resolution without guaranteeing successful recovery for every initially unsuccessful case.

The feedback process operated through bounded repeated processing rather than unrestricted regeneration. When a generated candidate failed a required validation condition, the detected issue could be carried into another processing cycle so that the system could attempt to address the identified problem. The retry limit provided a defined stopping condition for cases that remained unresolved. This allowed the system to incorporate feedback into the refactoring workflow while preventing unsuccessful cases from being processed indefinitely.

The finding can be related to the structured correction approaches discussed in Chapter II. Xu et al. [58] reported an iterative exchange between code rewriting and reviewing roles in MANTRA, while Huang et al. [59] organized AgentCoder around separate generation, testing, and feedback stages. Li et al. [7] likewise combined language-model generation with program analysis so that detected program conditions could guide subsequent repair rather than relying solely on another unrestricted model response. HorizonAI follows a similar principle by using detected conditions to guide subsequent processing, while applying the feedback cycle within a bounded multi-agent refactoring workflow.

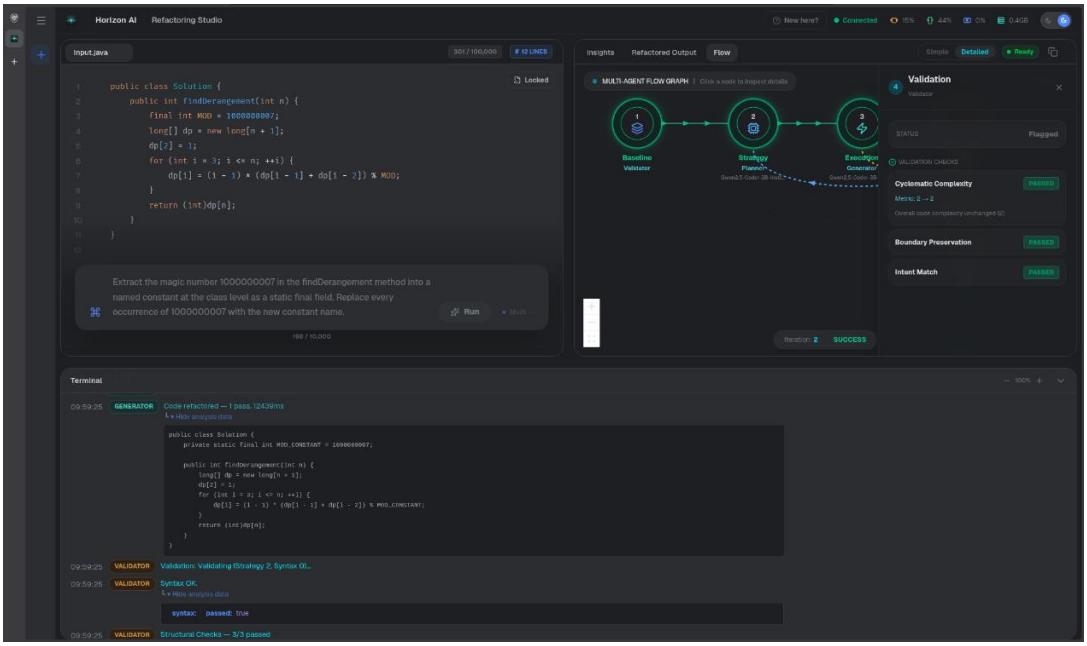


Figure 21. Actual HorizonAI Interface Showing an Initial Failure, Feedback Cycle, and Successful Resolution

Figure 21 demonstrates an example of how the feedback process operated during actual execution. The first candidate was prevented from reaching final acceptance after a validation condition was triggered. The detected issue was then carried into another processing cycle. When the succeeding output satisfied the required checks, the task reached a successful exit. The sequence provides implementation-level evidence of the feedback behavior represented in the benchmark results and illustrates how an intercepted candidate could be processed again rather than being immediately treated as the final output.

While the 41.2% retry rescue rate demonstrates that the feedback mechanism recovered a meaningful portion of initially unresolved cases, the 60 cases that exhausted the retry limit define the practical boundary of the system's self-correction capability. An analysis of these unresolved cases reveals that structural metric checks—which verify whether the generated code reflects the quantitative change required by the refactoring instruction—accounted for 71% of all recorded validator issues. Notably, syntax errors were never the source of failure in any of these exhausted cases. The system's repeated attempts were consistently blocked by structural conditions rather than malformed code.

Two specific patterns accounted for 60% of these unresolved cases: failures to decrease conditional nodes when instructed to merge sequential passes, and failures to increase loop counts when instructed to split nested loops. A closer examination indicates that repeated feedback did not resolve these conditions because the requested structural change was inherently entangled with data or sequencing dependencies in the original algorithm. For instance, splitting a nested loop where the inner computation depends on results accumulated from previous iterations makes an independent split mathematically inconsistent with the original logic.

In these situations, the Generator produced distinct candidate code on each retry attempt, and the validator's feedback remained accurate, yet no candidate could satisfy the structural condition without altering the program's intended behavior. This indicates that the retry mechanism's limitation in these instances was not due to degraded reasoning or the model disregarding feedback. Rather, the constraint originated at the level of the refactoring intent itself. When a requested transformation and the original algorithm's correctness are mutually exclusive due to underlying data dependencies, additional iterative feedback cannot force a resolution, establishing a clear boundary condition for multi-agent self-correction.

Overall, HorizonAI resolved 219 of the 279 benchmark cases, corresponding to a system resolution rate of 78.5%. Among these resolved cases, 177 were completed during the first processing pass, giving a first-pass success rate of 80.8%. The remaining 42 resolved cases were obtained after additional processing, representing 41.2% of the 102 cases that did not reach resolution on their first pass. These results show that the feedback mechanism provided a means of recovering some initially unresolved cases while maintaining a bounded processing workflow. The findings establish the observed resolution behavior of HorizonAI, while the detailed implementation and routing of the feedback mechanism are discussed separately in the implementation of the algorithms section.

Refactoring Performance on the CodeEditorBench Dataset

The second specific problem examined the refactoring performance of HorizonAI against the single-model baseline using the CodeEditorBench dataset. The comparison covered compilation success rate, behavioral equivalence rate, cyclomatic complexity, and maintainability index. The benchmark followed a funnel procedure so that each measure was evaluated at the stage where it remained meaningful. Compilation Success Rate was calculated across all 279 benchmark tasks, while BER, CC, and MI were evaluated only among successfully compiled outputs so that non-compiling outputs would not be included in the subsequent behavioral and code-quality measurements.

Compilation Success Rate (CSR)

Compilation Success Rate measures the proportion of generated Java outputs that can be compiled successfully. It provides an initial technical check of whether a generated output can compile and proceed to later evaluation. A generated output may appear structurally reasonable as text, but if it cannot be compiled, it cannot proceed to the subsequent behavioral and code-quality evaluations.

As shown in Table 16, the single-model baseline successfully compiled 189 of 279 outputs, resulting in a CSR of 67.7%. HorizonAI successfully compiled 163 of 279 outputs, equivalent to 58.4%. The baseline therefore recorded a 9.3 percentage point difference in compilation success relative to HorizonAI.

Architecture	Successfully Compiled	Total Tasks	CSR
Single-Model Baseline	189	279	67.7%
HorizonAI Multi-Agent System	163	279	58.4%
Difference	26	0	9.3 percentage points

Table 16. Compilation Success Rate of the Single-Model Baseline and HorizonAI

The results in Table 16 show that the single-model baseline produced 26 more compilable outputs than HorizonAI across the same 279 benchmark tasks. The baseline had 90 outputs that did not compile, while HorizonAI had 116. Since both architectures were evaluated using the same set of 279 benchmark tasks, the

difference in CSR reflects the proportion of outputs that reached the compilation stage under each architecture.

The CSR results also determine which outputs could proceed to the succeeding evaluation measures. In the study's evaluation procedure, BER, cyclomatic complexity, and maintainability index were computed only for outputs that successfully compiled. Thus, 189 baseline outputs and 163 HorizonAI outputs proceeded to these subsequent measures. This ensured that non-compiling outputs were not included when evaluating behavioral correctness and code-quality characteristics.

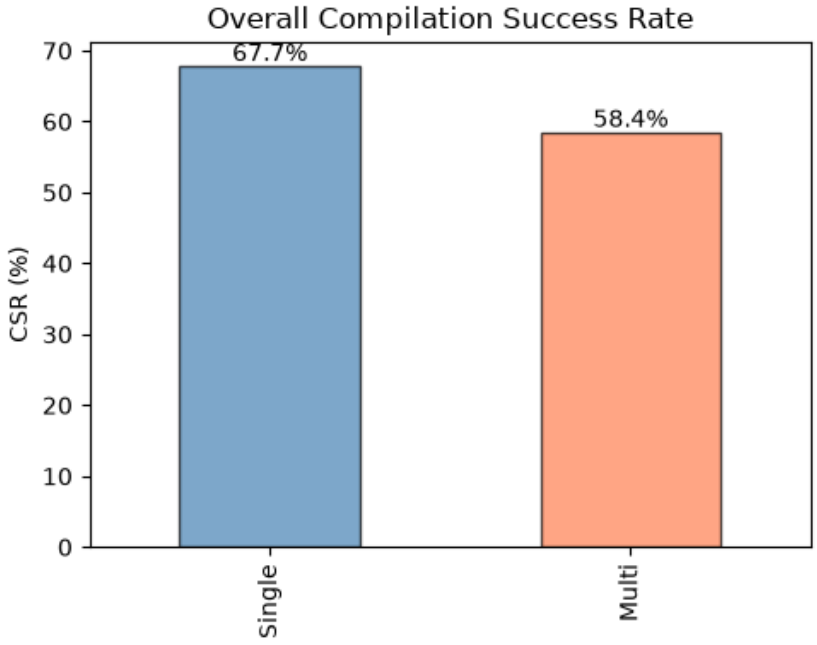


Figure 22. Comparative Compilation Success Rate on the CodeEditorBench Dataset

Figure 22 presents the compilation results for the two architectures across the 279 benchmark tasks. The single-model baseline reached a compilable state in 189 cases, while HorizonAI reached a compilable state in 163 cases. The figure therefore illustrates the 9.3 percentage-point difference observed in CSR.

The result also shows why compilation alone cannot be used to determine whether a refactoring was successfully performed. A compilable output may still fail to preserve the original behavior or may not produce the intended structural change. Seres et al. [31] emphasized the importance of behavioral equivalence in determining

whether a transformation preserves the expected program behavior. For this reason, compilation was treated as an initial condition for the succeeding evaluation rather than as evidence of complete refactoring correctness.

This evaluation sequence is also consistent with the concerns discussed by Cordeiro et al. [62] and Shirafuji et al. [63] regarding the need to establish compilation and behavioral correctness when evaluating generated code. In the present study, outputs that did not compile were not carried into the succeeding behavioral and code-quality measurements. This allowed BER, cyclomatic complexity, and maintainability index to be evaluated specifically on outputs that had reached a compilable state.

Therefore, under the CodeEditorBench evaluation, HorizonAI achieved a lower CSR than the single-model baseline, with 163 of 279 outputs compiling successfully compared with 189 of 279 for the baseline. The result establishes a difference at the compilation stage, while the succeeding Behavioral Equivalence Rate examines whether the successfully compiled outputs also preserved the expected program behavior.

Behavioral Equivalence Rate (BER)

Behavioral Equivalence Rate measures whether compiled refactored code preserves the expected behavior of the original program. It is evaluated after compilation because a Java output may compile successfully while still producing a different result from the original program. For refactoring, behavioral preservation is therefore an additional requirement beyond successful compilation.

Table 17 presents the BER evaluation based on the outputs that successfully passed the preceding compilation stage. The single-model baseline produced 189 compiled outputs, of which 21 passed the behavioral evaluation, resulting in a BER of 11.1%. In contrast, HorizonAI had 163 compiled outputs, with 16 passing the evaluation, yielding a BER of 9.8%. The difference of 26 compiled outputs reflects the varying number of outputs that reached the BER stage, ultimately resulting in an absolute difference of 5 behaviorally equivalent outputs and a marginal gap of 1.3 percentage points in BER.

Table 17. Behavioral Equivalence Rate of Compiled Outputs

Architecture	Behaviorally Equivalent	Compiled Outputs	BER
Single-Model Baseline	21	189	11.1%
HorizonAI Multi-Agent System	16	163	9.8%
Difference	5	26	1.3 percentage points

This data implies that transitioning to a multi-agent architecture did not improve the system's ability to preserve the original program's exact behavior. The marginal 1.3 percentage point decrease suggests that the segmented processing and task handoffs within the HorizonAI pipeline may introduce slight functional deviations during code refactoring. In contrast, the single-model baseline appears to maintain a slightly tighter contextual grasp on maintaining exact functional parity.

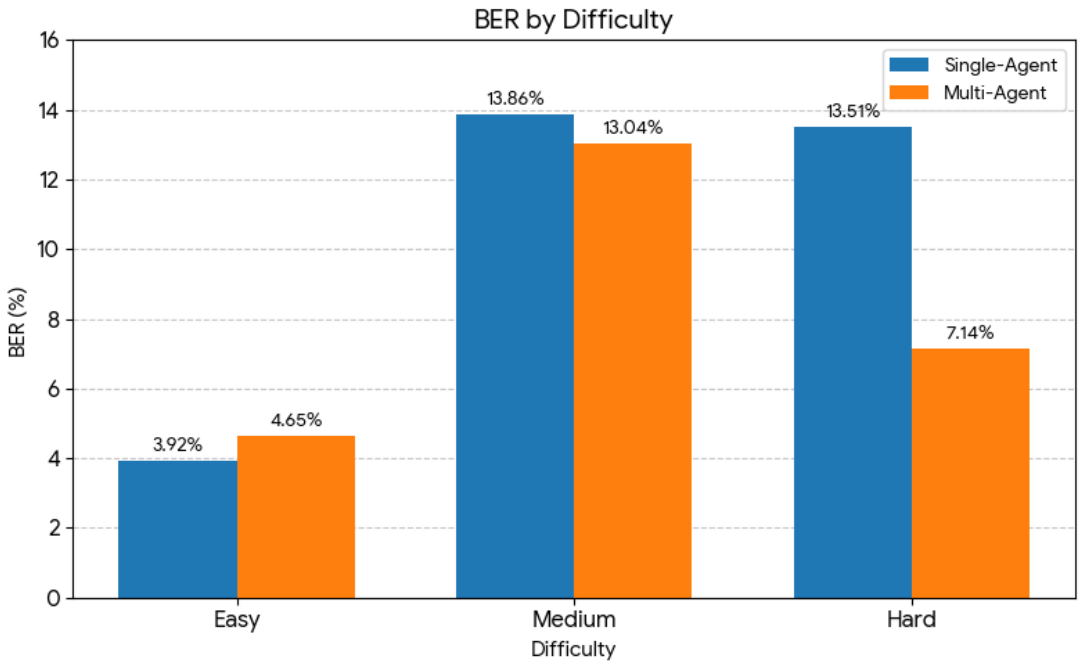


Figure 23. Behavioral Equivalence Rate by Task Difficulty

Figure 23 illustrates that the observed Behavioral Equivalence Rate (BER) varied significantly depending on the complexity of the task. As detailed in the chart, the multi-agent system (HorizonAI) recorded a higher BER for Easy tasks at 4.65%, compared to the single-agent baseline at 3.92%. Conversely, the single-agent

baseline demonstrated higher BER values for both Medium tasks (13.86% versus 13.04%) and Hard tasks (13.51% versus 7.14%).

It is important to note that these values strictly describe the observed equivalence rates within each difficulty tier and do not, by themselves, establish the underlying causes for these performance differences.

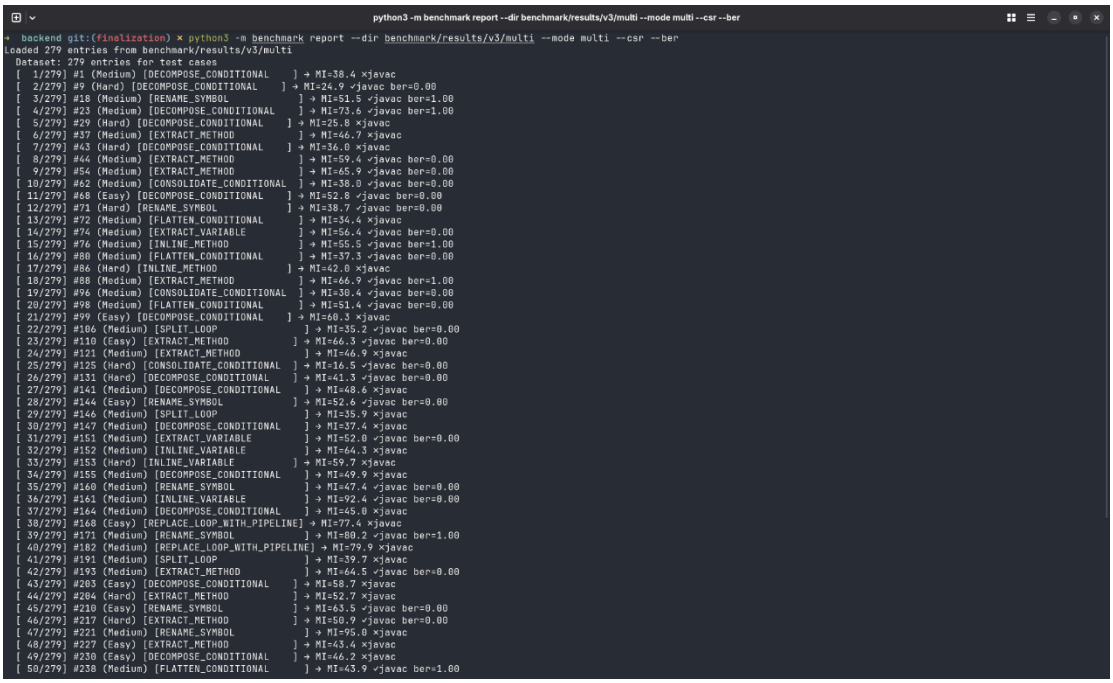


Figure 24. Behavioral Equivalence Testing of a HorizonAI Refactoring Output

Figure 24 illustrates the behavioral-equivalence stage of the evaluation. The candidate first passes Java compilation before behavioral testing is conducted. The subsequent test determines whether the refactored output preserves the expected behavior. Only outputs that satisfy this requirement contribute to the behaviorally equivalent count presented in Table 17.

The finding is consistent with Seres et al. [31], who emphasized behavioral equivalence as an important requirement when determining whether a code transformation constitutes a genuine refactoring. This supports treating behavioral preservation as a separate evaluation condition after compilation rather than assuming that compilability establishes correctness.

The result is also related to the concerns discussed in Chapter II based on Cordeiro et al. [62] and Shirafuji et al. [63]. Their studies were discussed in relation to evaluating generated refactoring without consistently establishing compilation and behavioral correctness before assessing subsequent code-quality measures. In the present evaluation, compilation was established first, followed by behavioral testing, before the succeeding code-quality measures were examined.

Cyclomatic Complexity

Cyclomatic Complexity measures the number of linearly independent paths through a program's source code. In practical terms, a higher value corresponds to a more complex control-flow structure involving additional branches and decision paths. Because automated refactoring often aims to improve code maintainability and readability, a lower Cyclomatic Complexity is generally desirable. Consistent with the established evaluation funnel, this metric was applied exclusively to outputs that successfully passed the compilation stage.

Table 18 details the mean Cyclomatic Complexity derived from these compiled populations: 189 outputs from the single-model baseline and 163 from HorizonAI. The single-model baseline recorded a mean Cyclomatic Complexity of 4.90, whereas the HorizonAI Multi-Agent System achieved a lower mean of 4.53, resulting in a measurable decrease of 0.37.

Table 18. Mean Cyclomatic Complexity of Compiled Outputs

Architecture	Mean Cyclomatic Complexity
Single-Model Baseline	4.90
HorizonAI Multi-Agent System	4.53
Difference	-0.37

This reduction explicitly demonstrates that the compiled outputs generated by HorizonAI possess a lower average Cyclomatic Complexity within the evaluated dataset. It suggests that while the multi-agent pipeline operates under strict constraints, the refactored code it successfully compiles tends to feature cleaner, less convoluted logic and simpler control flows compared to the monolithic single-model baseline.

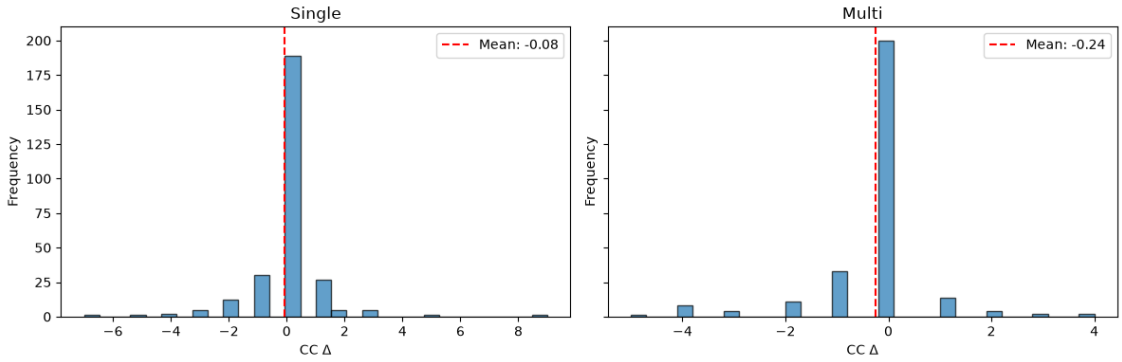


Figure 25. Overall Mean Cyclomatic Complexity Comparison

Figure 25 further presents the change in Cyclomatic Complexity between the original and refactored code. This is a separate measure from the mean Cyclomatic Complexity reported in Table 18. The figure expresses the change as ΔCC, where a negative value represents a decrease in Cyclomatic Complexity after refactoring. The mean ΔCC was −0.08 for the single-model baseline and −0.24 for HorizonAI.

The two values indicate that the average change in Cyclomatic Complexity was negative for both architectures. HorizonAI's −0.24 mean ΔCC represents a greater average reduction than the baseline's −0.08. These values describe the direction and average magnitude of change from the original code to the refactored code, whereas the 4.90 and 4.53 values in Table 18 describe the resulting mean Cyclomatic Complexity of the evaluated outputs.

The distribution shown in Figure 23 also contains both positive and negative changes in Cyclomatic Complexity. This means that the refactoring process did not uniformly reduce complexity for every output. Some outputs experienced a decrease in Cyclomatic Complexity, while others experienced an increase. The reported mean therefore summarizes the direction of the changes across the evaluated outputs rather than indicating that every refactoring reduced complexity.

The result is consistent with the finding discussed by Shirafuji et al. [61], who reported reductions in Cyclomatic Complexity in their study of LLM-assisted refactoring. The comparison is limited to the direction of the observed change because the datasets, systems, and evaluation procedures differed. Therefore, the

present result should not be interpreted as a direct comparison of the magnitude of complexity reduction between the two studies.

Cyclomatic Complexity should also be interpreted separately from behavioral correctness. A reduction in complexity does not establish that the requested refactoring preserved the original program behavior. In the preceding BER analysis, only 21 of 189 compiled baseline outputs and 16 of 163 compiled HorizonAI outputs passed the behavioral-equivalence evaluation, corresponding to 11.1% and 9.8%, respectively. These results establish why structural complexity and behavioral preservation were treated as separate measures in the evaluation.

Overall, HorizonAI recorded a mean Cyclomatic Complexity of 4.53, compared with 4.90 for the single-model baseline, giving a difference of 0.37 lower for HorizonAI. The ΔCC results in Figure 24 also showed mean changes of −0.08 for the baseline and −0.24 for HorizonAI. Together, these results describe the observed change in control-flow complexity while keeping structural complexity separate from compilation and behavioral correctness. The succeeding Maintainability Index analysis then examines another code-quality characteristic of the compiled refactored outputs.

Maintainability Index (MI) Improvement

The Maintainability Index provides a composite measure of code maintainability, where a higher value represents greater estimated maintainability according to the metric. In this evaluation, the Maintainability Index was computed among the outputs that successfully reached the compiled-output stage. This follows the evaluation funnel established in the preceding analyses, where code-quality measures were applied only to outputs that successfully compiled.

As shown in Table , the single-model baseline recorded a mean Maintainability Index of 51.88, while HorizonAI recorded 53.99. This resulted in a 2.11-point difference between the two architectures, with HorizonAI obtaining the higher mean value. The result indicates that the compiled outputs produced by

HorizonAI had a higher average Maintainability Index than those of the single-model baseline.

Table 19. Mean Maintainability Index of Compiled Outputs

Architecture	Mean Maintainability Index
Single-Model Baseline	51.88
HorizonAI Multi-Agent System	53.99
Difference	2.11 higher

Figure 22 presents the Maintainability Index distributions of the original and refactored outputs, together with the changes in MI between the corresponding original and refactored outputs. In the top graphs, the X-axis represents the Maintainability Index (MI) scores, while in the bottom graphs, it represents the change in MI scores (MI Δ). In all graphs, the Y-axis represents the count or frequency of the corresponding scores. The distributions provide a view of how maintainability values varied across the evaluated cases rather than only comparing the two overall mean values reported in Table 19.

The change in MI should be interpreted separately from the overall mean MI comparison in Table 19. The 2.11-point difference was obtained by comparing the mean MI of HorizonAI with that of the single-model baseline, while the MI Δ compares the original and refactored versions within each architecture. These therefore represent different comparisons and should not be treated as equivalent measures of improvement.

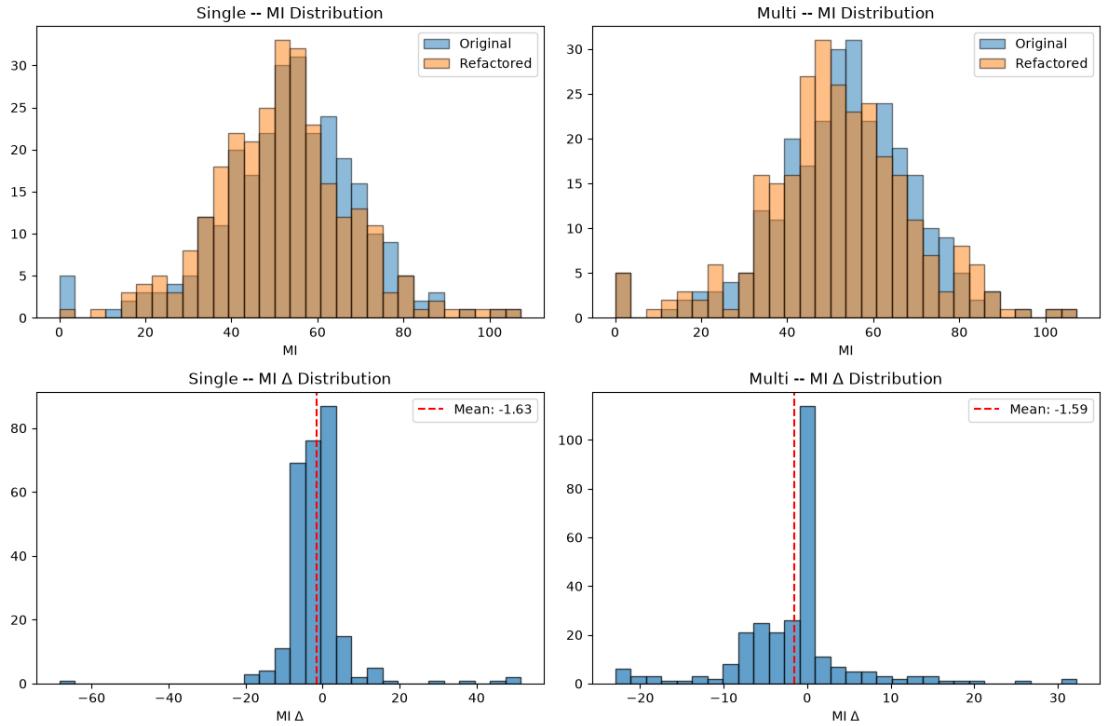


Figure 26. Maintainability Index and Change Distributions of Original and Refactored Outputs

The distributions in Figure 26 also show that changes in Maintainability Index were not uniform across the evaluated outputs. Some refactored outputs had lower MI values than their corresponding original versions, while others had higher values. Therefore, a higher overall mean MI for one architecture does not mean that every individual refactoring increased maintainability.

The Maintainability Index result can also be considered alongside the Cyclomatic Complexity results presented in the preceding subsection. HorizonAI recorded a mean Cyclomatic Complexity of 4.53, compared with 4.90 for the single-model baseline. The difference was 0.37 lower for HorizonAI, as calculated from 4.90 − 4.53. At the aggregate level, HorizonAI therefore had both a lower mean Cyclomatic Complexity and a higher mean Maintainability Index among the compiled outputs.

However, these structural measures do not establish behavioral correctness by themselves. The preceding BER analysis showed that the single-model baseline recorded 21 behaviorally equivalent outputs out of 189 compiled outputs, or 11.1%,

while HorizonAI recorded 16 out of 163, or 9.8%. These values demonstrate why Maintainability Index was treated as a separate code-quality measure rather than as a direct indicator of successful behavioral preservation.

The finding should also be interpreted in relation to the broader refactoring results. HorizonAI recorded a higher mean Maintainability Index of 53.99 compared with 51.88 for the baseline, while the baseline recorded higher CSR and BER values. Thus, the MI result describes one dimension of the evaluated refactoring output and should not be interpreted independently of compilation and behavioral correctness.

Overall, HorizonAI recorded a mean Maintainability Index of 53.99, compared with 51.88 for the single-model baseline. The resulting 2.11-point difference indicates a higher average Maintainability Index for HorizonAI among the compiled outputs evaluated. At the same time, the distribution of individual MI changes shows that maintainability did not increase uniformly across all refactored outputs. The result therefore establishes a difference in aggregate MI between the two architectures while maintaining the distinction between overall maintainability scores and changes observed within individual refactoring cases.

Computational Efficiency of HorizonAI During Local Execution

The third specific problem determined the computational efficiency of HorizonAI during local execution on consumer-grade hardware in terms of inference time and peak GPU memory usage. These measures were used to examine the processing cost introduced by the role-driven multi-agent workflow and whether the system could operate within the study's strict 4-gigabyte VRAM constraint. The evaluation was conducted under the same controlled local environment described in Chapter III, using an NVIDIA RTX 2050 GPU and 4-bit quantized models. Inference time was recorded in seconds, while peak GPU memory consumption was measured in megabytes during execution. The benchmarking procedure also treated the measurements from HorizonAI and the single-model baseline as paired observations because both architectures processed the same set of benchmark tasks.

Inference Time

In this study, inference time referred to the end-to-end execution time required to process a refactoring task during local execution. The single-model baseline performed the refactoring process using a monolithic 7-billion-parameter model, whereas HorizonAI distributed the task among three specialized logical roles: Planner, Generator, and Judge. The multi-agent framework used two 3-billion-parameter model weight sets and incorporated deterministic validation checks and iterative repair into the refactoring workflow.

As shown in Table 20, the single-model baseline recorded a mean execution time of 19.47 seconds per task, while HorizonAI recorded 97.26 seconds per task across the benchmark. HorizonAI therefore required approximately five times as much execution time per task as the single-model baseline.

Table 20. Inference Time Statistics of HorizonAI and the Single-Model Baseline

Metric	Single-Model Baseline	HorizonAI Multi-Agent System
Mean Execution Time, All Runs	19.47s	97.26s
Mean Execution Time, Successful Runs Only	19.40s	80.10s
Mean Execution Time, ABORT_STRATEGY Runs	N/A	159.90s

The outcome-specific measurements further showed that HorizonAI's execution time varied according to how the refactoring process concluded. Successful HorizonAI runs required an average of 80.1 seconds, whereas executions that ended through ABORT_STRATEGY required 159.9 seconds. The latter was 79.8 seconds longer than the successful-run mean. For the single-model baseline, the mean execution time was 19.5 seconds across all runs and 19.4 seconds among successful runs, showing only a 0.07-second difference between the two displayed means.

The 97.26-second overall HorizonAI mean was therefore higher than its 80.1-second successful-run mean, while the 159.9-second ABORT_STRATEGY mean represented the longest execution category shown in the benchmark. These

results are consistent with the iterative structure of HorizonAI, in which a refactoring task can pass through planning, generation, deterministic validation, and additional processing before reaching a final outcome.

The finding is consistent with the model-size and local-execution trade-offs discussed in Chapter II. Tung et al. [55] examined quantized language-model inference on consumer-grade devices and discussed the relationship among model size, processing speed, and available hardware. Their work demonstrates that smaller quantized models can make local inference feasible, but this does not imply that an architecture using smaller models will necessarily have shorter end-to-end execution time.

The HorizonAI results further illustrate this trade-off in a multi-stage workflow. Although each active model is smaller than the 7-billion-parameter baseline, the system performs several sequential inference and validation operations for a single refactoring task. The smaller size of an individual model therefore does not eliminate the cumulative processing time required by the complete orchestration workflow.

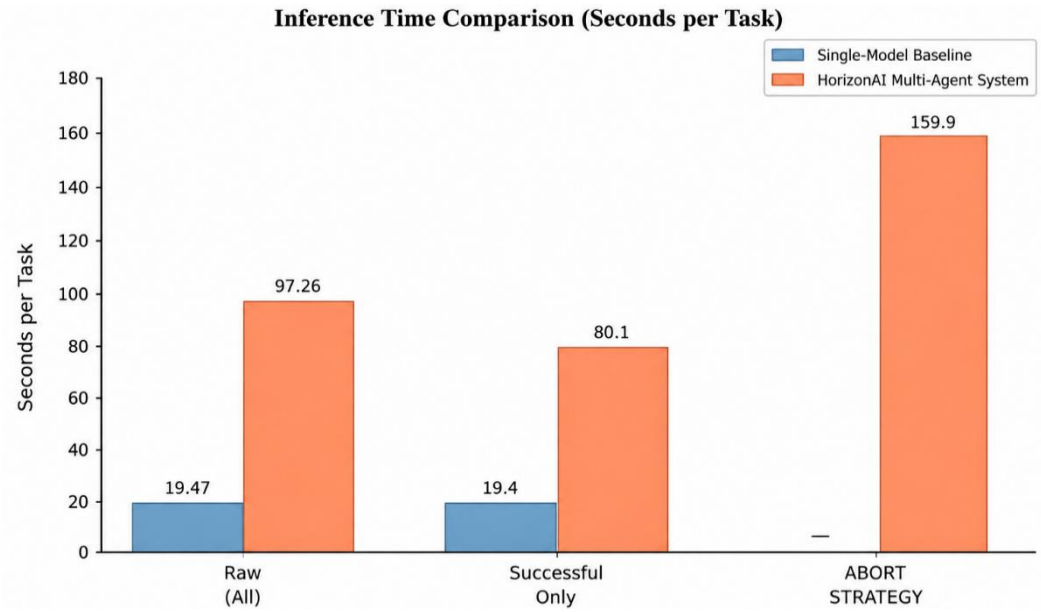


Figure 27. Inference Time Comparison Between HorizonAI and the Single-Model Baseline

Figure 27 presents the mean execution time for the two architectures and the reported HorizonAI execution outcomes. The single-model baseline remained close to 19.5 seconds per task, whether all runs or successful runs were considered. HorizonAI showed a substantially higher overall mean of approximately 97.3 seconds, with successful runs averaging 80.1 seconds and ABORT_STRATEGY runs averaging 159.9 seconds.

The higher mean for ABORT_STRATEGY executions is consistent with the additional processing associated with executions that did not reach a successful final result within the permitted workflow. HorizonAI was designed to perform iterative repair after validation identified conditions requiring correction. The process continued through controlled attempts until the required conditions were satisfied or the permitted strategy was exhausted. Consequently, executions reaching ABORT_STRATEGY consumed more processing time on average than those that completed successfully.

The result indicates that the latency of the multi-agent system was influenced not only by the models used, but also by the number of processing stages required to complete a task. Although HorizonAI employed smaller 3-billion-parameter models than the 7-billion-parameter baseline, its complete workflow involved multiple specialized operations. The smaller model size therefore did not translate directly into shorter end-to-end execution time.

The paired execution-time measurements were then examined across the 279 benchmark tasks to determine whether the difference between the two architectures was statistically significant. The Shapiro-Wilk test produced $W = 0.788$ with $p = 9.18 \times 10^{-19}$, indicating that the distribution of paired duration differences did not satisfy the normality assumption. Because the normality assumption was not satisfied, the Wilcoxon signed-rank test was used for the paired comparison.

As shown in Table 21, the Wilcoxon signed-rank test produced $W = 0.0$ with $p = 1.66 \times 10^{-47}$, indicating a statistically significant paired difference in execution time between the single-model baseline and HorizonAI. The statistical result supports the

observed difference in the descriptive means, where HorizonAI recorded 97.26 seconds per task compared with 19.47 seconds per task for the baseline

Table 21. Normality and Wilcoxon Signed-Rank Test Results for Paired Inference Time

Test	Statistic	p-value	Interpretation
Shapiro-Wilk (Normality)	W = 0.788	9.18×10^{-19}	Non-normal distribution
Wilcoxon Signed-Rank	W = 0.0	1.66×10^{-47}	Statistically significant

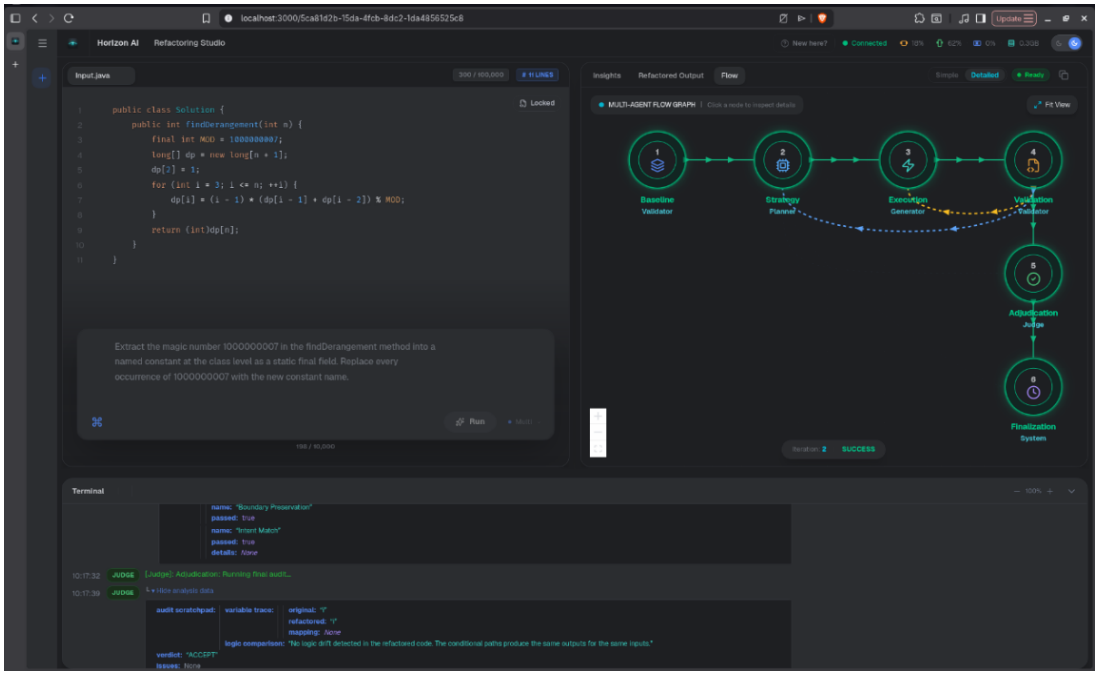


Figure 28. Actual HorizonAI Execution Showing the Sequential Refactoring Stages

Figure 28 presents an actual HorizonAI execution showing the sequence of processing stages involved in a refactoring task. The execution proceeds through the system's specialized workflow rather than producing the final result through a single model interaction. The Planner, Generator, and Judge roles operate within the broader process, while deterministic validation and iterative repair control the progression of generated outputs.

The sequential structure helps explain the additional execution time observed in the benchmark, as the measured duration reflects not only model inference but also the processing required by the complete orchestration workflow.

This is particularly relevant to the 159.90-second mean recorded for ABORT_STRATEGY executions, which exceeded the 80.10-second mean of successful HorizonAI runs.

Peak GPU Memory Usage

Peak GPU memory usage measured the highest amount of GPU memory occupied during execution. This metric was particularly relevant because HorizonAI was designed to operate within a 4-GiB VRAM constraint. Rather than keeping multiple agent models resident simultaneously, HorizonAI applied Strict Sequential Inference Scheduling, wherein the model required for the current stage was loaded, used for inference, and released before the model required for the subsequent stage was loaded.

Table 22 shows that the single-model baseline recorded a mean peak GPU memory usage of 4,044 MB, leaving approximately 52 MB below the study's 4,096-MB ceiling. In contrast, HorizonAI recorded a mean peak GPU memory usage of 3,232 MB, representing an 812-MB or 20.1% reduction relative to the baseline. This corresponded to approximately 864 MB of headroom below the specified ceiling. The result indicates that HorizonAI's sequential model scheduling was associated with a lower mean peak GPU memory requirement than the single-model baseline.

Table 22. Peak GPU Memory Usage of HorizonAI and the Single-Model Baseline

Metric	Single-Model Baseline	HorizonAI Multi-Agent System	Difference
Mean Peak GPU Memory	4,044 MB	3,232 MB	−812 MB
Relative Reduction	N/A	N/A	20.1%

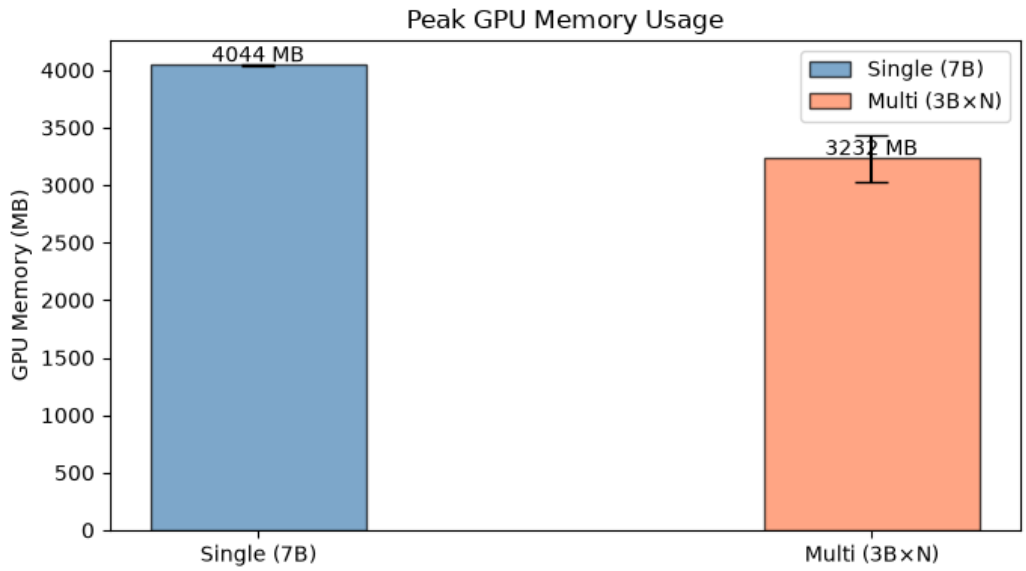


Figure 29. Peak GPU Memory Profile of HorizonAI and the Single-Model Baseline

As shown in Figure 29, the single-model baseline measurements clustered near the 4,096-MB ceiling, whereas HorizonAI's measurements clustered around 3.2 GB. This distribution indicates that the sequential scheduling strategy reused GPU memory across agent stages rather than requiring multiple specialized models to remain resident simultaneously.

To determine whether the observed memory difference was statistically significant across the paired benchmark tasks, the same normality and significance-testing procedure used for inference time was applied.

As shown in Table 23, the Shapiro-Wilk test indicated that the paired memory differences were not normally distributed ($W = 0.530$, $p = 1.15 \times 10^{-26}$). Accordingly, the Wilcoxon signed-rank test was used. The median paired memory difference was −704 MB (HorizonAI − single-model baseline), indicating a lower median peak GPU memory requirement for HorizonAI. The Wilcoxon signed-rank test produced $W = 0.0$ with $p = 3.95 \times 10^{-48}$, indicating that the paired difference in peak GPU memory usage was statistically significant.

Table 23. Normality and Wilcoxon Signed-Rank Test Results for Paired Peak GPU Memory (HorizonAI – Single-Model Baseline, n = 279)

Test	Statistic	p-value	Interpretation
Shapiro-Wilk (Normality)	W = 0.530	1.15×10^{-26}	Non-normal distribution
Wilcoxon Signed-Rank	W = 0.0	3.95×10^{-48}	Statistically significant

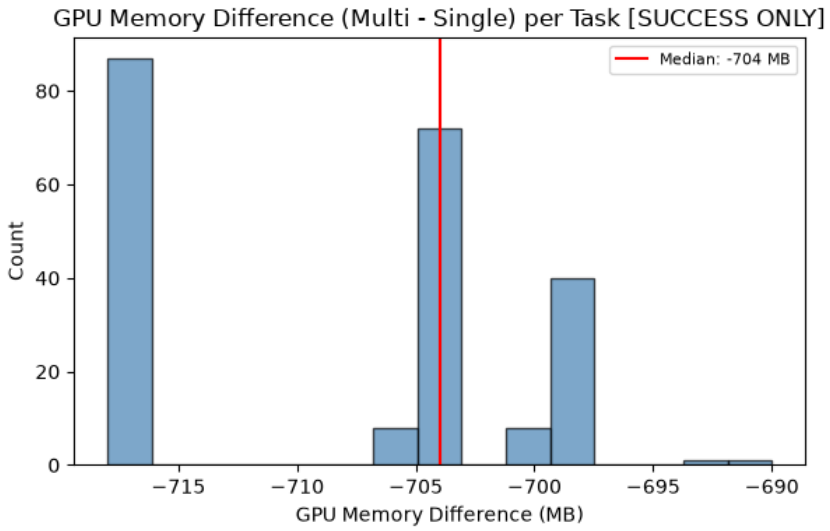


Figure 30. Paired GPU Memory Differences Between HorizonAI and the Single-Model Baseline (Successful Runs)

Figure 30 shows the distribution of paired GPU memory differences between HorizonAI and the single-model baseline. The predominance of negative differences indicates that HorizonAI generally used less peak GPU memory across the paired benchmark tasks. This finding is consistent with the resource-scheduling approach discussed under Algorithmic Resource Scheduling and Mutual Exclusion in Chapter II, in which GPU memory is treated as a shared resource and model residency is managed sequentially.

The result is also consistent with the related studies reviewed under Computational Latency and VRAM Profiling in Edge Inference. Alistarh [65] reported that sequential model loading can reduce peak VRAM consumption relative to parallel loading by reusing memory resources between processing stages. Similarly, Nyamsuren [66] reported reduced peak VRAM utilization when sequential inference scheduling was applied to quantized language models operating on GPUs with

limited memory. These findings provide supporting context for the lower peak GPU memory usage observed in HorizonAI.

Taken together, the inference-time and peak GPU memory results show a computational trade-off associated with HorizonAI's role-driven architecture. The statistically significant increase in inference time ($p = 1.66 \times 10^{-47}$) indicates an additional latency cost associated with sequential Planner, Generator, and Judge stages relative to the single-model baseline. Conversely, the statistically significant reduction in peak GPU memory usage ($p = 3.95 \times 10^{-48}$) indicates that sequential model loading and unloading substantially reduced the system's peak VRAM requirement. Thus, within the conditions of the benchmark, HorizonAI achieved lower peak GPU memory usage at the cost of increased inference time.

Assessment of the End Users

The developed system was evaluated by twenty-five (25) end users, composed of computing students and faculty members from the University of Cabuyao, St. Vincent College of Cabuyao, and Westbridge Institute of Technology, Inc. The respondents were primarily engaged in Information Technology and Computer Science and had basic to intermediate exposure to Java programming. The evaluation was based on four main criteria: interaction capability, functional suitability, security, and reliability, in accordance with the ISO/IEC 25010:2023 quality standards. These criteria describe how clearly, appropriately, safely, and consistently users experienced the Glassbox interface of HorizonAI during actual use, and each respondent evaluated it after completing the prescribed refactoring workflow.

Responses were recorded on a four-point scale: 4 = Highly Acceptable (HA), 3 = Acceptable (A), 2 = Unacceptable (U), and 1 = Highly Unacceptable (HU). The mean was used as the measure of central tendency, and the overall mean of each criterion served as the basis for its interpretation.

Interaction Capability

Interaction Capability refers to how acceptable the system interface is in allowing users to interact with HorizonAI and complete the intended refactoring task. For interaction capability, users evaluated the proposed system based on the following questions:

Q1 - Understandability of the interface in showing that HorizonAI is intended for Java code refactoring.

Q2 - Ease of understanding HorizonAI's purpose before starting a refactoring task.

Q3 - Suitability of the interface information in helping users recognize HorizonAI as a tool for generating and reviewing refactored Java code.

Q4 - Ease of learning how to submit Java code and refactoring instructions.

Q5 - Understandability of the basic steps involved in using the Glassbox interface.

Q6 - Helpfulness of the interface guidance in showing first-time users what to do next.

Q7 - Ease of using the main functions of the Glassbox interface, including the code submission box and comparison viewer, during the refactoring task.

Q8 - Comfort of using the interface layout to complete the Java refactoring workflow.

Q9 - Understandability of navigation among the different sections of the refactoring interface.

Q10 - Understandability of the interface elements in showing what each part of the system does.

Q11 - Helpfulness of system messages in explaining the current stage of the refactoring process.

Q12 - Suitability of the information shown for explaining the purpose of each displayed result.

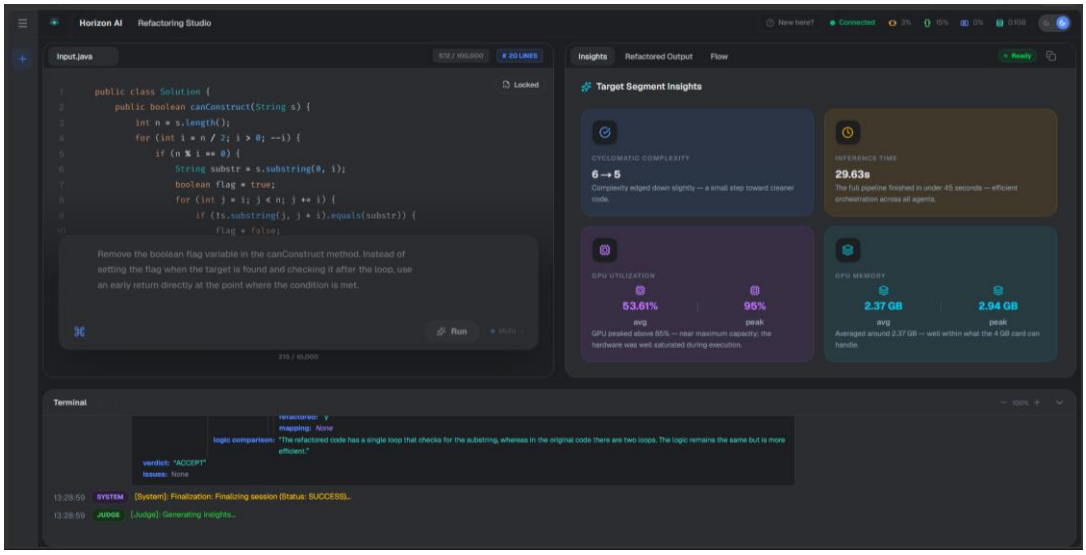


Figure 31. Glassbox Interface of HorizonAI

Interaction Capability reflects the extent to which the Glassbox interface, shown in Figure 31, communicates its purpose, can be learned and operated with reasonable ease, and presents its elements and messages in a manner that users can understand.

As shown in Table 24, the overall mean for Interaction Capability was 3.72, interpreted as Highly Acceptable. All twelve items were likewise rated Highly Acceptable, with item means ranging from 3.45 to 3.85, and every response fell under either Highly Acceptable or Acceptable, with no respondent rating any item as Unacceptable or Highly Unacceptable. This pattern indicates that the favorable interpretation was not driven by a few strongly rated items but was sustained across the entire set of items, which covered the interface's ability to convey the purpose of HorizonAI, guide first-time users, and support the completion of the refactoring task.

Table 24. End Users' Evaluation in Terms of Interaction Capability

NO.	HA (4)	A (3)	U (2)	HU (1)	MEAN	INTERPRETATION
Q1	20 (80.0%)	5 (20.0%)	0 (0.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q2	19 (76.0%)	6 (24.0%)	0 (0.0%)	0 (0.0%)	3.70	**HIGHLY ACCEPTABLE**
Q3	20 (80.0%)	5 (20.0%)	0 (0.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Q4	17 (68.0%)	8 (32.0%)	0 (0.0%)	0 (0.0%)	3.65	**HIGHLY ACCEPTABLE**
Q5	17 (68.0%)	8 (32.0%)	0 (0.0%)	0 (0.0%)	3.70	**HIGHLY ACCEPTABLE**
Q6	20 (80.0%)	5 (20.0%)	0 (0.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q7	13 (52.0%)	12 (48.0%)	0 (0.0%)	0 (0.0%)	3.50	**HIGHLY ACCEPTABLE**
Q8	17 (68.0%)	8 (32.0%)	0 (0.0%)	0 (0.0%)	3.70	**HIGHLY ACCEPTABLE**
Q9	12 (48.0%)	13 (52.0%)	0 (0.0%)	0 (0.0%)	3.45	**HIGHLY ACCEPTABLE**
Q10	19 (76.0%)	6 (24.0%)	0 (0.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q11	20 (80.0%)	5 (20.0%)	0 (0.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Q12	21 (84.0%)	4 (16.0%)	0 (0.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Overall Mean					**3.72**	**HIGHLY ACCEPTABLE**

Mapped against the questionnaire's sub-characteristics, Items 1–3 correspond to Appropriateness Recognizability, Items 4–6 to Learnability, Items 7–9 to Operability, and Items 10–12 to Self-Descriptiveness. The two lowest item means in the criterion, Q7 (3.50) and Q9 (3.45), both fall under Operability, suggesting that while users readily recognized HorizonAI's purpose and learned its basic steps, translating that understanding into fluent day-to-day use of the comparison viewer and inter-section navigation required comparatively more effort — a common gap

between conceptual clarity and procedural fluency in first-time use of AI-assisted developer tools.

An overall mean of 3.72 lies in the upper portion of the Highly Acceptable range, meaning that the respondents did not merely find the interface usable but regarded it as clear and supportive of the Java refactoring workflow. The result suggests that the Glassbox interface was able to communicate what HorizonAI is for and what was taking place at each stage of the process, which is consistent with its intent of keeping the workflow visible to the user.

This finding aligns with Felder et al. [28], who highlight that visible process information makes an AI-assisted workflow easier to inspect, and is further supported by Bernardo and Seva [29], who emphasize that the presentation of these transparent processes acts as an affective cue that calibrates user trust. At the same time, because the overall mean did not reach the upper limit of the scale, the result qualifies the broader transparency argument of Bersani et al. [50] and Von Eschenbach [52]. A system can make its internal process visible and still leave some room to improve how users move through and operate the interface. The Glassbox interface may therefore be regarded as highly acceptable in terms of interaction capability, with continued refinement of its interaction design as a reasonable direction for future iterations.

Functional Suitability

For functional suitability, users evaluated the proposed system based on the following questions:

Q1 - Availability of the necessary functions for submitting Java code and refactoring instructions.

Q2 - Completeness of the information presented for reviewing the generated refactoring output.

Q3 - Suitability of the interface functions (e.g., the code submission box, the comparison viewer) for completing the refactoring workflow.

Q4 - Correctness of the code information displayed during the refactoring process.

Q5 - Accuracy of the displayed validation results for the refactoring process.

Q6 - Consistency of the information presented throughout the refactoring workflow.

Q7 - Suitability of the workflow in guiding users from code submission to result review.

Q8 - Usefulness of the comparison among the original code, HorizonAI output, and baseline output.

Q9 - Ease of using the displayed information to determine whether the refactored output satisfies the given instruction.

Table 25 presents the end users' evaluation of the system in terms of Functional Suitability. As shown in Table 25, the overall mean was 3.72, interpreted as Highly Acceptable. All nine items were rated Highly Acceptable, with item means ranging from 3.40 to 3.90, and all responses were distributed between Highly Acceptable and Acceptable, with none falling under Unacceptable or Highly Unacceptable. The result shows that the respondents consistently perceived the functions of the Glassbox interface as adequate for carrying out the refactoring workflow from code submission to result review. Items 1–3 in this criterion correspond to Functional Completeness, Items 4–6 to Functional Correctness, and Items 7–9 to Functional Appropriateness. The lowest item mean, Q1 at 3.40 (Functional Completeness), indicates that while the available submission functions were judged adequate, some users perceived room for additional controls — consistent with the broader observation below that the function set could still be extended in future versions.

Table 25. End Users' Evaluation in Terms of Functional Suitability

NO.	HA (4)	A (3)	U (2)	HU (1)	MEAN	INTERPRETATION
Q1	11 (44.0%)	14 (56.0%)	0 (0.0%)	0 (0.0%)	3.40	**HIGHLY ACCEPTABLE**
Q2	21 (84.0%)	4 (16.0%)	0 (0.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Q3	22 (88.0%)	3 (12.0%)	0 (0.0%)	0 (0.0%)	3.90	**HIGHLY ACCEPTABLE**
Q4	19 (76.0%)	6 (24.0%)	0 (0.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q5	15 (60.0%)	10 (40.0%)	0 (0.0%)	0 (0.0%)	3.55	**HIGHLY ACCEPTABLE**
Q6	20 (80.0%)	5 (20.0%)	0 (0.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q7	19 (76.0%)	6 (24.0%)	0 (0.0%)	0 (0.0%)	3.75	**HIGHLY ACCEPTABLE**
Q8	19 (76.0%)	6 (24.0%)	0 (0.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Q9	15 (60.0%)	10 (40.0%)	0 (0.0%)	0 (0.0%)	3.60	**HIGHLY ACCEPTABLE**
Overall Mean					**3.72**	**HIGHLY ACCEPTABLE**

Because the ratings were uniformly favorable across items that addressed the submission of input, the review of generated output, and the flow of the workflow that connects them, the results suggest that users viewed the displayed information as a coherent workflow rather than as a set of separate features. This supports the Human-AI Collaboration Framework in Chapter II, where HorizonAI generates and evaluates while the user remains the final reviewer. The spread of item means below the top of the scale also suggests that, while the function set was regarded as adequate, it could still be extended in future versions.

It should be emphasized, however, that these ratings represent the users' perception of the information shown by the interface and not proof that every refactoring was objectively correct. Objective correctness was measured through the Compilation Success Rate and the Behavioral Equivalence Rate under SOP 2, and

the user-perceived functional suitability reported here should be read alongside those results.

Security

Security refers to how acceptable the system is in protecting submitted code, generated outputs, and session data. For security, users evaluated the proposed system based on the following questions:

Q1 - Confidence that the submitted Java code is handled appropriately during the evaluation session.

Q2 - Understandability of the information provided regarding how submitted Java code and generated outputs are handled.

Q3 - Suitability of the offline-first setup in limiting external exposure of submitted code and generated outputs.

Table 26 presents the end users' evaluation of the system in terms of Security. As shown in Table 26, the overall mean was 3.65, interpreted as Highly Acceptable. All three items were rated Highly Acceptable, with item means ranging from 3.45 to 3.75, and all responses were either Highly Acceptable or Acceptable. The result indicates that the respondents perceived the confidentiality-related aspects of the system, including its offline-first execution model, as highly acceptable. Among the four criteria assessed by the end users, Security obtained the lowest overall mean, although its interpretation remained the same as that of the other criteria.

Table 26. End Users' Evaluation in Terms of Security

NO.	HA (4)	A (3)	U (2)	HU (1)	MEAN	INTERPRETATION
Q1	17 (68.0%)	8 (32.0%)	0 (0.0%)	0 (0.0%)	3.75	**HIGHLY ACCEPTABLE**
Q2	13 (52.0%)	12 (48.0%)	0 (0.0%)	0 (0.0%)	3.45	**HIGHLY ACCEPTABLE**
Q3	18 (72.0%)	7 (28.0%)	0 (0.0%)	0 (0.0%)	3.75	**HIGHLY ACCEPTABLE**
Overall Mean					**3.65**	**HIGHLY ACCEPTABLE**

All three items in this criterion fall under the single ISO/IEC 25010:2023 sub-characteristic of Confidentiality, so the spread between Q1/Q3 (3.75) and Q2 (3.45) reflects variation in confidence about specific facets of data handling — general trust in appropriate handling versus understanding of how that handling works — rather than differing constructs. This is consistent with Mitchell [49], who argues that genuine trust relies on 'type transparency' (confidence in general reliability) precisely because the ability to fully verify specific operations ('token transparency') removes the vulnerability necessary for trust to exist, with Von Eschenbach [50], who identifies opacity as a barrier to justified confidence, and with Winfield et al. [51], who hold that transparency should be designed into a system rather than added later. The result should not be read as evidence that HorizonAI is technically secure, but as evidence that users perceived its confidentiality-related aspects as highly acceptable. Stating explicitly where code is processed and what information is retained would further strengthen that confidence.

Reliability

Reliability refers to how acceptable the system is in performing its functions without unexpected interruptions or failures. For reliability, users evaluated the proposed system based on the following questions:

Q1 - Stability of the Glassbox interface throughout the Java refactoring session.

Q2 - Consistency of the displayed information throughout the refactoring process without unexpected errors.

Q3 - Helpfulness of the displayed messages in explaining an unsuccessful refactoring attempt.

Q4 - Availability of the system interface throughout the evaluation session.

Q5 - Accessibility of the generated results after the refactoring process is completed.

Q6 - Suitability of the interface for keeping generated results available during review.

Q7 - Understandability of the recovery information presented after an unsuccessful refactoring attempt.

Q8 - Acceptability of the recovery options provided after an unsuccessful refactoring attempt.

Q9 - Usefulness of the displayed logs or messages in helping users understand what happened after an interruption or failure.

Table 27 presents the end users' evaluation of the system in terms of Reliability. As shown in Table 27, the overall mean was 3.76, interpreted as Highly Acceptable and the highest among the four criteria assessed by the end users. All nine items were rated Highly Acceptable, with item means ranging from 3.65 to 3.85. The result shows that the respondents found the interface and the generated results to remain accessible during the session, and that the logs and messages shown after an unsuccessful attempt were useful in helping them understand what had happened. Items 1–3 correspond to Faultlessness, Items 4–6 to Availability, and Items 7–9 to Recoverability.

Table 27. End Users' Evaluation in Terms of Reliability

NO.	HA (4)	A (3)	U (2)	HU (1)	MEAN	INTERPRETATION
Q1	18 (72.0%)	7 (28.0%)	0 (0.0%)	0 (0.0%)	3.70	**HIGHLY ACCEPTABLE**
Q2	18 (72.0%)	7 (28.0%)	0 (0.0%)	0 (0.0%)	3.65	**HIGHLY ACCEPTABLE**
Q3	16 (64.0%)	9 (36.0%)	0 (0.0%)	0 (0.0%)	3.65	**HIGHLY ACCEPTABLE**
Q4	19 (76.0%)	6 (24.0%)	0 (0.0%)	0 (0.0%)	3.75	**HIGHLY ACCEPTABLE**
Q5	20 (80.0%)	5 (20.0%)	0 (0.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q6	21 (84.0%)	4 (16.0%)	0 (0.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Q7	19 (76.0%)	4 (16.0%)	2 (8.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q8	19 (76.0%)	5 (20.0%)	1 (4.0%)	0 (0.0%)	3.80	**HIGHLY ACCEPTABLE**
Q9	21 (84.0%)	3 (12.0%)	1 (4.0%)	0 (0.0%)	3.85	**HIGHLY ACCEPTABLE**
Overall Mean					**3.76**	**HIGHLY ACCEPTABLE**

This finding is consistent with the Neuro-Symbolic Governance and Safety Nets discussed in Chapter II. Garcez and Lamb [42], Hitzler and Sarker [43], Manhaeve et al. [44], and Brain and Polgreen [45] support architectures where flexible neural generation is constrained by explicit symbolic or probabilistic-logical rules. HorizonAI operationalizes this division of labor through its validation and feedback mechanisms, translating these theoretical constraints into a system experience that users regarded as stable and dependable.

Notably, the only Unacceptable responses in the entire end-user assessment appear under this criterion — and specifically cluster within Recoverability (Q7–Q9), accounting for no more than 8.0% of responses on any item. This is a meaningful pattern rather than random noise: Faultlessness and Availability describe steady-state behavior that every respondent experienced uniformly, whereas judging Recoverability required having personally encountered an

unsuccessful refactoring attempt, a less uniform experience across the 25 respondents. The dip is therefore better read as evidence that recovery-state communication is harder to evaluate consistently than normal-state communication, not as a weaker overall recovery experience. These were too few to change the Highly Acceptable interpretation, but they indicate that a small number of respondents found the information provided after a failed attempt insufficient. The ratings should therefore not be read as meaning that HorizonAI never fails, since the earlier specific problems show that unsuccessful runs occur, but as showing that the interface presented both normal and unsuccessful states in a stable and understandable way.

Taken together, the overall means for the four criteria ranged from 3.65 to 3.76, and all four were interpreted as Highly Acceptable. End users therefore perceived the Glassbox interface of HorizonAI as clear in its interaction, suitable in its functions, acceptable in handling submitted code, and dependable in its behavior during the refactoring workflow.

Overall User Acceptability

As summarized in Table 28, all four evaluated characteristics were interpreted as Highly Acceptable. Reliability had the highest mean (3.76), followed by functional suitability (3.72), interaction capability (3.72), and security (3.65). The overall mean across the four quality characteristics was 3.71, indicating a Highly Acceptable level of user acceptability for the developed Glassbox interface.

Table 28. Overall End-User Acceptability of the HorizonAI Glassbox Interface

ISO/IEC 25010:2023 Quality Characteristic	Mean	Interpretation
Interaction Capability	3.72	Highly Acceptable
Functional Suitability	3.72	Highly Acceptable
Security	3.65	Highly Acceptable
Reliability	3.76	Highly Acceptable
Overall Mean	**3.71**	**Highly Acceptable**

Overall, the findings show that the Glassbox design was generally successful in translating HorizonAI's underlying multi-agent orchestration into an interface that representative users could understand, operate, review, and continue using even when the backend process encountered unsuccessful outcomes. The

results broadly align with the literature reviewed in Chapter II, particularly studies emphasizing explainability, human involvement, transparent reasoning traces, and deterministic verification as key components of trustworthy AI-assisted software systems. No reviewed literature in Chapter II directly contradicts the survey findings. Rather, the comparatively lower security and operability scores provide qualifying evidence that transparency and offline execution alone do not eliminate the need for clearer data-handling explanations and continued interface refinement.

Assessment of IT Experts

Ten (10) IT experts evaluated the proposed system using the validated evaluation instrument. Each expert possessed at least three (3) years of relevant professional or academic experience in software development, system design, artificial intelligence, or related computing fields, which qualified them to examine HorizonAI's multi-agent architecture, deterministic validation process, resource behavior under a constrained local environment, and maintainability as a software system. The evaluation was based on four main criteria: functional suitability, performance efficiency, reliability, and maintainability, in accordance with the ISO/IEC 25010:2023 quality standards.

Each item was rated on a four-point scale: 4 = Strongly Agree (SA), 3 = Agree (A), 2 = Disagree (D), and 1 = Strongly Disagree (SD). The mean was used as the measure of central tendency, with a Strongly Agree rating corresponding to a Highly Acceptable interpretation and an Agree rating corresponding to an acceptable interpretation under the study's scale. The overall mean of each criterion served as the basis for its interpretation.

Functional Suitability

Functional Suitability examined whether HorizonAI provided the necessary, correct, and appropriate functions for multi-agent Java code refactoring within the stated scope. For functional suitability, IT experts evaluated the proposed system based on the following questions:

Q1 - HorizonAI provides the necessary components to support the intended Java refactoring workflow.

Q2 - The system includes sufficient control functions to support automated Java refactoring within the defined local execution setup.

Q3 - The available system functions are complete enough to support the comparison between HorizonAI and the single-model baseline.

Q4 - The deterministic validation process correctly identifies syntax-related issues before the refactored output is accepted.

Q5 - The documented routing behavior correctly directs failed or invalid outputs to the appropriate correction stage.

Q6 - The displayed system outputs correspond precisely to the actual refactoring process shown in the technical review.

Q7 - The separation of planning, generation, and evaluation roles is appropriate for the intended Java refactoring task.

Q8 - The use of deterministic validation before final output acceptance is appropriate for improving output control.

Q9 - The overall workflow is appropriate for supporting Java refactoring under a local, resource-constrained environment.

Table 29 presents the IT experts' evaluation of the system in terms of Functional Suitability. As shown in Table 29, the overall mean was 3.46, interpreted as Strongly Agree, or Highly Acceptable. All nine items were interpreted as Strongly Agree, with item means ranging from 3.30 to 3.70, and no expert selected Strongly Disagree on any item. Disagree responses were few, accounting for no more than

20.0% of the responses on any item. The result indicates that the experts perceived HorizonAI as providing the components, control functions, and workflow needed to support multi-agent Java refactoring within a local execution setup. Items 1–3 map to Functional Completeness, Items 4–6 to Functional Correctness, and Items 7–9 to Functional Appropriateness.

Table 29. IT Experts' Evaluation in Terms of Functional Suitability

NO.	SA (4)	A (3)	D (2)	SD (1)	MEAN	INTERPRETATION
Q1	5 (50.0%)	5 (50.0%)	0 (0.0%)	0 (0.0%)	3.50	**STRONGLY AGREE**
Q2	7 (70.0%)	3 (30.0%)	0 (0.0%)	0 (0.0%)	3.70	**STRONGLY AGREE**
Q3	5 (50.0%)	5 (50.0%)	0 (0.0%)	0 (0.0%)	3.50	**STRONGLY AGREE**
Q4	3 (30.0%)	7 (70.0%)	0 (0.0%)	0 (0.0%)	3.30	**STRONGLY AGREE**
Q5	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Q6	5 (50.0%)	3 (30.0%)	2 (20.0%)	0 (0.0%)	3.30	**STRONGLY AGREE**
Q7	6 (60.0%)	4 (40.0%)	0 (0.0%)	0 (0.0%)	3.60	**STRONGLY AGREE**
Q8	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Q9	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Overall Mean					**3.46**	**STRONGLY AGREE**

The result supports the architecture described in Chapter II, where structured orchestration and explicit role separation are used to make agent handoffs more controlled and traceable [40]. It is also consistent with the neuro-symbolic approaches discussed by Garcez and Lamb [42] and others [43], [44], [47], in which statistical neural generation is evaluated and bounded by explicit logical frameworks. The item means at the lower end of the range indicate that the experts were more measured when endorsing claims about the correctness of the validation process and

the exact correspondence between the displayed output and the actual refactoring process.

This favorable perception should be interpreted together with the objective benchmark findings. HorizonAI still showed limitations in compilation success compared with the single-model baseline, even though the two architectures produced statistically comparable behavioral equivalence on successful runs. The expert evaluation therefore supports the suitability of the system's architecture and control functions without implying that every generated refactoring is correct.

Performance Efficiency

Performance Efficiency evaluates whether HorizonAI performs its functions within acceptable execution behavior and resource use, particularly under the study's 4GB VRAM constraint. For performance efficiency, IT experts evaluated the proposed system based on the following questions:

Q1 - The system's processing time is acceptable for a locally executed multi-agent Java refactoring workflow.

Q2 - The reported inference time is acceptable in relation to the number of executed agent stages and the available hardware.

Q3 - The system provides acceptable response behavior throughout the complete refactoring workflow.

Q4 - The Strict Sequential Inference Scheduling protocol is appropriate for managing model execution under the 4GB VRAM constraint.

Q5 - The system's model loading and unloading approach is acceptable for reducing excessive GPU memory usage.

Q6 - The reported peak GPU memory usage indicates that HorizonAI can operate within the defined local hardware limitation.

Table 30 presents the IT experts' evaluation of the system in terms of Performance Efficiency. As shown in Table 30, the overall mean was 3.35, interpreted as Strongly Agree, or Highly Acceptable. Three of the six items obtained means of

3.10, 3.20, and 3.20, which correspond to Agree and are interpreted as Acceptable, while the remaining three items obtained Strongly Agree ratings and were interpreted as Highly Acceptable. Item means ranged from 3.10 to 3.70, and Disagree responses accounted for no more than 30.0% of the responses on any item, with no Strongly Disagree responses recorded. Among the four characteristics evaluated by the IT experts, Performance Efficiency obtained the lowest overall mean, which indicates that the experts were less emphatic in their endorsement of this characteristic than of the others, although the overall interpretation remained favorable. Items 1–3 correspond to Time Behaviour and Items 4–6 to Resource Utilization. The pattern is not incidental: Time Behaviour items averaged 3.20 while Resource Utilization items averaged 3.53, a split that anticipates the benchmark comparison discussed next.

Table 30. IT Experts' Evaluation in Terms of Performance Efficiency

NO.	SA (4)	A (3)	D (2)	SD (1)	MEAN	INTERPRETATION
Q1	3 (30.0%)	5 (50.0%)	2 (20.0%)	0 (0.0%)	3.10	**AGREE**
Q2	3 (30.0%)	7 (70.0%)	0 (0.0%)	0 (0.0%)	3.30	**STRONGLY AGREE**
Q3	4 (40.0%)	4 (40.0%)	2 (20.0%)	0 (0.0%)	3.20	**AGREE**
Q4	7 (70.0%)	2 (20.0%)	1 (10.0%)	0 (0.0%)	3.60	**STRONGLY AGREE**
Q5	7 (70.0%)	3 (30.0%)	0 (0.0%)	0 (0.0%)	3.70	**STRONGLY AGREE**
Q6	5 (50.0%)	2 (20.0%)	3 (30.0%)	0 (0.0%)	3.20	**AGREE**
Overall Mean					**3.35**	**STRONGLY AGREE**

The expert evaluation reflects the computational trade-off measured during benchmarking. Across all benchmark runs, the single-model baseline recorded a mean inference time of 19.47 seconds, while HorizonAI recorded 97.26 seconds, making the multi-agent workflow approximately five times slower. In contrast, HorizonAI recorded a mean peak GPU memory usage of 3,232 MB, compared with 4,044 MB for the baseline, representing an 812 MB or 20.1% reduction. These results explain why the experts evaluated the memory-related aspects of the system more

favorably than its time-related aspects, and they indicate that HorizonAI's primary efficiency advantage lies in memory utilization rather than execution speed.

Reliability

Reliability evaluates whether HorizonAI performs its intended functions without uncontrolled failures and whether the system remains controlled when invalid outputs, failed attempts, or aborted runs occur. For reliability, IT experts evaluated the proposed system based on the following questions:

Q1 - HorizonAI performs its normal refactoring workflow without unexpected system faults during standard testing.

Q2 - The system consistently displays the required information during normal operation.

Q3 - The system's normal execution flow is stable enough to support technical evaluation of the refactoring process.

Q4 - The system handles invalid generated outputs without causing uncontrolled failure.

Q5 - The deterministic circuit breakers and iteration limits effectively prevent infinite correction loops.

Q6 - The system's abort behavior is acceptable when a valid refactoring output cannot be produced within the defined limits.

Q7 - The system provides a clear final state after an unsuccessful refactoring attempt.

Q8 - The system preserves enough logs or status information to explain what happened after a failed or aborted attempt.

Q9 - The system allows the original code or previous valid state to remain available when the refactoring process cannot produce an acceptable output.

Table 31 presents the IT experts' evaluation of the system in terms of Reliability. As shown in Table 31, the overall mean was 3.43, interpreted as Strongly Agree, or Highly Acceptable. Eight of the nine items were interpreted as Strongly

Agree, with only one item falling to Agree at a mean of 3.20, and item means ranged from 3.20 to 3.70. No expert selected Strongly Disagree on any item, and Disagree responses accounted for no more than 20.0% of the responses on any item. These results indicate that the experts generally perceived the system as stable during normal operation, capable of handling invalid outputs and bounding correction attempts, and able to retain a usable state when a refactoring attempt could not be completed.

Table 31. IT Experts' Evaluation in Terms of Reliability

NO.	SA (4)	A (3)	D (2)	SD (1)	MEAN	INTERPRETATION
Q1	7 (70.0%)	1 (10.0%)	2 (20.0%)	0 (0.0%)	3.50	**STRONGLY AGREE**
Q2	6 (60.0%)	2 (20.0%)	2 (20.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Q3	5 (50.0%)	5 (50.0%)	0 (0.0%)	0 (0.0%)	3.50	**STRONGLY AGREE**
Q4	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Q5	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Q6	3 (30.0%)	6 (60.0%)	1 (10.0%)	0 (0.0%)	3.20	**AGREE**
Q7	5 (50.0%)	3 (30.0%)	2 (20.0%)	0 (0.0%)	3.30	**STRONGLY AGREE**
Q8	6 (60.0%)	3 (30.0%)	1 (10.0%)	0 (0.0%)	3.50	**STRONGLY AGREE**
Q9	7 (70.0%)	3 (30.0%)	0 (0.0%)	0 (0.0%)	3.70	**STRONGLY AGREE**
Overall Mean					**3.43**	**STRONGLY AGREE**

Items 1–3 correspond to Faultlessness, Items 4–6 to Fault Tolerance, and Items 7–9 to Recoverability. The lone item interpreted as Agree rather than Strongly Agree, Q6 at 3.20 (Fault Tolerance), concerns the acceptability of the system's abort behavior. Experts appear more measured in endorsing an outright halt than in endorsing detection or recovery mechanisms, likely because aborting without producing output carries a direct usability cost even when it is the technically correct

response to a bounded failure — a tension between safety and task completion inherent to deterministic safety-net designs.

The objective system results support this perception of controlled reliability. Among the validation components, Intent Match recorded the highest interception rate at 23.3%, followed by Judge evaluation at 7.9%, while Cyclomatic Complexity and Boundary validation each recorded 5.4%, and Syntax validation recorded 3.2%. Furthermore, HorizonAI recovered 42 of 102 initially unresolved tasks, resulting in an iterative resolution rate of 41.2%. These results indicate that failures were not eliminated but were detected, routed, retried within defined limits, and terminated in a controlled manner when unresolved.

Maintainability

Maintainability evaluates whether HorizonAI's architecture can be analyzed, tested, and modified effectively by intended technical maintainers. For maintainability, IT experts evaluated the proposed system based on the following questions:

Q1 - The system architecture clearly separates its major agent, validation, orchestration, and interface components.

Q2 - Changes to one major component appear manageable without requiring unnecessary changes to the entire system.

Q3 - The role-based agent structure supports independent review or improvement of specific system components.

Q4 - The system logs and reasoning traces provide enough information to analyze how a refactoring output was produced.

Q5 - The validation results and failure messages are useful for diagnosing unsuccessful refactoring attempts.

Q6 - The system structure makes it possible to identify which component or processing stage requires improvement.

Q7 - The system provides measurable outputs that can be used for objective technical testing.

Q8 - The benchmark setup provides a feasible basis for objectively testing HorizonAI against the single-model baseline.

Q9 - The provided technical evidence is sufficient to verify whether the core system requirements are met.

Table 32 presents the IT experts' evaluation of the system in terms of Maintainability. As shown in Table 32, the overall mean was 3.58, interpreted as Strongly Agree, or Highly Acceptable, and the highest among the four ISO/IEC 25010:2023 characteristics evaluated by the IT experts. All nine items were interpreted as Strongly Agree, with item means ranging from 3.30 to 3.90. Most responses were concentrated in Strongly Agree and Agree, and the Disagree and Strongly Disagree responses were isolated. The result indicates that the experts perceived the separation of components, the usefulness of logs and validation messages, and the availability of measurable outputs as adequate for analyzing, testing, and improving the system. Items 1–3 correspond to Modularity, Items 4–6 to Analysability, Items 7–9 to Modifiability, and Items 10–12 to Testability.

Table 32. IT Experts' Evaluation in Terms of Maintainability

NO.	SA (4)	A (3)	D (2)	SD (1)	MEAN	INTERPRETATION
Q1	8 (80.0%)	1 (10.0%)	1 (10.0%)	0 (0.0%)	3.70	**STRONGLY AGREE**
Q2	5 (50.0%)	3 (30.0%)	2 (20.0%)	0 (0.0%)	3.30	**STRONGLY AGREE**
Q3	7 (70.0%)	3 (30.0%)	0 (0.0%)	0 (0.0%)	3.70	**STRONGLY AGREE**
Q4	8 (80.0%)	1 (10.0%)	0 (0.0%)	1 (10.0%)	3.60	**STRONGLY AGREE**
Q5	9 (90.0%)	1 (10.0%)	0 (0.0%)	0 (0.0%)	3.90	**STRONGLY AGREE**
Q6	5 (50.0%)	5 (50.0%)	0 (0.0%)	0 (0.0%)	3.50	**STRONGLY AGREE**
Q7	7 (70.0%)	3 (30.0%)	0 (0.0%)	0 (0.0%)	3.70	**STRONGLY AGREE**
Q8	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Q9	5 (50.0%)	4 (40.0%)	1 (10.0%)	0 (0.0%)	3.40	**STRONGLY AGREE**
Overall Mean					**3.58**	**STRONGLY AGREE**

The finding supports the Chapter II discussion on structured orchestration and transparency. Adimulam, Gupta, and Kumar [40] describe role-based orchestration as a mechanism for separating responsibilities and governing component interactions, Bersani et al. [50] frame explainability as a software requirement, and Winfield et al. [53] emphasize that transparency must be designed into the system itself. The strong maintainability rating suggests that these principles were reflected in the way HorizonAI was structured.

It is important to distinguish this ISO/IEC 25010:2023 maintainability result from the Maintainability Index used in the refactoring benchmark. The expert evaluation concerns the maintainability of HorizonAI as a software system and architecture, while the benchmark Maintainability Index measures properties of the generated Java code. The 3.58 expert mean therefore indicates that the system itself was perceived as maintainable, even though the generated-code Maintainability

Index results showed only comparable average changes relative to the single-model baseline.

Taken together, the overall means for the four characteristics ranged from 3.35 to 3.58, and all four were interpreted as Strongly Agree, or Highly Acceptable. The IT experts therefore perceived HorizonAI as functionally suitable, reliable under failure conditions, and highly maintainable, with performance efficiency being the characteristic that drew the most moderate endorsement, owing to the longer execution time of the multi-agent workflow.

Overall IT Expert Evaluation of HorizonAI

As summarized in Table 33, all four evaluated characteristics were interpreted as Strongly Agree. Maintainability obtained the highest mean at 3.58, followed by functional suitability at 3.46, reliability at 3.43, and performance efficiency at 3.35. The overall mean across the four evaluated quality characteristics was 3.45, indicating that the IT experts strongly agreed that the developed HorizonAI system satisfied the assessed technical quality requirements.

Table 33. Overall IT Expert Evaluation of the HorizonAI System

ISO/IEC 25010:2023 Quality Characteristic	Mean	Interpretation
Functional Suitability	3.46	Strongly Agree
Performance Efficiency	3.35	Strongly Agree
Reliability	3.43	Strongly Agree
Maintainability	3.58	Strongly Agree
Overall Mean	**3.45**	**Strongly Agree**

Overall, the findings indicate that HorizonAI's implemented multi-agent orchestration architecture was generally perceived by IT experts as technically appropriate for supporting automated Java refactoring. The results reflect positive evaluations regarding the system's modular structure, validation mechanisms, execution workflow, and local resource management strategies. The findings are consistent with the objectives discussed in Chapter II, particularly the reviewed studies emphasizing role-based AI collaboration, deterministic validation, transparency mechanisms, and controlled execution as approaches for improving the reliability and maintainability of AI-assisted software engineering systems. The

comparatively lower rating in performance efficiency suggests that while the system was considered acceptable within the defined hardware constraints, execution speed remains an area for further optimization.

Implementation of the Multi-Agent Orchestration Architecture

This section details the technical implementation of the HorizonAI architecture. It outlines how generative roles are assigned to dedicated agents governed by a centralized configuration file, managed through Strict Sequential Inference Scheduling (SSIS) to survive the 4-gigabyte VRAM limit, and coordinated by a stateful orchestration loop that integrates deterministic validation and layered error recovery.

Model Configuration and Agent-to-Model Assignment

Every model parameter and every numeric limit used during orchestration was kept in one external settings file, model_config.yaml, which is read at startup by OrchestrationConfig.from_yaml. This file contains one section for each AI agent role, plus a separate "orchestration" section listing all of the caps and retry limits used throughout the pipeline.

```
planner:
  name: "Qwen2.5-Coder-3B-Instruct"
  temperature: 0.1
  max_tokens: 4096
  context_size: 6144
  layers: 36
  filename: "qwen_coder.gguf"

generator:
  name: "Qwen2.5-Coder-3B-Instruct"
  temperature: 0.1
  max_tokens: 4096
  context_size: 6144
  layers: 36
  filename: "qwen_coder.gguf"

judge:
  name: "Llama-3.2-3B-Instruct"
  temperature: 0.1
  max_tokens: 4096
  context_size: 6144
  layers: 28
  filename: "llama_engine.gguf"
```

```
single:
  name: "Qwen2.5-Coder-7B-Instruct"
  temperature: 0.1
  max_tokens: 4096
  context_size: 4096
  layers: 20
  filename: "Qwen2.5-Coder-7B-Instruct-Q4_K_M.gguf"
```

Figure 32. Complete Model and Orchestration Parameter Configuration

Figure 32 details the configuration schema governing the multi-agent pipeline, highlighting that the Planner and Generator share the identical qwen_coder.gguf model file, whereas the Judge utilizes a distinct llama_engine.gguf file. This shared configuration represents a necessary engineering accommodation to survive hardware limitations. Operating within a strict 4-gigabyte VRAM constraint means loading three separate model files sequentially would introduce prohibitive disk I/O latency during phase transitions.

To compensate for utilizing the same neural weights, role specialization between the Planner and Generator is strictly enforced at the prompt and context levels. Each agent operates under a distinct system prompt, processes unique input compositions, and is restricted to different structured output schemas. Crucially, an explicit key-value (KV) cache flush (clear_context()) is executed between these stages. This forcibly purges the model's short-term memory of the current exchange, guaranteeing that residual reasoning from the Planner's strategy phase does not pollute the Generator's execution phase.

Model Lifecycle and Sequential Scheduling

To respect the hardware's limited graphics memory, the system enforces a strict rule: only one AI model may be active in memory at a time. This rule, called the Strict Sequential Inference Scheduling protocol, is implemented inside the AgentService class, which wraps the llama-cpp-python library, a tool for running GGUF models. It exposes four operations — load, unload, swap, and clear_context — that together manage a model's full life cycle in the graphics card's memory (VRAM).

The load operation locks itself before creating a new model instance, so that no other part of the program can try to load a model at the same time. Before doing any real work, it first checks whether the model that was asked for is already the one currently loaded; if so, it returns immediately rather than reloading it, which avoids wasting time and memory. When a different model actually needs to be loaded, the swap operation first calls unload and then load. Unloading closes the current model, removes it from memory, and explicitly asks Python's garbage collector to clean up freed memory — using a background thread so the rest of the program isn't blocked — followed by a half-second pause to give the graphics driver time to fully release the memory before the next model is loaded.

```
class AgentService:
    """Handles lifecycle of Small Language Models. Optimised for 4GB VRAM."""

    async def load(self, config: dict) -> None:
        async with self._model_lock:
            path = str(MODELS_DIR / config["filename"])
            if self.current_model_path == path and self.model is not None:
                return  # Bypass I/O: model already resident in VRAM
            self.model = await asyncio.wait_for(
                asyncio.to_thread(
                    Llama,
                    model_path=path,
                    n_gpu_layers=config["layers"],
                    n_ctx=config["context_size"],
                    flash_attn=True,
                    verbose=False,
                ),
                timeout=120,
            )
            self.current_model_path = path

    async def unload(self) -> None:
        async with self._model_lock:
            if self.model is not None:
                self.model.close()
                del self.model
                self.model = None
                self.current_model_path = None
                await asyncio.to_thread(gc.collect)
        await asyncio.sleep(0.5)  # Allow CUDA driver to unmap memory

    async def swap(self, config: dict) -> None:
        path = str(MODELS_DIR / config["filename"])
        if self.current_model_path == path:
            return  # Same model already loaded
        await self.unload()
        await self.load(config)
```

```
async def clear_context(self) -> None:
    async with self._model_lock:
        if self.model is not None:
            await asyncio.to_thread(self.model.reset)
            # Purges KV-cache without unloading model weights
```

Figure 33. Source Code of the AgentService Sequential Model Lifecycle

Presented in Figure 33, the clear_context operation calls the model's own "reset" function, which wipes its short-term conversational memory without unloading the model itself from the graphics card. This is how the system keeps the Planner's three internal sub-steps cleanly separated from one another: the same underlying Qwen2.5-Coder model stays loaded throughout all three sub-steps, but its short-term memory is cleared between each one so that context from an earlier sub-step doesn't leak into and confuse the next.

The generate operation streams a model's response piece by piece, checking after every piece whether the user has disconnected, so that a response can be cancelled partway through if needed. A "repeat penalty" of 1.2 is applied to every call to discourage repetitive output. When a specific response format is required — for the Planner and the Judge — the system enforces "grammar-constrained decoding," meaning the model is given a strict template (a JSON schema) it must follow, guaranteeing its output is always in the expected structured format rather than free-form text.

Orchestration State and the Main Loop

All of the information that changes during a session is stored in one place: OrchestrationState, a fresh instance of which is created at the start of every session. It tracks the system's two retry loops: strategy_iter, the "outer loop" counter (starts at 1, capped at 3), and syntax_iter, the "inner," syntax-fixing loop counter (starts at 0, capped at 3). It also stores cumulative_feedback, a running list of the most recent problems found (capped at three entries) that the Planner reviews on any retry; intent_packet and active_plan, which hold the structured results passed between one stage and the next; working_code, which always holds the latest version of the candidate code, overwritten each time the Generator produces something new; and

exit_status, a flag that determines whether the main loop keeps running and, if not, how the session ended.

The main loop repeats as long as the session's status is still "processing," and on each pass it hands control to whichever stage the state says should run next. Each stage updates the shared state before handing control back — either moving the pipeline forward to the next stage, or sending it back to an earlier stage (2 or 3) to try again. A safety check, or "circuit breaker," runs after every stage to make sure the process cannot loop forever.

```
class OrchestrationState(BaseModel):
    session_id: str
    base_code: str
    working_code: str
    user_instruction: str
    intent_packet: dict | None = None
    active_plan:   dict | None = None
    strategy_iter: int = 1          # Outer Loop (Max 3)
    syntax_iter:   int = 0          # Inner Loop (Max 3)
    cumulative_feedback: list[dict] = []
    feedback_cap: int = 3
    syntax_error_context: dict | None = None
    exit_status: ExitStatus = ExitStatus.PROCESSING
    current_phase: int = 1

# Main orchestration loop (execute_orchestration)
while state.exit_status == ExitStatus.PROCESSING:
    if   state.current_phase == 2: await self._run_phase_2(client, state)
    elif state.current_phase == 3: await self._run_phase_3(client, state)
    elif state.current_phase == 4: await self._run_phase_4(client, state)
    elif state.current_phase == 5: await self._run_phase_5(client, state)
    elif state.current_phase == 6: break

    # Global circuit breaker
    if state.strategy_iter > 3:
        state.exit_status = ExitStatus.ABORT_STRATEGY
        state.current_phase = 6
        break
```

Figure 34. Source Code of the OrchestrationState Model and Main Orchestration Loop

As depicted in Figure 34, all of the information that changes during a session is stored in one place: OrchestrationState, a fresh instance of which is created at the start of every session. Every time the pipeline moves from one stage to the next, that update is sent live to the connected front-end application, which shows each stage as one of several states — active, waiting, completed successfully, completed with a

failure, flagged, or skipped — letting the interface display a live diagram of the process as it happens.

Logic: Structured Decomposition Across Phases

To ensure a clear and focused reasoning process, the Planner's workload is divided into three sequential sub-steps. After each step, the model's short-term memory is explicitly cleared. This prevents residual context from earlier tasks from polluting the current operation, allowing the system to build a progressively detailed refactoring plan without confusion.

```
async def run(self, client, state) -> None:
    if not state.intent_packet:
        await self._classify(client, state)        # Sub-step 1: Intent Classification

    await self._agent.clear_context()
    await self._analyze(client, state)             # Sub-step 2: Structural Analysis

    await self._agent.clear_context()
    await self._synthesize(client, state)          # Sub-step 3: Plan Synthesis

    if state.active_plan is None:
        return

    self._enrich(state)       # ASTMatcher: resolve targets against live AST
    self._deduplicate(state)  # Deduplication and mutation cap enforcement
    state.current_phase = 3
```

Figure 35. Source Code Implementation of the Planner's Sequential Decomposition and Context Isolation

As presented in Figure 35, This three-step progression forms the core of the system's AST-Constrained Semantic Alignment. Instead of treating code as plain text, every instruction is anchored directly to the Abstract Syntax Tree (AST). By mapping the user's intent to specific structural boundaries rather than relying on approximate string matching, the system ensures that modifications are exact. The workflow first categorizes the user's request, grounds it in concrete AST targets, and then synthesizes a step-by-step mutation plan.

Separating classification, analysis, and synthesis into distinct passes is a deliberate design choice. It prevents the model from attempting to recognize intent, analyze structure, and design code mutations all at once—a cognitive overload that frequently caused failures during early system testing. The Generator, on the other

hand, is dedicated entirely to writing code. To keep its workload light, it receives a simplified, plain-text version of the mutation plan, freeing it from the computational overhead of parsing complex structured data.

When it comes to generating the final code, candidate selection follows a similarly disciplined approach. The Generator produces multiple code samples at varying creativity levels (temperatures). However, rather than blindly trusting the model's preferred output, the system ranks these candidates using a strict, rule-based scoring function.

Every generated candidate is first validated using javalang to ensure the Java syntax is perfectly formed. Once a candidate proves to be syntactically valid, it is evaluated based on its Cyclomatic Complexity (CC)—a standard metric that measures how tangled or simplified the program's control flow has become.

```
samples: list[dict] = []
for sample_temp in (retry_temp, 0.3, 0.5):
    raw = await self._agent.generate(messages, temp=sample_temp, max_tokens=3072)
    coder_text = raw["choices"][0]["message"].get("content") or ""
    sample_code = ResponseParser.extract_xml(coder_text, "code")

    if sample_code:
        sample_code = repair_generator_output(state.base_code, sample_code)
        try:
            javalang.parse.parse(wrapped)
            syntax_ok = True
        except (javalang.parser.JavaSyntaxError, javalang.tokenizer.LexerError):
            syntax_ok = False

        cc = self._validator.get_complexity(sample_code) if syntax_ok else 999
        samples.append({
            "code": sample_code,
            "syntax_ok": syntax_ok,
            "cc": cc,
            "temp": sample_temp
        })

# Select: prefer syntactically valid, then lowest CC
def sample_score(s):
    if not s["syntax_ok"]:
        return (-1000, 0)
    return (0, -(s["cc"] - state.original_complexity))

best = max(samples, key=sample_score)
```

Figure 36. Source Code for Multi-Sample Candidate Generation and Lexicographic Deterministic Evaluation

The scoring function shown in Figure 36 prioritizes basic functionality over structural improvements. Syntactic validity acts as an absolute requirement, immediately disqualifying any broken code. Only candidates that successfully compile are compared against each other, with the system ultimately selecting the one that best reduces complexity compared to the original baseline. This strict hierarchy enforces a fundamental software engineering principle: a syntactically broken candidate is completely unusable, no matter how elegant its underlying logic might be.

Prompt Engineering: Schema Enforcement and Intent-Specific Guidance

To guarantee that the system can reliably interpret AI-generated responses, every inference call made by the Planner and Judge is paired with grammar-constrained decoding. Instead of hoping the model formats its output correctly, this technique forces the model to adhere token-by-token to a strict JSON schema derived from the system's Pydantic models. By enforcing structural validity exactly at the point of generation, the system eliminates an entire class of errors where a model reasons correctly but produces an unreadable format. Because the Generator must output raw Java code rather than structured JSON, it is handled differently—relying on XML tag extraction followed by a dedicated post-processing repair step.

```
# repair_generator_output() — applied before any validation
# 1. Strips `throws ExceptionX` not present in original
# 2. Removes null-check if-statements introduced by the model
# 3. Removes unauthorized `public` visibility modifiers
```

Figure 37. Excerpt of Post-Processing Repair Steps Applied to Generator Output

As outlined in Figure 37, these post-processing corrections specifically target the most common failure modes observed during system development. Small language models fine-tuned on general programming corpora often develop overly cautious habits, such as over-defending against exceptions and null values, or unnecessarily widening access modifiers to public even when the original codebase does not require it. Rather than wasting computational resources by re-prompting the

model for every minor slip, the system deterministically strips these unwanted artifacts from the generated Java code before it ever reaches the validation layer.

Beyond strict schema enforcement, the system dynamically enhances each agent's prompt with intent-specific guidance. Depending on the exact refactoring task classified by the Planner, the prompt is injected with relevant worked examples, strict constraints, and explicit anti-patterns tailored to that specific transformation.

```
ANTI-PATTERNS:
- Do NOT add new exception types, error messages, or throws that were not in the original code.
- Do NOT change any existing exception type or error message.
- Do NOT leave any if-inside-if nesting.
```

Figure 38. Prompt Excerpt Detailing Anti-Pattern Constraints for the Conditional-Flattening Refactor Intent

Figure 38 presents a representative example of this dynamic guidance, explicitly commanding the Generator to avoid introducing new exception types or leaving deeply nested conditional blocks. The Judge model is calibrated in the exact same spirit. Its prompt is populated with concrete ACCEPT versus REVISE precedents tailored to the specific refactoring task. This calibration is critical; without it, an AI Judge tends to flag any code transformation that looks superficially unfamiliar, even if it is structurally perfect. By supplying these intent-specific constraints—such as the anti-patterns shown in Figure 7—alongside worked precedents, the system anchors the AI's reasoning to structural correctness rather than surface-level familiarity.

Edge Case Handling: Layered Recovery Mechanism

Error recovery within the system is governed by the Dual-Loop Deterministic-Neural Routing (D2NR) algorithm. This mechanism divides failure handling between two nested circuit-breakers, which are designed to halt an escalating retry chain once a fixed limit is reached, preventing infinite loops. To manage this efficiently, the system first classifies failures into a formal taxonomy.

```
class FailureTier(str, Enum):
    TIER_1_SYNTAX       = "TIER_1_SYNTAX"
    TIER_2_A_COMPLEXITY  = "TIER_2_A_COMPLEXITY"
    TIER_2_B_BOUNDARY    = "TIER_2_B_BOUNDARY"
    TIER_2_C_INTENT_MATH = "TIER_2_C_INTENT_MATH"
    TIER_3_JUDGE         = "TIER_3_JUDGE"
```

Figure 39. Source Code Implementation of the FailureTier Enumeration Defining the D2NR Taxonomy

As defined in Figure 39, the FailureTier enumeration categorizes errors to route them to the appropriate recovery loop. The inner circuit-breaker handles TIER_1_SYNTAX failures deterministically: if a syntax error occurs, the offending code and the precise parser error message are immediately routed back to the Generator for a rapid correction attempt, capped at a maximum of three iterations. The outer circuit-breaker manages all structural or neural failures (Tiers 2 and 3)—such as complexity violations, boundary leaks where changes spill outside the intended scope, intent-specific AST arithmetic failures, and outright Judge rejections. Each of these outer failures appends a typed entry to the system's cumulative feedback log, which the Planner consults to synthesize a new strategy. Crucially, the escalation boundary between these two loops is dictated strictly by the failure tier; the system never relies on a probabilistic neural model to decide when to escalate a problem.

A specific edge case that motivated the design of the adjudication phase is "Judge hallucination." This occurs when the model acting as the Judge misinterprets the code it is evaluating—for example, falsely flagging an unmodified block for "logic drift," or claiming a genuinely refactored output remains unchanged. Because this vulnerability is an inherent risk of using small language models for evaluation, the system mitigates it using a strict deterministic override rather than trusting the model's unverified judgment.

```
# Hallucination guard: cross-check verdict against deterministic AST comparison
if audit_res.verdict == "REVISE" and audit_res.issues:
    issue = audit_res.issues[0]
    if issue.issue_type == "IDENTICAL_CODE":
        if self._validator.has_structural_change(state.base_code, state.working_code):
            audit_res.verdict = "ACCEPT"
            audit_res.issues = []
    elif issue.issue_type == "LOGIC_DRIFT":
        if not self._validator.has_structural_change(state.base_code, state.working_code):
            audit_res.verdict = "ACCEPT"
            audit_res.issues = []
```

Figure 40. Source Code for Deterministic Override Logic Correcting Hallucinated Judge Verdicts

Figure 40 details this deterministic override logic. The safeguard relies on a SHA-256 comparison of the code's AST signature—a cryptographic fingerprint that definitively confirms whether the underlying code structure has changed, independent of surface-level formatting or wording. Whenever the Judge's semantic verdict contradicts this mathematical comparison, the deterministic check overrides the AI. The model's verdict is never accepted unconditionally on its own.

If a refactoring attempt triggers the circuit breakers and exhausts every available recovery avenue, the system initiates a final failsafe during Phase 6, falling back to a secure default state to protect the user's codebase.

```
# Graceful degradation: original code as safe fallback on any abort
final_code = state.working_code if state.exit_status == ExitStatus.SUCCESS \
             else state.base_code

await client.send_result(final_code=final_code, ...)

if state.exit_status == ExitStatus.SUCCESS:
    insights = await self._generate_insights(...)
else:
    abort_reasons = {
        ExitStatus.ABORT_STRATEGY: (
            "Refactoring aborted after max retries. "
            "The system attempted multiple strategies but could not produce valid code. "
            "Original code restored."
        ),
        ExitStatus.ABORT_SYNTAX: (
            "Refactoring aborted — generated code contained syntax errors "
            "that could not be resolved. Original code restored."
        ),
    }
    insights = abort_reasons.get(state.exit_status,
                                 "Refactoring aborted. Original code restored.")
```

Figure 41. Source Code for Graceful-Degradation Logic Restoring the Original Code on Abort

The graceful degradation logic presented in Figure 40 enforces a simple but critical software engineering principle: the user is never exposed to a half-finished, uncompilable, or broken refactoring. If the multi-agent process succeeds, the system returns the finalized working code alongside a generated insight summary. However, if the process fails and the maximum retry limits are reached, the system quietly restores the original, unmodified code and reports the specific abort reason. This guarantees that the user is always left with a functional codebase, preventing the tool from ever delivering a structural regression.

Taken together, these recovery paths function not as isolated exception handlers, but as integrated layers of a unified safety net. Every category of failure the pipeline might encounter—from a malformed user request to an unhandled exception deep within the orchestration logic—is intercepted at a specific layer and routed to a predictable, controlled outcome. Table 34 lays out this full correspondence end-to-end, detailing the failure scenario, the layer responsible for detecting it, the algorithmic recovery path, and what the user ultimately sees.

Table 34. Failure Scenarios, Detection Layers, and Recovery Pathways

FAILURE SCENARIO	DETECTION LAYER	RECOVERY PATH	USER OUTCOME
Invalid / too-short input	Pydantic RefactorRequest	WebSocket error payload	Error message, no session
Non-Java or syntactically broken input	Phase 1 (check_semantics, check_syntax)	ABORT_INPUT or force_proceed	Interactive warning
Planner produces malformed JSON	Pydantic + schema decoder	Outer-loop retry	Transparent
Generator produces no code block	XML tag extraction returns None	Inner-loop syntax heal	Transparent
Generated code has syntax errors	javalang.parse.parse()	Up to 3 inner heals → outer escalation	Phase flagged
Structural check failure	RefactorVerifier registry	1 structural heal → outer escalation	Phase flagged
Judge hallucination	SHA-256 AST override	Verdict overridden to ACCEPT	Transparent
Judge/Validator rejects all 3 outer attempts	strategy_iter > 3	ABORT_STRATEGY + rollback	Abort message + rollback
User cancels mid-run	agent_service.stop()	InterruptedError → DB marked Halted	Halt notification
Unexpected exception	except Exception in orchestrator	DB marked failed, VRAM released	Error payload to frontend

As noted in the table, the system uses a persistent WebSocket connection to push live updates directly to the interface. More importantly, the "Transparent" and "Phase flagged" outcomes highlight that the vast majority of processing errors are absorbed silently by the retry loops and are never exposed to the user as fatal crashes.

Ultimately, this architecture proves that complex Java refactoring can be safely automated on consumer-grade hardware. By carefully scheduling memory usage, anchoring AI instructions to concrete code structures, and enforcing strict fallback rules, the system ensures that no single AI mistake ruins the final output. The Planner learns from feedback, the Generator is constrained by syntax checks, and the Judge is supervised by structural math. These safeguards successfully maintained control during the benchmark evaluations, proving that automated recovery under strict hardware constraints is a highly viable reality.

CHAPTER V
SUMMARY OF FINDINGS, CONCLUSION, AND RECOMMENDATIONS

This chapter presents the summary of significant findings, conclusions, and recommendations derived from the development and evaluation of HorizonAI, a role-driven multi-agent system for automated Java code refactoring under local, resource-constrained execution. The findings summarize the measurable outcomes obtained relative to the specific problems established in Chapter I, the conclusions interpret these findings in relation to the study's objectives, and the recommendations identify areas for improving the developed system and extending future research.

Summary of Significant Findings

This section presents the significant findings of the study based on the specific problems outlined in Chapter I. The findings were derived from the development and testing of HorizonAI, its benchmark evaluation against the single-model baseline, and the evaluation results gathered from end users and Information Technology experts. Each finding directly corresponds to the identified specific problems and reflects the measurable outcomes obtained from the study.

1. HorizonAI was implemented as a role-driven multi-agent system that separated planning, generation, and evaluation stages while integrating deterministic validation and bounded feedback mechanisms. The validation layer screened generated candidates according to syntax, complexity, boundary, intent, and evaluation conditions rather than accepting them based on successful generation alone. The system achieved a 78.5% resolution rate across the 279 benchmark tasks, with the feedback mechanism allowing some initially unsuccessful cases to proceed through additional processing. However, unresolved cases remained after the feedback process was applied.

2. The comparison of HorizonAI with the single-model baseline revealed that neither architecture demonstrated consistent superiority across all evaluation measures. The baseline achieved higher functional success indicators, obtaining a Compilation Success Rate of 67.7% compared with HorizonAI's 58.4%, and a Behavioral

Equivalence Rate of 11.1% compared with HorizonAI's 9.8%. Meanwhile, HorizonAI produced more favorable structural quality measurements among successfully compiled outputs, with a lower mean Cyclomatic Complexity of 4.53 against the baseline's 4.90, and a higher Maintainability Index of 53.99 against the baseline's 51.88.

3. The evaluation of computational efficiency showed a trade-off between processing time and memory usage introduced by HorizonAI's multi-stage orchestration workflow. The system required additional processing time because refactoring tasks passed through multiple stages involving specialized agents, validation checks, and feedback cycles, resulting in an average execution time of 97.26 seconds compared with 19.47 seconds for the single-model baseline. Despite this latency increase, HorizonAI achieved a lower peak GPU-memory requirement of 3,232 MB compared with 4,044 MB from the baseline, allowing the system to operate within the study's 4-gigabyte VRAM limitation.

4. The evaluation of the Glassbox interface by twenty-five end users showed that the system was perceived as Highly Acceptable across interaction capability, functional suitability, security, and reliability, with an overall mean of 3.71. Reliability obtained the highest mean rating (3.76), while security obtained the comparatively lowest mean rating (3.65) among the four evaluated characteristics.

5. HorizonAI was evaluated by Information Technology experts based on functional suitability, performance efficiency, reliability, and maintainability, following the ISO/IEC 25010:2023 quality model. The evaluation results showed an overall mean of 3.45, interpreted as Strongly Agree, indicating that the experts agreed that the developed system satisfied the assessed technical quality requirements. Among the four characteristics, maintainability obtained the highest mean rating of 3.58, while performance efficiency obtained the lowest mean rating of 3.35.

Conclusions

This section presents the conclusions drawn from the findings of the study on the development and evaluation of HorizonAI. Based on the findings, the study demonstrated that HorizonAI is feasible for local execution and provides measurable advantages in structural code quality and memory efficiency, while also revealing trade-offs and limitations relative to the single-model baseline and areas for further refinement based on user and expert feedback.

1. The study concludes that HorizonAI was successfully developed and evaluated as a role-driven multi-agent orchestration architecture using smaller language models while incorporating deterministic validation and bounded feedback mechanisms. The separation of planning, generation, and evaluation made the processing stages explicit and allowed failed candidates to be detected and routed rather than automatically accepted. However, the presence of unresolved benchmark tasks demonstrates that orchestration and validation do not eliminate generation failures. The primary contribution of the architecture lies in making such failures more observable, controlled, and bounded within the system workflow.

2. The study concludes that HorizonAI achieved mixed rather than universally superior refactoring performance compared with the single-model baseline. The larger single-model architecture retained an advantage in Compilation Success Rate and Behavioral Equivalence Rate, indicating greater success in producing compilable and behavior-preserving outputs across the evaluated tasks. HorizonAI, however, produced lower mean Cyclomatic Complexity and higher mean Maintainability Index values among the applicable compiled outputs, indicating advantages in structural organization and maintainability-related measures. Role-driven orchestration should therefore be understood as an architectural trade-off that provides additional validation and structural control rather than as a guarantee of higher performance across every code-quality measure.

3. The study concludes that HorizonAI is feasible for local multi-agent execution within the defined 4-gigabyte VRAM constraint. Strict Sequential Inference Scheduling enabled the system to execute its specialized roles while using

substantially less peak GPU memory than the 7-billion-parameter baseline. This memory advantage was accompanied by considerably longer inference time because the refactoring process required multiple sequential model and validation stages. HorizonAI is therefore better characterized as memory-efficient and locally feasible, prioritizing local feasibility and reduced memory demand at the cost of increased execution time, rather than as speed-efficient relative to the single-model baseline.

4. The study concludes that the evaluated users perceived the Glassbox interface as Highly Acceptable in terms of interaction capability, functional suitability, security, and reliability. The results indicate that the interface successfully exposed relevant system information and maintained a generally understandable workflow for reviewing the multi-agent process. Nevertheless, comparatively lower ratings in interface operability and the communication of code-handling practices show that transparency of internal processing should be complemented by clearer navigation, controls, and explanations of local data handling.

5. The IT expert evaluation demonstrates that HorizonAI achieved a technically acceptable level of quality across functional suitability, performance efficiency, reliability, and maintainability, with an overall mean of 3.45 interpreted as Strongly Agree. This addresses the objective of determining how Information Technology experts perceived the technical quality of the developed system. The high maintainability rating signifies that the experts recognized the value of the system's modular architecture and structured validation mechanisms, while the comparatively lower performance efficiency rating reflects the increased execution time associated with the multi-agent workflow observed during benchmark evaluation. This indicates that while the developed architecture was perceived as technically sound and maintainable, further improvement in processing efficiency may be considered to strengthen the system's overall performance.

Recommendations

The following recommendations are based on the findings and conclusions of the study. These proposed improvements aim to enhance the model flexibility, processing efficiency, and practical adoptability of the developed HorizonAI system.

1. Future researchers should explore alternative open-source language models for the Planner, Generator, and Judge roles under the same resource-constrained execution conditions used in this study. This may help identify which model combinations are best suited for each specialized role and further improve refactoring performance.

2. System developers extending HorizonAI may implement an optional model configuration feature that allows technically capable users to select or load compatible models for the Planner, Generator, and Judge on a per-session basis. The validated model configuration used in the present study may remain as the default setup, while configurable sessions can serve as an experimental mode for comparing alternative model combinations. This would extend the platform from a fixed orchestration implementation into a more flexible environment for studying how role assignment and model selection affect refactoring performance.

3. Future researchers and system developers should focus further optimization on improving the system's time behaviour without removing the validation and resource-management mechanisms that enabled HorizonAI to operate within the defined hardware constraint. Areas such as model loading overhead, repeated inference calls, prompt and context size, and retry behaviour may be examined as contributors to end-to-end execution time. This may reduce the system's execution time and improve its practicality for time-sensitive refactoring workflows.

4. Local development teams, MSMEs, and academic institutions with consumer-grade hardware comparable to the study's 4-gigabyte VRAM configuration should pilot HorizonAI to evaluate its fit for their own refactoring, deployment, or instructional needs. This may allow these organizations to assess the system's demonstrated memory efficiency and local execution feasibility as a practical, cost-sustainable alternative to cloud-based tools.

5. Future studies may conduct broader configuration and hardware evaluations to examine how HorizonAI behaves under different consumer-grade computing environments. Comparisons involving model size, quantization level, available GPU memory, and role configurations may help identify practical combinations that offer a better balance among output quality, inference time, and resource utilization. Such evaluations can extend the findings of the present study and provide stronger guidance for deploying role-driven Small Language Model systems across a wider range of locally available hardware.

Literature Cited

[1] K. F. Pilz, J. Sanders, R. Rahman, and L. Heim, “Trends in AI Supercomputers,” 2025, *arXiv*. doi: 10.48550/ARXIV.2504.16026.

[2] M. Mills, “The Rise of AI: A Reality Check on Energy and Economic Impacts,” National Center for Energy Analytics, 2025. [Online]. Available: https://energyanalytics.org/wp-content/uploads/2025/11/Mills_HAMM_AI_1125_Final_01.pdf

[3] M.-A. Storey, “From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI,” 2026, *arXiv*. doi: 10.48550/ARXIV.2603.22106.

[4] X. Hou *et al.*, “Large Language Models for Software Engineering: A Systematic Literature Review,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, Nov. 2024, doi: 10.1145/3695988.

[5] J. C. Gonçalves and M. D. A. Maia, “An Empirical Study on the Effectiveness of Iterative LLM-Based Improvements for Static Analysis Issues,” in *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*, Brasil: Sociedade Brasileira de Computação, Sep. 2025, pp. 382–392. doi: 10.5753/sbes.2025.9964.

[6] I. Jaoua, O. B. Sghaier, and H. Sahraoui, “Combining Large Language Models with Static Analyzers for Code Review Generation,” 2025, *arXiv*. doi: 10.48550/ARXIV.2502.06633.

[7] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities,” 2024, *arXiv*. doi: 10.48550/ARXIV.2405.17238.

[8] A. Nikolakopoulos, A. Litke, A. Psychas, E. Veroni, and T. Varvarigou, “Exploring the Potential of Offline LLMs in Data Science: A Study on Code Generation for Data Analysis,” *IEEE Access*, vol. 13, pp. 64087–64114, 2025, doi: 10.1109/ACCESS.2025.3556973.

[9] M. Goyal and P. Bhasin, “Moving From Monolithic To Microservices Architecture for Multi-Agent Systems,” *World J. Adv. Eng. Technol. Sci.*, vol. 15, no. 1, pp. 2119–2124, Apr. 2025, doi: 10.30574/wjaets.2025.15.1.0480.

[10] H. Hays, “Attention mechanisms in neural networks,” Jan. 06, 2026, *arXiv*: arXiv:2601.03329. doi: 10.48550/arXiv.2601.03329.

[11] P. Belcak *et al.*, “Small Language Models are the Future of Agentic AI,” Sep. 15, 2025, *arXiv*: arXiv:2506.02153. doi: 10.48550/arXiv.2506.02153.

[12] Z. Ren, T. Zhang, and W. Chen, “Decoupling Correctness from Policy: A Deterministic Causal Structure for Multi-Agent Systems,” Oct. 07, 2025, *arXiv*: arXiv:2510.05621. doi: 10.48550/arXiv.2510.05621.

[13] B. Yang, X. He, H. Gao, Y. Cao, X. Li, and D. Hsu, “CodeAgents: A Token-Efficient Framework for Codified Multi-Agent Reasoning in LLMs,” 2025, *arXiv*. doi: 10.48550/ARXIV.2507.03254.

[14] T. Händler, “A Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures:,” in *Proceedings of the 15th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*, Rome, Italy: SCITEPRESS - Science and Technology Publications, 2023, pp. 85–98. doi: 10.5220/0012239100003598.

[15] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, no. 1, p. 9, Oct. 2024, doi: 10.1007/s44336-024-00009-2.

[16] C. Yu *et al.*, “A Survey on Agent Workflow -- Status and Future,” in *2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD)*, May 2025, pp. 770–781. doi: 10.1109/ICAIBD64986.2025.11082076.

[17] Z. Zhang *et al.*, “A Survey on the Memory Mechanism of Large Language Model based Agents,” 2024, *arXiv*. doi: 10.48550/ARXIV.2404.13501.

[18] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” 2023, *arXiv*. doi: 10.48550/ARXIV.2309.06180.

[19] Y. Zhao *et al.*, “Atom: Low-bit Quantization for Efficient and Accurate LLM Serving,” 2023, *arXiv*. doi: 10.48550/ARXIV.2310.19102.

[20] E. Tabassi, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” National Institute of Standards and Technology (U.S.), Gaithersburg, MD, NIST AI 100-1, Jan. 2023. doi: 10.6028/NIST.AI.100-1.

[21] ISO/IEC 23894:2023, “Information technology — Artificial intelligence — Guidance on risk management,” Feb. 2023.

[22] X.-W. Yang *et al.*, “Neuro-Symbolic Artificial Intelligence: Towards Improving the Reasoning Abilities of Large Language Models,” in *Proceedings of the Thirty-*

Fourth International Joint Conference on Artificial Intelligence, Montreal, Canada: International Joint Conferences on Artificial Intelligence Organization, Sep. 2025, pp. 10770–10778. doi: 10.24963/ijcai.2025/1195.

[23] S. O. Semerikov, T. A. Vakaliuk, O. B. Kanevska, O. A. Ostroushko, and A. O. Kolhatin, "Edge intelligence unleashed: a survey on deploying large language models in resource-constrained environments," *J. Edge Comput.*, vol. 4, no. 2, pp. 179–233, Nov. 2025, doi: 10.55056/jec.1000.

[24] J. He, C. Treude, and D. Lo, "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead," Jul. 18, 2025, *arXiv*: arXiv:2404.04834. doi: 10.48550/arXiv.2404.04834.

[25] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," 2023, *arXiv*. doi: 10.48550/ARXIV.2307.07924.

[26] N. Mündler, J. He, H. Wang, K. Sen, D. Song, and M. Vechev, "Type-Constrained Code Generation with Language Models," 2025, doi: 10.48550/ARXIV.2504.09246.

[27] Stack Overflow, "2025 Developer Survey: AI," Stack Overflow, 2025.

[28] L. Felder, J. Miller, M. Wallinger, S. Kobourov, and C. Chen, "ReTrace: Interactive Visualizations for Reasoning Traces of Large Reasoning Models," Nov. 14, 2025, *arXiv*: arXiv:2511.11187. doi: 10.48550/arXiv.2511.11187.

[29] E. Bernardo and R. Seva, "Affective Design Analysis of Explainable Artificial Intelligence (XAI): A User-Centric Perspective," *Informatics*, vol. 10, no. 1, p. 32, Mar. 2023, doi: 10.3390/informatics10010032.

[30] "ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," *Int. Organ. Stand.*, [Online]. Available: https://www.iso.org/standard/78175.html

[31] B. Seres, D. Horpácsi, and S. Thompson, "Is This Really a Refactoring? Automated Equivalence Checking for Erlang Projects," in *Proceedings of the 23rd ACM SIGPLAN International Workshop on Erlang*, Milan Italy: ACM, Aug. 2024, pp. 55–66. doi: 10.1145/3677995.3678194.

[32] S. Saha, A. A. Hasan, F. T. Shifat, and M. M. Imran, “Improving Code Comprehension through Cognitive-Load Aware Automated Refactoring for Novice Programmers,” 2026, *arXiv*. doi: 10.48550/ARXIV.2603.16791.

[33] J. Guo *et al.*, “CodeEditorBench: Evaluating Code Editing Capability of Large Language Models,” 2024, *arXiv*. doi: 10.48550/ARXIV.2404.03543.

[34] M. Jahrens and T. Martinetz, “Why LLMs Cannot Think and How to Fix It,” 2025, *arXiv*. doi: 10.48550/ARXIV.2503.09211.

[35] Val Andrei Fajardo, *Build a Multi-Agent System (from Scratch)*. Manning Publications, 2025.

[36] ANTHONY TINLINE, *The multi-agent systems handbook: Leveraging MCP and A2A protocols to build modular, self-evolving AI workforces*.

[37] E. Pantazis, *Designing with multi-agent systems: a computational methodology for form-finding using behaviors*. in De Gruyter STEM. Berlin Boston: De Gruyter, 2024. doi: 10.1515/9783110797435.

[38] Emergent Mind, “Planner-Executor-Evaluator Loop.” [Online]. Available: https://www.emergentmind.com/topics/planner-executor-evaluator-loop

[39] J. Fadlon, “Modular intelligence: a human-like model for agent orchestration,” AI21. [Online]. Available: https://www.ai21.com/blog/modular-intelligence-agent-orchestration/

[40] A. Adimulam, R. Gupta, and S. Kumar, “The Orchestration of Multi-Agent Systems: Architectures, Protocols, and Enterprise Adoption,” Jan. 20, 2026, *arXiv*: arXiv:2601.13671. doi: 10.48550/arXiv.2601.13671.

[41] T. Zhou, J. Xu, G. Liu, J. Liu, H. Wang, and E. Wu, “An approach for systematic decomposition of complex llm tasks,” Jan. 18, 2026, *arXiv*: arXiv:2510.07772. doi: 10.48550/arXiv.2510.07772.

[42] A. d’Avila Garcez and L. C. Lamb, “Neurosymbolic AI: the 3rd wave,” *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 12387–12406, Nov. 2023, doi: 10.1007/s10462-023-10448-w.

[43] P. Hitzler and M. K. Sarker, Eds., *Neuro-Symbolic Artificial Intelligence: The State of the Art*, vol. 342. in Frontiers in Artificial Intelligence and Applications, vol. 342. IOS Press, 2021. doi: 10.3233/FAIA342.

[44] R. Manhaeve, S. Dumančić, A. Kimmig, T. Demeester, and L. D. Raedt, "DeepProbLog: Neural Probabilistic Logic Programming," Dec. 12, 2018, *arXiv*: arXiv:1805.10872. doi: 10.48550/arXiv.1805.10872.

[45] N. Wadhwa *et al.*, "CORE: Resolving Code Quality Issues using LLMs," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 789–811, Jul. 2024, doi: 10.1145/3643762.

[46] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, Apr. 2026, pp. 2938–2950. doi: 10.1145/3744916.3764546.

[47] M. Brain and E. Polgreen, "A Pyramid Of (Formal) Software Verification," in *Formal Methods*, vol. 14934, A. Platzer, K. Y. Rozier, M. Pradella, and M. Rossi, Eds., in Lecture Notes in Computer Science, vol. 14934. , Cham: Springer Nature Switzerland, 2025, pp. 393–419. doi: 10.1007/978-3-031-71177-0_24.

[48] A. Silberschatz, P. Galvin, and G. Gagne, 10th ed. Wiley, 2023.

[49] L. Yang, "Research on Mutual Exclusion and Condition Synchronization in Concurrent Programming," in *2021 3rd International Academic Exchange Conference on Science and Technology Innovation (IAECST)*, Guangzhou, China: IEEE, Dec. 2021, pp. 745–749. doi: 10.1109/IAECST54258.2021.9695664.

[50] M. M. Bersani, M. Camilli, L. Lestingi, R. Mirandola, M. Rossi, and P. Scandurra, "A Conceptual Framework for Explainability Requirements in Software-Intensive Systems," in *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*, Hannover, Germany: IEEE, Sep. 2023, pp. 309–315. doi: 10.1109/REW57809.2023.00059.

[51] T. Mitchell, "Trust and Transparency in Artificial Intelligence," *Philos. Technol.*, vol. 38, no. 3, p. 87, Sep. 2025, doi: 10.1007/s13347-025-00916-2.

[52] W. J. Von Eschenbach, "Transparency and the Black Box Problem: Why We Do Not Trust AI," *Philos. Technol.*, vol. 34, no. 4, pp. 1607–1622, Dec. 2021, doi: 10.1007/s13347-021-00477-0.

[53] A. F. T. Winfield *et al.*, "IEEE P7001: A Proposed Standard on Transparency," *Front. Robot. AI*, vol. 8, p. 665729, Jul. 2021, doi: 10.3389/frobt.2021.665729.

[54] B. Hui *et al.*, "Qwen2.5-Coder Technical Report," Nov. 12, 2024, *arXiv*: arXiv:2409.12186. doi: 10.48550/arXiv.2409.12186.

[55] Tung, Nguyen, and T. Nguyen, "An Evaluation of LLMs Inference on Popular Single-board Computers," Oct. 20, 2025, *arXiv*: arXiv:2511.07425. doi: 10.48550/arXiv.2511.07425.

[56] P. A. Golnari *et al.*, "DevBench: A Realistic, Developer-Informed Benchmark for Code Generation Models," Mar. 07, 2026, *arXiv*: arXiv:2601.11895. doi: 10.48550/arXiv.2601.11895.

[57] Y. Kim *et al.*, "Towards a Science of Scaling Agent Systems," 2025, *arXiv*. doi: 10.48550/ARXIV.2512.08296.

[58] Y. Xu, F. Lin, J. Yang, Tse-Hsun, Chen, and N. Tsantalis, "MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration," Mar. 27, 2025, *arXiv*: arXiv:2503.14340. doi: 10.48550/arXiv.2503.14340.

[59] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation," 2023, *arXiv*. doi: 10.48550/ARXIV.2312.13010.

[60] A. H. Fernando *et al.*, "Simulated vs Actual Application of Symbiotic Model on Six Wheel Modular Multi-Agent System for Linear Traversal Mission," *J. Adv. Comput. Intell. Intell. Inform.*, vol. 28, no. 1, pp. 12–20, Jan. 2024, doi: 10.20965/jaciii.2024.p0012.

[61] A. D. Guia and M. A. Ballera, "Multi-agent class timetabling for higher educational institutions using prometheus platform," *Indones. J. Electr. Eng. Comput. Sci.*, vol. 22, no. 3, p. 1679, Jun. 2021, doi: 10.11591/ijeecs.v22.i3.pp1679-1687.

[62] J. Cordeiro, S. Noei, and Y. Zou, "An Empirical Study on the Code Refactoring Capability of Large Language Models," Nov. 04, 2024, *arXiv*: arXiv:2411.02320. doi: 10.48550/arXiv.2411.02320.

[63] A. Shirafuji, Y. Oda, J. Suzuki, M. Morishita, and Y. Watanobe, "Refactoring Programs Using Large Language Models with Few-Shot Examples," in *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*, Dec. 2023, pp. 151–160. doi: 10.1109/APSEC60848.2023.00025.

[64] F. Li, J. Jiang, J. Sun, and H. Zhang, “Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis,” 2024, *arXiv*. doi: 10.48550/ARXIV.2406.00992.

[65] Dan Alistarh, Mark Kurtz, Alexandre Marques, and Eldar Kurtić, “We ran over half a million evaluations on quantized LLMs—here’s what we found.” [Online]. Available: https://developers.redhat.com/articles/2024/10/17/we-ran-over-half-million-evaluations-quantized-llms

[66] E. Nyamsuren, “Evaluating Quantized Large Language Models for Code Generation on Low-Resource Language Benchmarks,” 2024, *arXiv*. doi: 10.48550/ARXIV.2410.14766.

[67] World Bank, “World Bank Helps Boost Digital Transformation in the Philippines,” Nov. 2024. [Online]. Available: https://www.worldbank.org/en/news/press-release/2024/11/12/world-bank-helps-boost-digital-transformation-in-ph

[68] J. Hollan, E. Hutchins, and D. Kirsh, “Distributed cognition: toward a new foundation for human-computer interaction research,” *ACM Trans. Comput.-Hum. Interact.*, vol. 7, no. 2, pp. 174–196, Jun. 2000, doi: 10.1145/353485.353487.

[69] The Decision Lab, “Human-AI collaboration.” [Online]. Available: https://thedecisionlab.com/reference-guide/computer-science/human-ai-collaboration

[70] ELM Learning, “Constructivism learning theory.”

[71] C. Sun, Y. Sheng, O. Padon, and C. Barrett, “Clover: Closed-Loop Verifiable Code Generation,” Nov. 16, 2024, *arXiv*: arXiv:2310.17807. doi: 10.48550/arXiv.2310.17807.

[72] C. Wang *et al.*, “Slm-mux: Orchestrating small language models for reasoning,” Oct. 06, 2025, *arXiv*: arXiv:2510.05077. doi: 10.48550/arXiv.2510.05077.

[73] E. Y. Chang, *Multi-LLM Agent Collaborative Intelligence: The Path to Artificial General Intelligence*, 1st ed. New York, NY, USA: ACM, 2025. doi: 10.1145/3749421.

[74] T. Dettmers and L. Zettlemoyer, “The case for 4-bit precision: k-bit Inference Scaling Laws,” Feb. 28, 2023, *arXiv*: arXiv:2212.09720. doi: 10.48550/arXiv.2212.09720.

[75] Z. Lu *et al.*, "Small Language Models: Survey, Measurements, and Insights," Feb. 26, 2025, *arXiv*: arXiv:2409.15790. doi: 10.48550/arXiv.2409.15790.

[76] S. Baltes and P. Ralph, "Sampling in software engineering research: a critical review and guidelines," *Empir. Softw. Eng.*, vol. 27, no. 4, p. 94, Jul. 2022, doi: 10.1007/s10664-021-10072-8.

[77] R. B. Bouncken, W. Czakon, and F. Schmitt, "Purposeful sampling and saturation in qualitative research methodologies: recommendations and review," *Rev. Manag. Sci.*, vol. 20, no. 2, pp. 579–615, Feb. 2026, doi: 10.1007/s11846-025-00881-2.

[78] Tung, Nguyen, and T. Nguyen, "An Evaluation of LLMs Inference on Popular Single-board Computers," Oct. 20, 2025, *arXiv*: arXiv:2511.07425. doi: 10.48550/arXiv.2511.07425.

[79] M. Sharma *et al.*, "Towards Understanding Sycophancy in Language Models," May 10, 2025, *arXiv*: arXiv:2310.13548. doi: 10.48550/arXiv.2310.13548.

[80] Z. Gu *et al.*, "AgentGroupChat-V2: Divide-and-Conquer Is What LLM-Based Multi-Agent System Need," 2025, *arXiv*. doi: 10.48550/ARXIV.2506.15451.

[81] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," 2025, *arXiv*. doi: 10.48550/ARXIV.2501.06322.

[82] J. Liang, G. Su, H. Lin, Y. Wu, R. Zhao, and Z. Li, "Reasoning RAG via System 1 or System 2: A Survey on Reasoning Agentic Retrieval-Augmented Generation for Industry Challenges," 2025, *arXiv*. doi: 10.48550/ARXIV.2506.10408.

[83] K. Oueslati, M. Lamothe, and F. Khomh, "RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring," 2025, *arXiv*. doi: 10.48550/ARXIV.2511.03153.

[84] S. Martínez-Fernández *et al.*, "Software Engineering for AI-Based Systems: A Survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, pp. 1–59, Apr. 2022, doi: 10.1145/3487043.

[85] M. Evtikhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, "Out of the BLEU: How should we assess quality of the Code Generation models?," *J. Syst. Softw.*, vol. 203, p. 111741, Sep. 2023, doi: 10.1016/j.jss.2023.111741.

[86] Y. Zhang *et al.*, “Integrating Artificial Intelligence into Operating Systems: A Survey on Techniques, Applications, and Future Directions,” Nov. 11, 2025, *arXiv*: arXiv:2407.14567. doi: 10.48550/arXiv.2407.14567.

[87] S. de Oliveira, “A Comparative Analysis of LLM-Based Multi-Agent Frameworks,” Master’s Thesis, University of L’Aquila, 2025. [Online]. Available: https://www.doria.fi/bitstream/handle/10024/193122/serafim_de_oliveira_mariana_celi.pdf?sequence=2

[88] P. Ralph *et al.*, “Empirical Standards for Software Engineering Research,” 2021, *arXiv*. doi: 10.48550/ARXIV.2010.03525.

[89] E. Trichkova-Kashamova, “Applying the ISO/IEC 25010 Quality Models to an Assessment Approach for Information Systems,” in *2021 12th National Conference with International Participation (ELECTRONICA)*, Sofia, Bulgaria: IEEE, May 2021, pp. 1–4. doi: 10.1109/ELECTRONICA52725.2021.9513662.

APPENDIX A

Confidentiality and Non-Disclosure Agreement

Republic of the Philippines
Pamantasan ng Cabuyao
(UNIVERSITY OF CABUYAO)
Planning, Research, and Extension Division
Research and Development Department
Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

CONFIDENTIALITY AND NON-DISCLOSURE AGREEMENT (NDA) ON STUDENT RESEARCH

(PNC is a research data site)

This Confidentiality and Non-Disclosure Agreement ("Agreement") is made and entered into on **March 30, 2026** between **Christian V. Balinado, Andrew D. Dejito, Joshua P. Lopez, Jericho F. Varde** ("Recipient") and **Pamantasan ng Cabuyao (University of Cabuyao)** ("Disclosing Party") in connection with the proposed student research project titled **"Horizontal Intelligence: Optimizing Small Language Model Reasoning Via Role-Driven Multi-Agent Orchestration."**

Confidential Information. For purposes of this Agreement, "Confidential Information" means any and all information disclosed by the Disclosing Party to the Recipient, including without limitation, any and all data, research findings, results, and reports related to the proposed student research project.

Obligations of Recipient. Recipient agrees to hold all Confidential Information in strict confidence and to take all reasonable steps to prevent unauthorized access or disclosure of such Confidential Information. Recipient agrees not to use any Confidential Information for any purpose other than to complete the proposed student research project.

Non-Disclosure. Recipient agrees not to disclose any Confidential Information to any third party without the prior written consent of the Disclosing Party. Recipient further agrees not to disclose any Confidential Information to any employee, agent, or contractor of the Recipient, except on a need-to-know basis and only to the extent necessary to complete the proposed student research project.

Ownership. The Disclosing Party retains ownership of all Confidential Information disclosed to the Recipient, including all data, research findings, results, and reports related to the proposed student research project.

Term. This Agreement shall remain in effect for a period of **one year** from the date of execution. Recipient's obligations under this Agreement shall survive the termination or expiration of this Agreement.

Remedies. The Recipient acknowledges that any breach of this Agreement may cause irreparable harm to the Disclosing Party. In addition to any other remedies available at law or in equity, the Disclosing Party may seek injunctive relief to prevent any breach or threatened breach of this Agreement.

Governing Law. This Agreement shall be governed by and construed in accordance with the laws of Laguna, Philippines without giving effect to any principles of conflicts of law.

Republic of the Philippines
Pamantasan ng Cabuyao
(UNIVERSITY OF CABUYAO)
Planning, Research, and Extension Division
Research and Development Department
Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

Entire Agreement. This Agreement constitutes the entire agreement between the parties concerning the subject matter hereof and supersedes all prior negotiations, discussions, or agreements between the parties.

IN WITNESS WHEREOF, the parties have executed this Agreement as of the date first above written.

Pamantasan ng Cabuyao (University of Cabuyao)

DR. JESSICA D. REY
Vice President for Planning, Research, and Extension

Student Researchers

CHRISTIAN V. BALNADO

ANDREW D. DEJITO

JOSHUA P. LOPEZ

JERICHO F. VARDE

WITNESSES

DR. GEORGE V. LAMBOT
Vice President for Academics and Student Services

DR. GIMA B. MONTECILLO
Dean, College of Computing Studies

Dangal ng Bayan.
bringing pride and honor to the nation

Republic of the Philippines
Pamantasan ng Cabuyao
(UNIVERSITY OF CABUYAO)
Planning, Research, and Extension Division
Research and Development Department
Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

ACKNOWLEDGMENT

REPUBLIC OF THE PHILIPPINES)
CITY OF _____ City of Cabuyao, Laguna

BEFORE ME, a Notary Public for and in City of Cabuyao, on APR 06 2026, personally appeared the following, to wit:

Name	Valid ID / Passport Number	Date and Place Issued
CHRISTIAN V. BALINADO	2503-6098-6187-1905	07 AUGUST 2023 CITY OF CABUYAO, LAGUNA
ANDREW D. DEJITO	6483-7154-7521-6306	13 FEBRUARY 2024 CITY OF CABUYAO, LAGUNA
JOSHUA P. LOPEZ	6071-3025-8150-8538	22 NOVEMBER 2022 SOCORRO, ORIENTAL MINDORO
JERICHO F. VARDE	6238-9219-4670-4359	08 AUGUST 2023 CITY OF CABUYAO, LAGUNA

KNOWN TO ME to be the same persons who executed the foregoing Grant Agreement consisting of ___ pages including this page and acknowledged to me that the same is their own free act and deed.

WITNESS MY HAND AND SEAL on this ___ day of _____, 2025.

APR 06 2026

Doc No. 270
Page No. 60
Book No. 14
Series of 20

ATTY. DAN RAFAEL HAIN PALIS
Notary Public
Notary Public for and in the City of Cabuyao
F.B. Ballon St., Brgy. Sala, Cabuyao, Laguna
Not. Comm. No. 03-C-2024-NC until 12/31-2026
Roll No. 76653 | IBP No. INV 569688
PTR No. 2509906, 01-05-2026, Cabuyao, Laguna
MCLE Compliance No. VIII-0026445 until 04-14-2028

APPENDIX B

Short Report of Plagiarism Software

turnitin Page 2 of 67 - Integrity Overview Submission ID trn:oid:::29324:134183373

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- **118** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **16** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 9% Internet sources
- 6% Publications
- 9% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin Page 2 of 60 - AI Writing Overview Submission ID trn:oid:::29324:134183373

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

APPENDIX C

Report of Language Software

Report: CSA6 - REVISED MANUSCRIPT

CSA6 - REVISED MANUSCRIPT

by mia

General metrics

94,566	14,428	777	57 min 42 sec	1 hr 50 min
characters	words	sentences	reading time	speaking time

Writing Issues

No issues found

Plagiarism

This text hasn't been checked for plagiarism

APPENDIX D

Bionotes of Student Researchers

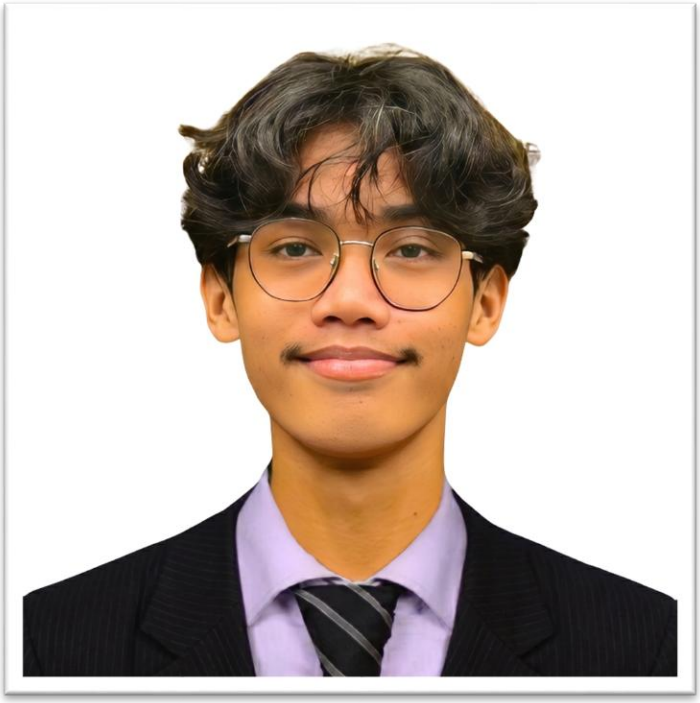


Christian V. Balinado is an undergraduate student currently pursuing a Bachelor of Science in Computer Science at the University of Cabuyao. He has developed foundational knowledge in Computer Programming, Web Technologies, and Technopreneurship, with academic interests focused on artificial intelligence, software engineering, resource-efficient systems, and technology-driven innovation.

He has participated in academic projects and coursework involving system development, startup innovation, case studies, and research activities, which contributed to the development of his skills in software design, technical documentation, problem-solving, leadership, and project management.

For the current study, he serves as a lead researcher, responsible for system conceptualization, data gathering, technical analysis, documentation, and overall research coordination. He has undergone basic orientation or training in research ethics and data privacy, ensuring compliance with institutional and regulatory standards.

Christian V. Balinado affirms that the information provided herein is accurate and limited to what is necessary for research evaluation purposes, in compliance with the Data Privacy Act of 2012.



Andrew D. Dejito is an undergraduate student currently pursuing a Bachelor of Science in Computer Science at the Pamantasan ng Cabuyao. He has developed foundational knowledge in Software Engineering, and Application Development, with academic interests focused on Artificial Intelligence, specifically multi-agent orchestration and Small Language Model (SLM) optimization.

He has participated in academic projects and coursework involving full-stack web development and the design of AI-driven systems for code refactoring, which contributed to the development of his skills in technical documentation, system integration, and prompt engineering.

For the current study, he serves as a primary researcher and documentalist, responsible for curating the Review of Related Literature, structuring the technical manuscript, and managing comprehensive research documentation. He has undergone basic orientation or training in research ethics and data privacy, ensuring compliance with institutional and regulatory standards.

Andrew D. Dejito affirms that the information provided herein is accurate and limited to what is necessary for research evaluation purposes, in compliance with the Data Privacy Act of 2012.



Joshua P. Lopez is an undergraduate student currently pursuing a degree in Bachelor of Science in Computer Science at the University. He has developed foundational knowledge in Software Engineering, Artificial Intelligence and Data Analysis, with academic interests focused on Data Engineering and Cloud Computing.

He has participated in academic projects and coursework involving web and application development, systems and architecture designing, artificial intelligence and multi-agent systems, which contributed to the development of his skills in AI Engineering.

For the current study, he serves as lead programmer, responsible for the planning, designing, implementation and testing of the system. He has undergone basic orientation or training in research ethics and data privacy, ensuring compliance with institutional and regulatory standards.

Joshua P. Lopez affirms that the information provided herein is accurate and limited to what is necessary for research evaluation purposes, in compliance with the Data Privacy Act of 2012.



Jericho F. Varde is an undergraduate student currently pursuing a Bachelor of Science in Computer Science at the University of Cabuyao. He has developed foundational knowledge in Software Engineering, Cloud Computing, Web Development, and UI/UX design, with academic interests focused on scalable web applications, cloud-based systems, and innovative software solutions.

He has participated in academic projects and coursework involving web application development, software design, technical problem-solving, and case-based academic initiatives, which contributed to the development of his skills in system development, analytical thinking, programming, collaboration, and technical communication.

For the current study, he serves as a researcher and technical contributor, responsible for system development support, data gathering, technical analysis, and documentation. He has undergone basic orientation or training in research ethics and data privacy, ensuring compliance with institutional and regulatory standards.

Jericho F. Varde affirms that the information provided herein is accurate and limited to what is necessary for research evaluation purposes, in compliance with the Data Privacy Act of 2012.